



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校	浙江大学
--------	------

参赛队号	24103350007
------	-------------

队员姓名	1.	许诺
	2.	孟旭瑶
	3.	刘相万

中国研究生创新实践系列大赛

“华为杯”第二十一届中国研究生

数学建模竞赛

题 目： 风电场有功功率优化分配

摘 要：

风机低疲劳运行能够明显降低机械系统故障率，为风电场运营商带来极大的经济和社会效益。基于传统平均分配方法的方案中，未考虑任何关于风机运行状态和风机累积疲劳损伤之间的关系，易致使部分风机长期处于疲劳受损的高负荷状态，损害其内部的机械部件和运行安全，进而影响其故障率和使用寿命。本文通过建立风机疲劳损伤程度量化指标计算模型、提出风机元件应力估算方法，建立了风机运行状态、功率参考值与风机元件疲劳损伤程度之间的关系，进而构建了以最小化风机疲劳损伤程度为目标函数的有功功率优化分配模型，并考虑了噪声和时延的影响，形成了包含降噪、去延时处理的降低风机疲劳损伤程度的优化功率分配方法，并基于滑动时间窗、BiLSTM 拟合、NSGA-II 算法等理论方法的应用，实现了有功功率优化分配的实时计算。

针对问题一，首先，基于 **Palmgren-Miner 疲劳损伤理论**和**四点式雨流计数法**建立风机的主轴和塔架的累积疲劳损伤程度量化指标计算的数学模型；随后，提出一种改进的可用于实时计算的 **TW-Rainflow-Goodman-Miner 累积疲劳损伤程度量化指标计算方案**，并与 **Rainflow-Miner**、**Rainflow-Goodman-Miner** 两种方案在计算时间、误差率等方面进行对比，验证了所提出的加入**滑动时间窗**的 **TW-Rainflow-Goodman-Miner 累积疲劳损伤程度量化指标计算方法**，可以实现根据单台风机的主轴扭矩、塔架推力到风机疲劳损伤程度的快速计算，且计算结果相似度良好，满足风机主轴和塔架任意时段累积疲劳损伤量化指标的实时计算的速度和精度要求。

针对问题二，首先，根据风机的风速模型和空气动力学原理，明确风机风速、输出功率与主轴扭矩和塔架推力之间的物理计算公式，建立风机的主轴扭矩和塔架推力的数学计算模型；其次，基于风电场 WF1 前 70% 的输入输出数据构建训练集，采用 **Nelder-Mead 单纯形算法**、**BiLSTM 拟合方法**描述桨距角、风速、功率参考值等输入参数与主轴扭矩和塔架推力之间的非线性映射关系；随后，基于训练好的非线性映射关系，对风电场 WF1 后 30% 的输入数据进行元件应力估算，估算该风电场每个时刻的所有风机的主轴扭矩和塔架应力；最后，对比两种拟合方法的应用效果，验证了使用 **BiLSTM 模型**的拟合效果更好；将基于 **BiLSTM** 数据拟合得到的估算值与参考值进行对比，验证了所提方法能够实现对于风机的主轴扭矩和塔架推力的**高精度估算**。

针对问题三，首先，在问题一和问题二的基础上构建了**风机累积疲劳损伤程度计算模型**，实现对每个运行周期内风电场内所有风机的累积疲劳损伤程度计算；随后，以当前运行周期内所有风机的主轴和塔架的总体累积疲劳损伤程度最小为目标函数，考虑功率平衡、风机出力和调节精度等约束条件，建立**考虑疲劳损伤程度的风机有功功率优化分配模**

型；然后，通过 **NSGA-II 算法** 进行模型求解，实现风电场内各风机有功功率的优化调度和实时更新，保证各个风机的疲劳损伤程度和功率分配的合理性；最后，验证了所求功率优化分配方案可以满足计算实时性和约束条件，将所求功率优化分配方案的运行效果与平均功率分配方案进行对比，验证建立的优化模型和模型求解方法的有效性。

针对问题四，首先，将噪声干扰和通信延迟作用下的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将计算结果与问题三的优化结果进行对比，分析噪声干扰和通信延迟为计算精度和优化效果带来的影响；随后，针对噪声和延迟问题设计优化方案，对采集数据进行合理的**预处理**，其中，针对噪声干扰问题，采用**奇异值分解 SVD** 进行降噪处理，针对通信延迟问题，基于**相位差**进行**时间序列滞后性判断**判断、筛选，并基于**改进三次样条插值法**进行缺失数据恢复；最后，将经过预处理后的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将得到的计算结果与基于未处理的采集数据的计算结果、基于无噪声和延迟的采集数据的计算结果进行对比，验证基于滤波和预测的预处理环节对于消除噪声干扰和通信延迟为计算精度和优化效果带来的影响的有效性。

关键词： 风机疲劳损伤程度；雨流计数法；滑动时间窗；BiLSTM 拟合；
NSGA-II 算法；噪声干扰；通信延迟；SVD 分解；改进三次样条插值

目 录

摘 要:	1
1. 问题背景与问题重述	5
1.1. 问题背景	5
1.2. 问题重述	5
1.3. 问题分析	5
2. 模型假设与符号说明	8
2.1. 模型假设	8
2.2. 符号说明	8
3. 问题一的模型建立与求解	9
3.1. 问题分析	9
3.2. 应用方法的基本概念	9
3.2.1. Palmgren-Miner疲劳损伤理论	9
3.2.2. 雨流计数法	10
3.2.3. Goodman曲线矫正	10
3.2.4. 滑动时间窗	11
3.3. 改进的累积疲劳损伤程度量化指标计算方法	11
3.3.1. 三种计算方案	11
3.3.2. 基于滑动时间窗的累积疲劳损伤程度量化指标计算方法	12
3.3.3. 计算性能分析方法	13
3.4. 结果对比分析	14
3.4.1. 计算时间验证	14
3.4.2. 与雨流计数法所得结果的相似程度	14
3.4.3. 风机累积疲劳损伤程度的增长过程	17
3.5. 问题小结	18
4. 问题二的模型建立与求解	19
4.1. 问题分析	19
4.2. 主轴扭矩和塔架推力计算模型	19
4.3. 基于数据拟合的主轴扭矩和塔架推力估算方法	20
4.3.1. Nelder-Mead拟合原理	20
4.3.2. BiLSTM拟合原理	21
4.3.3. 主轴扭矩和塔架推力估算方法	22
4.3.4. 误差评价方法	22
4.4. 结果分析	23
4.4.1. 估算说明	23
4.4.2. 基于BiLSTM拟合的估算结果	24
4.4.3. 基于Nelder-Mead拟合的估算结果	26
4.4.4. 基于两种拟合方法的估算值与参考值对比	27
4.5. 问题小结	28
5. 问题三的模型建立与求解	29
5.1. 问题分析	29
5.2. 基于功率参考值的风机累积疲劳损伤程度计算模型	29
5.3. 考虑疲劳损伤的风机有功功率优化分配模型	30
5.3.1. 目标函数	30

5.3.2. 约束条件	30
5.4. 基于NSGA-II算法的模型求解方法	31
5.5. 优化结果分析	32
5.5.1. 计算实时性展示	32
5.5.2. 优化模型约束条件满足情况	32
5.5.3. 优化结果与平均分配功率结果对比	33
5.6. 问题小结	36
6. 问题四的模型建立与求解	38
6.1. 问题分析	38
6.2. 考虑噪声干扰和通信延迟的风机有功功率优化与求解	38
6.2.1. 基于奇异值分解SVD的噪声干扰处理	38
6.2.2. 基于相位差和改进三次样条插值法的通信时延处理	39
6.2.3. 考虑噪声干扰和通信延迟的风机有功功率优化方法	40
6.3. 结果分析	41
6.3.1. 计算结果实时性展示	41
6.3.2. 基于奇异值分解SVD的降噪效果	42
6.3.3. 基于相位差和改进三次样条插值法的去延迟效果	44
6.3.4. 噪声干扰与通信延迟对功率分配优化结果的影响分析	46
6.3.5. 降噪和去延迟前后的功率分配优化结果对比	48
6.3.6. 模型鲁棒性检验	51
6.4. 问题小结	55
7. 模型总结	56
7.1. 模型优点	56
7.2. 模型缺点与改进方向	56
参考文献	57
附录	58

1. 问题背景与问题重述

1.1. 问题背景

我国风电快速发展，大型风机、大规模场站逐步投入运行。额定容量高的大型风机机械部件柔性更强，导致其疲劳损伤累积速度快，增加风机维护成本，降低风力发电效率；极端情况甚至会加剧风机倒塌、叶片断裂等安全风险。因此，亟需通过优化手段，降低风机运行过程中的累积疲劳损伤，以减小其因疲劳导致可用寿命缩短的风险。风机低疲劳运行能够明显降低机械系统故障率，为风电场运营商带来极大的经济效益：一方面，减少故障导致不正常停机所造成的发电量损失；另一方面，减少机械部件维护频次以节约运维人员和物料成本，且降低安全生产事故风险。

风电场运行过程中，电网调度指令 P_t 下发到场站控制端。场站自动发电控制系统（Automatic Generation Control, AGC）将以每秒钟一次的采样率响应调度指令 P_t 。AGC根据电网调度指令 P_t 计算场内每台风力发电机所需发电量，并将计算结果发送给各个风机（此过程即为有功功率分配过程，各个风机所分配的功率为其功率参考值），各风机依据功率参考值调整自身机械参数，实现发电量对功率参考值的跟踪（本题默认跟踪过程可以精确实现）。传统计算分配方案采用平均分配方法，即每台风机需发出功率 $P_{ref}^i = P_t / W_t$ ， W_t 为场内所有风机数量。每台风机的功率指令 P_{ref}^i 同时受限于由风速决定的当前可发出最大功率值。然而，平均分配方法未考虑任何关于风机运行状态和风机累积疲劳损伤之间的联系，导致部分风机长期处于疲劳的高负荷状态，进而影响其机械部件的运行安全。

1.2. 问题重述

基于上述问题背景，本题目提供了疲劳评估数据、风电机组采集数据、噪声和延迟作用下的采集数据等三项数据，提供了风机疲劳损伤建模的参考方法，根据问题描述、参考资料与提供的附件数据，本文需要研究解决以下问题：

问题一：风机主轴及塔架疲劳损伤程度量化指标计算低复杂度模型

针对目前常用雨流计数法统计不同幅值载荷相应的循环次数、但无法在线实时求解的问题，建立数学模型，实现对风机的主轴和塔架在任意时段累积疲劳损伤程度量化指标的实时计算。

问题二：利用风速及功率估算塔架推力和主轴扭矩

建立数学模型，根据风机所处位置的风速条件和功率参考值，估算当前风机所承受的应力/扭矩；模型可结合受力分析、能量守恒或其他任意合理思路进行建立。

问题三：有功调度优化问题构建与实时求解

为降低风电场运维成本，需将主轴和塔架的两种元件的疲劳损伤定义为两个目标，考虑功率平衡、风机出力和调节精度等约束条件，建立优化模型求解最优有功功率分配策略。由于大规模风电场含数百台风机，优化问题维度高，因此需在建模阶段兼顾模型复杂度和精度。

问题四：考虑通信延迟和测量噪声的有功功率优化与求解

实际风电场中，数据测量噪声和传输延迟导致理想条件下的优化问题难以满足实际需求。因此，可进一步考虑随机测量噪声和通信延迟对于模型精度和优化问题最优性的影响，完善问题三中的优化方案。

1.3. 问题分析

基于传统平均分配方法的方案中，风电场内所有风机的功率分配权重相同，该方法并未考虑任何关于风机运行状态和风机累积疲劳损伤之间的关系，易致使部分风机长期处于疲劳受损的高负荷状态，损害其内部的机械部件和运行安全，进而影响其故障率和使用寿命。

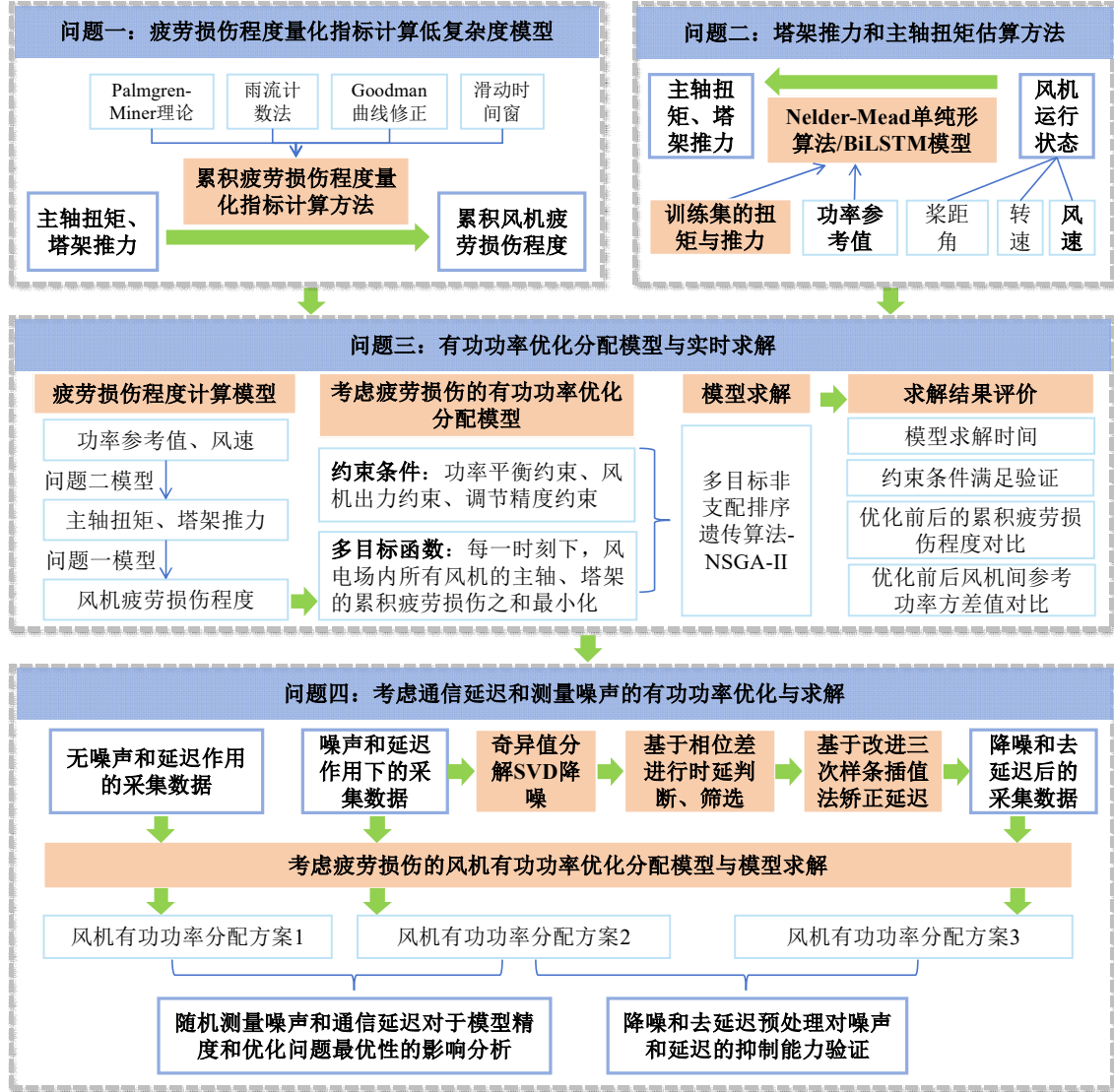


图1 风电场有功功率优化调度总体方案图

针对此问题，如图1所示，按照题目要求，按照四个子问题进行分步解决：

(1) 问题一要求建立风机主轴及塔架疲劳损伤程度量化指标计算低复杂度模型。本文针对目前常用的三点式雨流计数法无法在线实时求解的问题，首先，基于Palmgren-Miner疲劳损伤理论和四点式雨流计数法建立风机的主轴和塔架的累积疲劳损伤程度量化指标计算的数学模型；随后，提出一种改进的可用于实时计算的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方案，并与Rainflow-Miner、Rainflow-Goodman-Miner两种方案在计算时间、误差率等方面进行对比，验证了所提出的加入滑动时间窗的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方法，可以实现根据单台风机的扭矩、塔架推力到风机疲劳损伤程度的快速计算，且计算结果相似度良好，满足风机主轴和塔架任意时段累积疲劳损伤量化指标的实时计算的速度和精度要求。

(2) 问题二要求利用风速及功率估算塔架推力和主轴扭矩。为实现风机运行状态、功率参考值与主轴扭矩、塔架推力之间的高精度估算，本文首先根据风机的风速模型和空气动力学原理，明确风机风速、输出功率与主轴扭矩和塔架应力之间的物理计算公式；随后，构建数据训练集，并采用多种拟合方法训练风速、功率参考值与主轴扭矩、塔架推力之间的非线性映射关系；最后，对比拟合方法的应用效果，选取更为优势的拟合方法，将估算结果与参考值进行比较，验证所建模型和所提方法的有效性。

(3) 问题三要求构建风机有功调度优化模型并实时求解。本文针对此问题，在问题一和问题二的基础上，首先，构建风机疲劳损伤程度计算模型，在每个运行周期内基于风速和功率参考值实时计算风电场内所有风机的累积疲劳损伤程度；随后，以当前运行周期内所有风机的主轴和塔架的总体累积疲劳损伤程度最小为目标函数，考虑功率平衡、风机出力和调节精度等约束条件，建立风电场有功功率优化分配模型；然后，通过多目标非支配排序遗传算法进行模型求解，实现风电场内各风机有功功率的优化调度和实时更新，保证各个风机的疲劳损伤程度和功率分配的合理性；最后，将所求功率优化分配方案的运行效果与平均功率分配方案进行对比，验证建立的优化模型和模型求解方法的有效性。

(4) 问题四要求实现考虑通信延迟和测量噪声的有功功率优化与求解。本文针对此问题，首先，将噪声干扰和通信延迟作用下的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将计算结果与问题三的优化结果进行对比，分析噪声干扰和通信延迟为计算精度和优化效果带来的影响；随后，针对噪声和延迟问题设计优化方案，对采集数据进行合理的预处理，其中，针对噪声干扰问题，采用奇异值分解SVD进行降噪处理，针对通信延迟问题，基于相位差进行时间序列的滞后性判断、筛选，并基于改进三次样条插值法进行缺失数据恢复；最后，将经过预处理后的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将得到的计算结果与基于未处理的采集数据的计算结果、基于无噪声和延迟的采集数据的计算结果进行对比，验证基于降噪和去延迟的预处理环节对于消除噪声干扰和通信延迟为计算精度和优化效果带来的影响的有效性。

2. 模型假设与符号说明

2.1. 模型假设

在本文的建模过程中，在不影响模型意义与输出精度的前提下，为使得模型简单、明确，本文做出如下假设：

（1）假设不同幅值载荷的风机的主轴和塔架的疲劳损伤是独立的，且两个元件的疲劳损伤都可以使用S-N曲线进行评估；

（2）假设所有风机在起始时间时的应力循环和累积疲劳损伤值为0；

（3）假设风机依据功率参考值调整自身机械参数实现对发电功率参考值的跟踪过程可以精确实现且没有时延。

2.2. 符号说明

本文中用到的变量符号及其说明如下表所示：

表1 变量符号及其说明

序号	符号	符号说明
1	n_F	风机应力循环次数
2	D_F	风机塔架的累积疲劳损伤值
3	D_T	风机主轴的累积疲劳损伤值
4	S_i	应力幅值
5	L_i	经过Goodman修正的各级载荷幅值
6	N_F	承受应力的循环次数极限
7	V_w^i	风机 i 的轮毂处等效风速
8	P_{ref}^i	风机 i 的有功功率参考值
9	T_s^i	风机 i 的轴系扭矩
10	F_t^i	风机 i 的塔顶推力
11	P_m^i	风机 i 的机械功率
12	β_i	风机 i 的桨距角
13	$D_{T_total}^i$	风机 i 的主轴累积疲劳损伤程度量化指标
14	$D_{F_total}^i$	风机 i 的塔架累积疲劳损伤程度量化指标
15	P_t	电网下达给风电场的调度指令要求功率
16	W_t	风电场风机的数量

3. 问题一的模型建立与求解

3.1. 问题分析

问题一要求针对目前常用雨流计数法统计不同幅值载荷相应的循环次数、但无法在线实时求解的问题，建立数学模型，实现对风机的主轴和塔架在任意时段累积疲劳损伤程度量化指标的实时计算。

针对问题一，整体思路流程图如下图2所示。首先，结合Palmgren-Miner疲劳损伤理论、雨流计数法、Goodman曲线矫正和滑动时间窗（TW）理论方法，提出一种改进的可用于实时计算的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方案；随后，建立了风机主轴和塔架的累积疲劳损伤程度量化指标计算的数学模型，给出了TW-Rainflow-Goodman-Miner方案的详细实施步骤；最后，与Rainflow-Miner、Rainflow-Goodman-Miner两种方案在计算时间、误差率等方面进行对比，验证所提出的加入滑动时间窗的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方法，可以实现根据单台风机的主轴扭矩、塔架推力到风机疲劳损伤程度的快速计算，且计算结果相似度良好，满足风机主轴和塔架任意时段累积疲劳损伤量化指标的实时计算的速度和精度要求。



图2 问题一思路流程图

3.2. 应用方法的基本概念

3.2.1. Palmgren-Miner疲劳损伤理论

Palmgren-Miner线性累积损伤理论：元件的疲劳损伤是可以累积的，而不同应力水平下的疲劳损伤可以通过累积的循环次数来表示。一旦累积损伤值达到临界值，材料就会发生疲劳失效。其中，常用的描述方法为S-N曲线法和能量法。以S-N曲线法为例，S-N曲线法是用来描述材料的各个应力幅值与其最大循环载荷次数之间关系的曲线，可利用材料的S-N曲线进行疲劳寿命预测，S-N曲线有多种表示形式，基于幂函数的表示形式如下：

$$S^m \times N = C \sqrt{\frac{\sum L_i^m n_i}{N}} \quad (1)$$

式中， S 为各个载荷幅值， N 为对应载荷的最大循环载荷次数， m 为Wohler指数（本题取10）， C 为常数（本题取 9.77×10^{70} ）。

假设在某一应力幅值 F 下，材料的能够承受应力的循环次数极限为 N_F 次（ N_F 通过S-N曲线法获取）。实际已经经历了 n_F 次应力循环（根据定义人工计数），则其累积疲劳损伤值 D_F 可以表示为：

$$D_F = \frac{n_F}{N_F} \quad (2)$$

总损伤累积值 D 为各个应力水平（ $F_1, F_2, \dots, F_n$ ）下的累积损伤值之和 $D = D_{F_1} + D_{F_2} + \dots + D_{F_n}$ 。根据Palmgren-Miner线性累积损伤理论，当总损伤累积值 D 达到1时，材料即发生疲劳失效。

3.2.2. 雨流计数法

雨流计数法（Rainflow）是疲劳计数中最常用的方法，其主要作用是将实测载荷历程简化为多个载荷循环，用于估算疲劳寿命和生成疲劳试验载荷谱。该方法考虑了载荷动态幅值和静态均值这两个变量，符合疲劳载荷的固有特性。雨流法在工程领域被广泛应用，尤其在疲劳寿命计算中运用非常广泛。下图为雨流计数法的实现流程图：

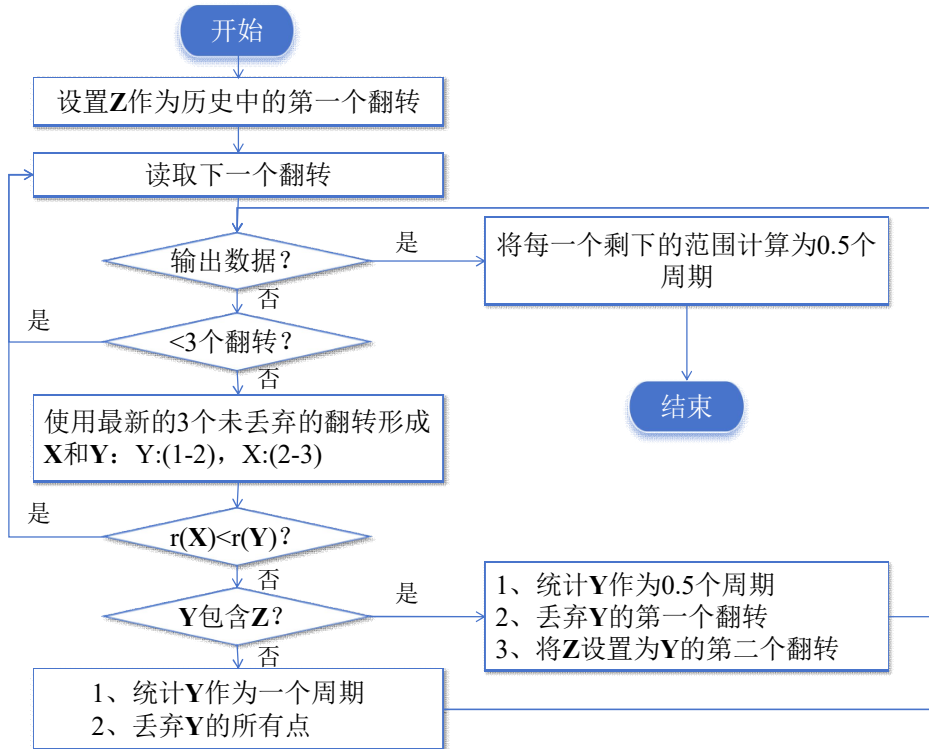


图3 雨流计数法原理图

通过雨流法最终可以得到幅值、均值和循环次数矩阵，当结构的疲劳对均值不敏感时，可以去除均值，使用幅值和循环次数的关系表述载荷分布。在疲劳计算中，为了方便比较和计算，通常采用等效疲劳载荷：

$$L_N = \sqrt[m]{\frac{\sum L_i^m n_i}{N}} \quad (3)$$

式中， L_i 为经过Goodman修正的各级载荷幅值， n_i 为该级载荷的循环次数， m 为S-N曲线的Wohler指数（本题取10）， N 取决于风电风机设计寿命（本题取42565440.4361）， L_N 为等效疲劳载荷。

3.2.3. Goodman曲线矫正

基础的S-N曲线是认为平均载荷为0的情况，那么在平均载荷不为0时，我们就需要对原有的S-N曲线进行修正，常用的办法为Goodman曲线修正，其修正表达式为：

$$1 = \frac{S_{ai}}{S_i} + \frac{S_{mi}}{\sigma_b} \quad (4)$$

式中， S_{ai} 为第一步中计算的第*i*个循环的幅值， S_i 为等效均值为0时的载荷幅值， S_{mi} 为第一步中计算的第*i*个循环的均值， σ_b 为材料在拉伸断裂时的最大载荷值，在本题中主轴和塔架的最大载荷值均为50000000。

3.2.4. 滑动时间窗

滑动窗口算法（Sliding Time Window）是一种常见的双指针算法。其基本思想是维护一个窗口，通过移动窗口（Time Window，简称TW）的两个边界来解决问题。通常使用两个指针left和right来表示窗口的左右边界，右指针right不断移动来扩大窗口，同时根据问题的要求调整左指针left来缩小窗口。当右指针right到达字符串或数组的末尾时，算法执行完毕。在调整窗口大小的过程中，可以记录一些中间结果，比如最大值、最小值、子串长度等，以便最终解决问题^[5]。

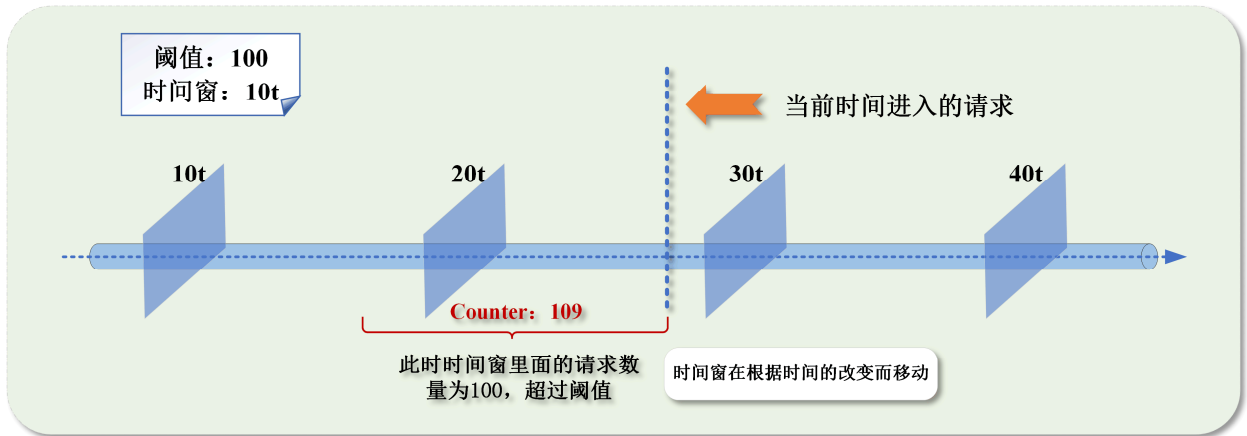


图4 滑动时间窗原理图

滑动窗口算法的实现方法相对简单，主要分为以下几个步骤：

- 1) 初始化左右指针left和right，并根据问题的要求进行一些初始化操作。
- 2) 不断移动右指针right，直到出现不符合条件的情况，或者扫描到字符串或数组的末尾。
- 3) 对于每个右指针位置right，更新一些中间结果。
- 4) 移动左指针left，直到出现符合条件的情况，或者左右指针重合。
- 5) 重复第2步~第4步，直到右指针扫描到字符串或数组的末尾。

3.3. 改进的累积疲劳损伤程度量化指标计算方法

3.3.1. 三种计算方案

结合Palmgren-Miner疲劳累积损伤理论、雨流计数法、Goodman曲线矫正和滑动时间窗等理论方法，提出一种改进的可用于实时计算的TW-Rainflow-Goodman-Miner疲劳损伤程度量化指标计算方案，并与Rainflow-Miner和Rainflow-Goodman-Miner方案进行对比：

- (1) Rainflow-Miner：采用四点式雨流计数法和Palmgren-Miner累积疲劳算法；

(2) Rainflow-Goodman-Miner: 采用四点式雨流计数法、Goodman曲线修正以及Palmgren-Miner累积疲劳算法;

(3) TW-Rainflow-Goodman-Miner: 采用基于滑动时间窗的结合四点式雨流计数法、Goodman曲线修正以及Palmgren-Miner的计算法。

在建模过程中, 对比上述三种方案的累积疲劳损伤程度的计算结果与附件所提供的四点式雨流法的计算结果, 并对比计算时间、误差率等运行指标, 筛选出最佳方案, 用于后续的实时优化。

3.3.2. 基于滑动时间窗的累积疲劳损伤程度量化指标计算方法

针对所提出的TW-Rainflow-Goodman-Miner方案, 进行方法详述和流程梳理。其他两个方案为该方案的步骤简化方案。

(1) 确定应力水平

基于风机 i 的主轴扭矩 T_i 和塔架推力 F_i , 通过模型将其转化为对应的应力幅值 S_i 。假设主轴扭矩与应力成线性关系, 塔架推力与应力幅值亦呈线性关系, 则两个元件的应力幅值与对应扭矩或推力的关系可以表示为:

$$S_i^T = k_T T_i \quad (5)$$

$$S_i^F = k_F F_i \quad (6)$$

式中, k_T 和 k_F 是材料常数, S_i^T 和 S_i^F 分别为风机 i 的主轴或塔架的应力幅值。

随后, 可基于S-N曲线获取主轴和塔架的极限循环次数:

$$N_F^i = f(S_i^T) \quad (7)$$

$$N_F^i = f(S_i^F) \quad (8)$$

式中, $f(\cdot)$ 是通过实验数据得到的S-N曲线的函数形式。

(2) 应力循环计数

由于数据的随机性以及波峰波谷不易辨识, 可基于雨流计数方法识别应力循环。在局部极值的基础上, 记录下应力幅值 S_i^T 、 S_i^F 以及对应的循环次数。

(3) 单台风机的累积疲劳损伤计算

根据Palmgren-Miner理论, 累积疲劳损伤可以通过不同幅值下的疲劳损伤值累加得到:

$$D_i = D_i^T + D_i^F \quad (9)$$

式中, D_i 表示风机 i 的总累积疲劳损伤, D_i^T 和 D_i^F 分别表示风机 i 的主轴和塔架的疲劳损伤。其中, 主轴和塔架的疲劳损伤值可以通过以下公式计算:

$$D_i^T = \sum_{j=1}^n \frac{n_F^i(S_j^T)}{N_F^i(S_j^T)} \quad (10)$$

$$D_i^F = \sum_{j=1}^n \frac{n_F^i(S_j^F)}{N_F^i(S_j^F)} \quad (11)$$

式中, $n_F^i(S_j^T)$ 和 $n_F^i(S_j^F)$ 分别是风机 i 的主轴和塔架在不同应力水平条件下进行的循环次数; $N_F^i(S_j^T)$ 和 $N_F^i(S_j^F)$ 分别是风机 i 的主轴和塔架在S-N曲线对应的在应力幅值下的最大承载循环次数。

(4) 累积疲劳损伤的实时计算

为了满足实时计算的要求, 可以简化累积计算步骤, 在每周周期数据输入后进行更新:

$$D_i^{total} = D_i^{pre} + D_i \quad (12)$$

式中， D_i^{pre} 表示风机 i 在上一个时刻的累积损伤值。

实时计算过程的流程图如下图5所示。首先，将100台风机的的主轴扭矩和塔架推力组成时间序列；其次，顺序对100台风机执行以下步骤：

- 1) 滑动读取时间窗内的时间序列，提取风机的主轴扭矩和塔架应力数据；
- 2) 提取当前时间窗内风机的主轴和塔架的应力循环幅值，使用四点式雨流法统计对应的应力循环次数；
- 3) 根据Goodman修正后的S-N曲线数据，计算当前时间窗内风机的主轴和塔架的疲劳损伤值；
- 4) 基于上一时间窗和当前时间窗的疲劳损伤值，累加应力幅值的疲劳损伤值，计算累积疲劳损伤值和等效疲劳荷载，获得当前时间窗的累积疲劳损伤量化指标；

最后，计算得到100台风机每一时刻的累积疲劳损伤值，记录在附件5中，并分析其变化情况，实现图表输出。

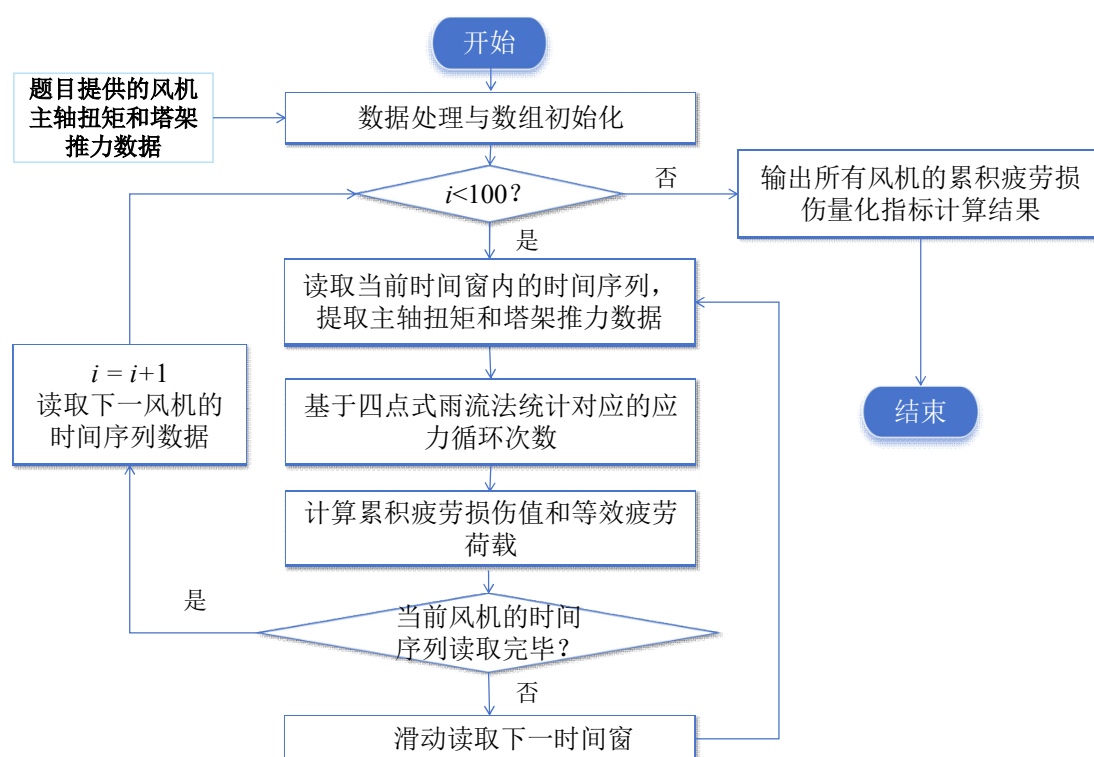


图5 TW-Rainflow-Goodman-Miner方案的程序流程图

3.3.3. 计算性能分析方法

(1) 算法复杂度计算方法

算法的时间复杂度是总运算次数表达式中受 n 的变化影响最大的那一项（不含系数），一个算法中的语句执行次数称为语句频度或时间频度，记为 $T(n)$ 。

一般情况下，算法的基本操作重复执行的次数是模块 n 的某一个函数 $f(n)$ ，因此，算法的时间复杂度记作： $T(n) = O(f(n))$ 。随着模块 n 的增大，算法执行的的时间的增长率和 $f(n)$ 的增长率成正比，即 $f(n)$ 越小，算法的时间复杂度越低，算法的效率越高。

(2) MATLAB实时计算时间统计方法

程序块开始前：tic;

程序块结束后：toc;

此时已经记录好所用时间在toc，运行完会自动输出

(3) 相似性量化指标

为将计算结果与基于雨流计数法的结果进行比较，可采用均方根误差（RMSE）和平均绝对百分比误差（MAPE）来量化相似性：

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^N \left(D_k^{cal} - D_k^{rainflow} \right)^2} \quad (13)$$

$$MAPE = \frac{1}{N} \left(\sum_{k=1}^N \left(\frac{D_k^{cal} - D_k^{rainflow}}{D_k^{rainflow}} \right) \times 100\% \right) \quad (14)$$

RMSE的取值范围为 $[0, +\infty)$ ，当预测值与真实值完全吻合时等于0，即完美模型；误差越大，该值越大。MAPE取值范围为 $[0, +\infty)$ ，MAPE为0%表示完美模型，MAPE大于100%则表示劣质模型。

3.4. 结果对比分析

基于MATLAB进行程序编写和执行，对程序的执行时间进行记录，此外对比所提出的三种方案，验证了所提出的基于滑动时间窗的疲劳损伤程度量化指标计算方法可较为精确地贴合雨流计数法的计算结果，且计算时间满足题目要求，验证了该方法在计算效果和计算性能方面的有效性。

3.4.1. 计算时间验证

如图6所示，在MATLAB中使用计时器计算程序对程序的运行时间进行记录，程序运行环境基于CPU计算，算法求解时间约0.009s，计算时间在1.00s以内，满足题目要求。

```

1 max_ = 50000000;
2 N_f = 42565440.4361;
3 m = 10;
4 C = 9.77*10^70;
5
6 D = [];
7 L = [];
8 tic
9
10 for i = 1:100
11     data = A(:,i);
12     [D(i), L(i)] = damage(data, max_, N_f, m, C);
13 end
14 time_to = toc
15
16

```

```

>>
max_ = 50000000;
N_f = 42565440.4361;
m = 10;
C = 9.77*10^70;

D = [];
L = [];
tic
for i = 1:100
    data = A(:,i);
    [D(i), L(i)] = damage(data, max_, N_f, m, C);
end
time_to = toc

time_to =
0.008968000000000
fx >>

```

图6 计时器时间<1s验证图

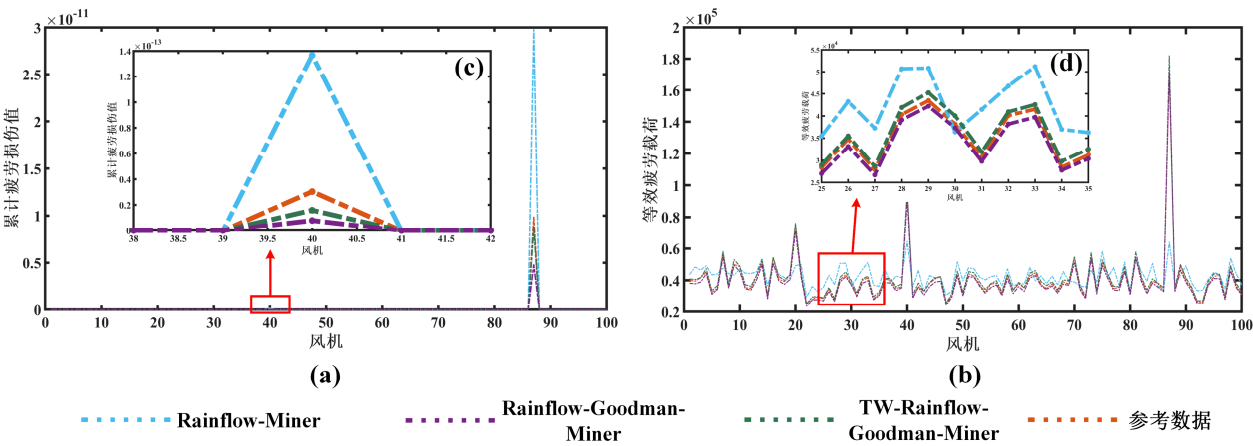
3.4.2. 与雨流计数法所得结果的相似程度

对比3.3节三种计算方案的计算结果与题目给出的三点式雨流计数法计算结果。将所提方法程序执行的计算结果中包括100s时长内所有100台风机的全部200件元件的每秒疲劳

损伤值列入附件5表格（在报告中不再重复给出），在本报告中用图片展示基于本文提出的累积疲劳损伤量化指标计算方法得到的结果与参考值的相似程度。

（1）主轴累积疲劳损伤程度计算结果与参考值贴合程度

图7展示了从0-100s内风机的主轴的累积疲劳损伤程度，对比了四种方案的累积疲劳损伤程度计算值与参考值，可以说明所提出的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方法相较于其他方案在数据贴合度方面更具优势。



(a)100台风机主轴的累积疲劳损伤值图 (b)100台风机主轴的等效疲劳载荷主轴扭矩
(c)风机主轴累积疲劳损伤值局部放大图 (d)风机主轴等效疲劳载荷局部放大图

图7 主轴的累积疲劳损伤程度

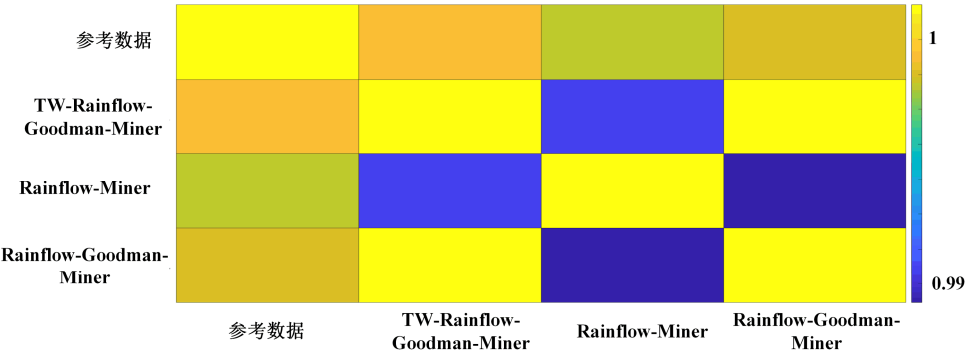


图8 主轴的累积疲劳损伤值的相关性分析热度图

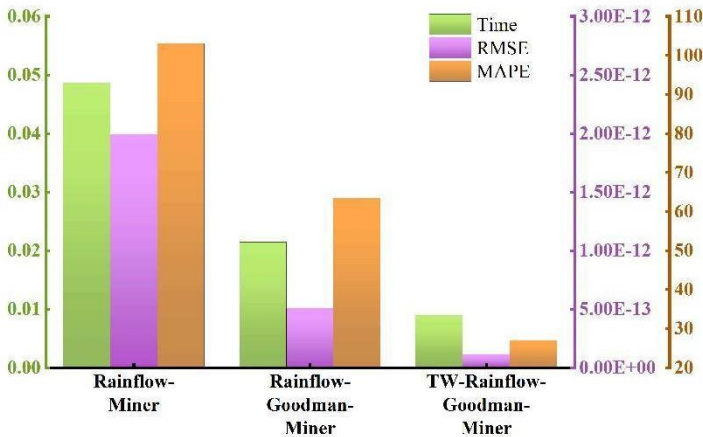


图9 主轴的累积疲劳损伤值的评价指标对比

图8是四种计算方案下主轴的累积疲劳损伤值的相关性分析的热度图的展示形式，可以看到，TW-Rainflow-Goodman-Miner与参考数据的相似度更高，计算效果更接近。

图9是四种计算方案下主轴的累积疲劳损伤值的运行性能和数据贴合程度评价指标，分别是时间、RMSE（均方根误差）、MAPE（平均绝对百分比误差），可以看到TW-Rainflow-Goodman-Miner在三种评价误差中都表现更佳。

（2）塔架累积疲劳损伤程度的增长过程

图10展示了从0-100s内风机的塔架的累积疲劳损伤程度，对比了四种方案的累积疲劳损伤程度计算值与参考值，可以说明所提出的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方法相较于其他方案在数据贴合度方面更具优势。

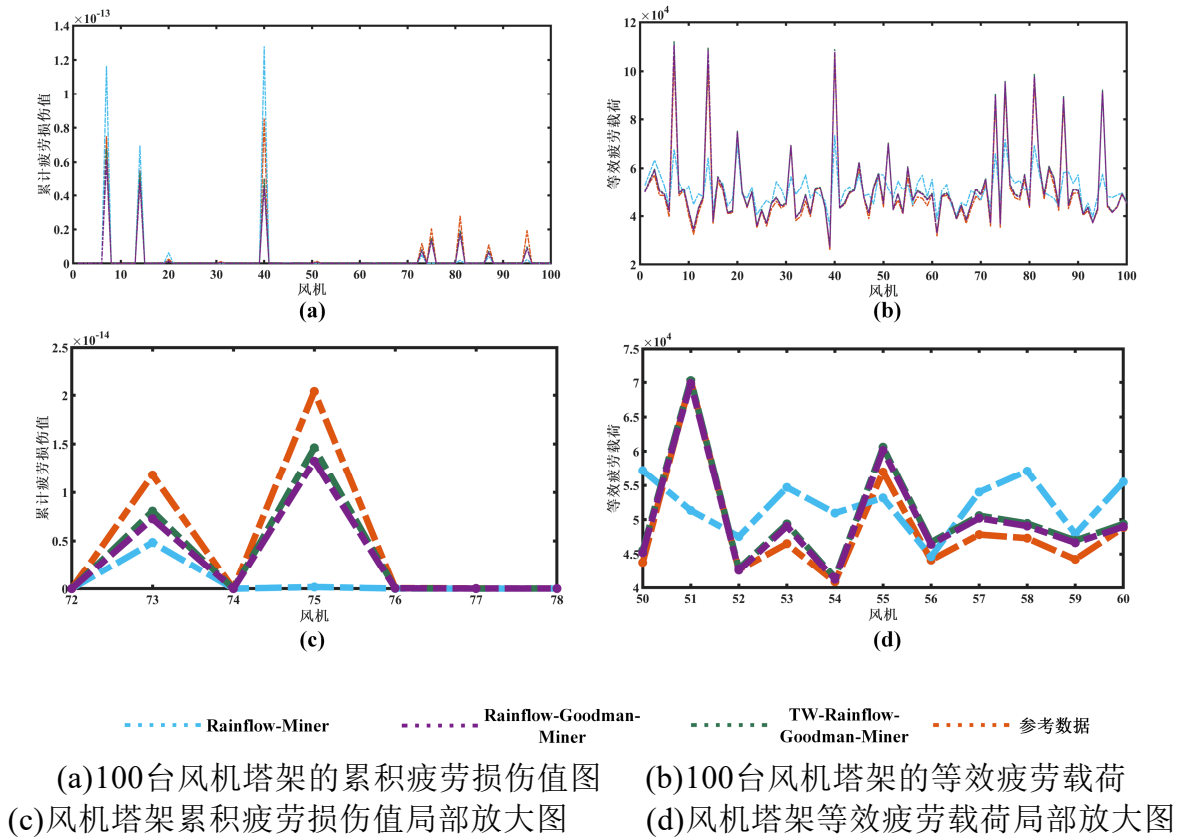


图10 塔架的累积疲劳损伤程度

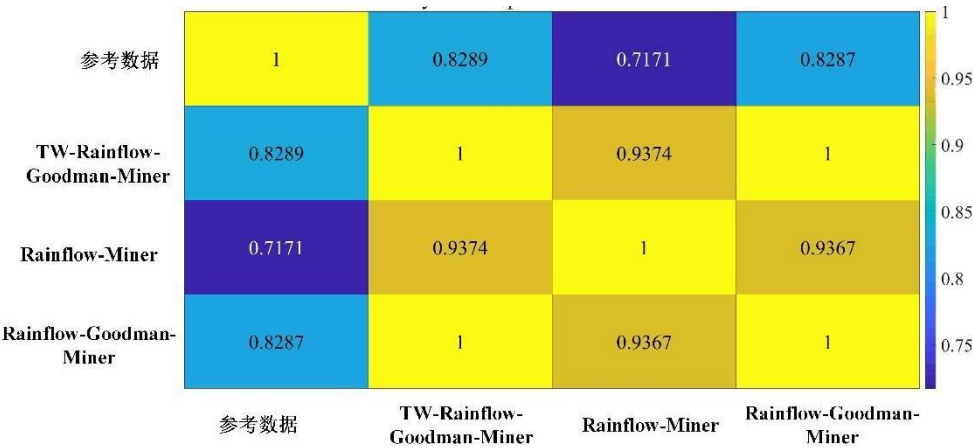


图11 塔架的累积疲劳损伤值的相关性分析热度图

图11是四种计算方案下塔架的累积疲劳损伤值的相关性分析的热度图的展示形式，可以看到，TW-Rainflow-Goodman-Miner与参考数据的相似度更高，计算效果更接近。图12四种计算方案下塔架的累积疲劳损伤值的运行性能和数据贴合程度评价指标，分别是时间、RMSE（均方根误差）、MAPE（平均绝对百分比误差），可以看到TW-Rainflow-Goodman-Miner在三种评价误差中都表现更佳。

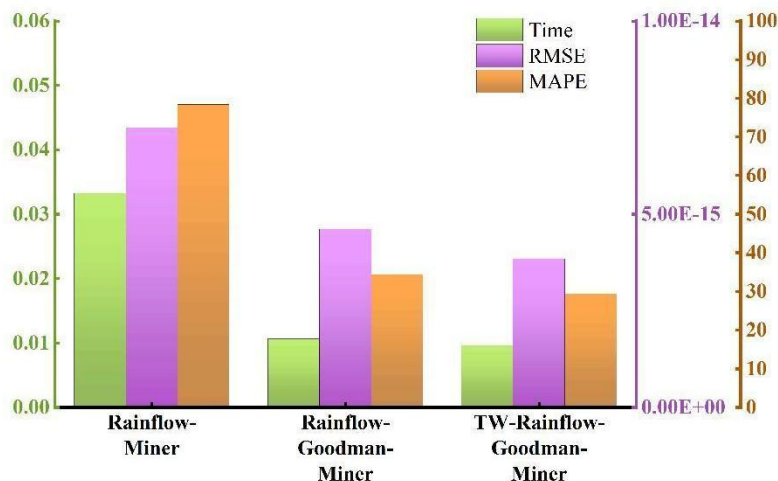


图12 塔架的累积疲劳损伤值的评价指标对比

3.4.3. 风机累积疲劳损伤程度的增长过程

图13和图14分别为10个风机（编号10、20、30、…、100）的主轴和塔架的累积疲劳损失程度量化指标在100s内的增长过程，验证了所提出的建模方法对100台风机的所有数据样本均有效。

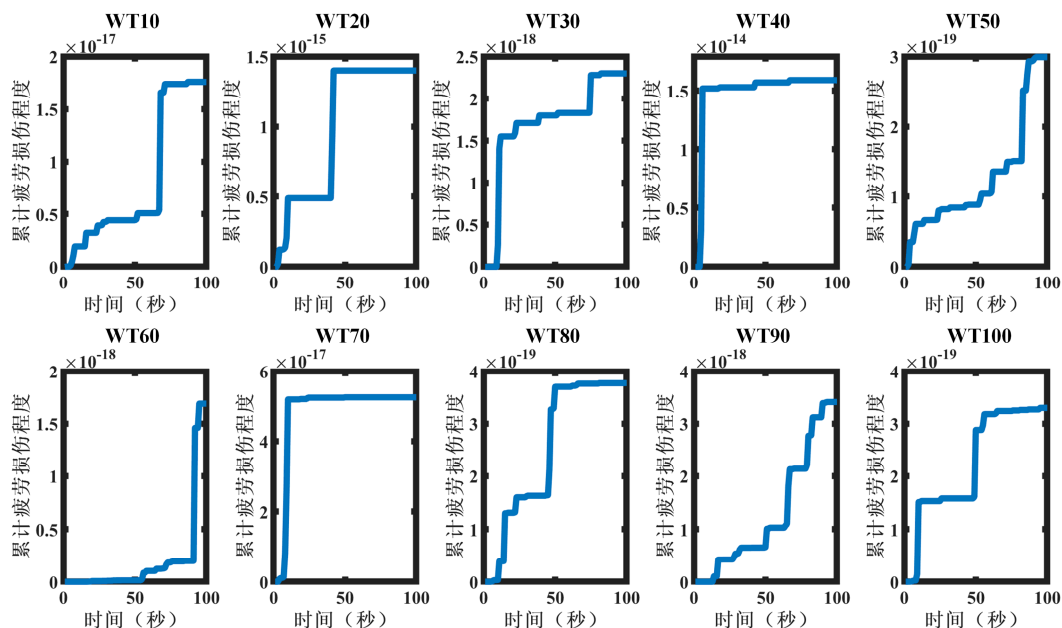


图13 10台风机的主轴累积疲劳损失在100s内的增长过程

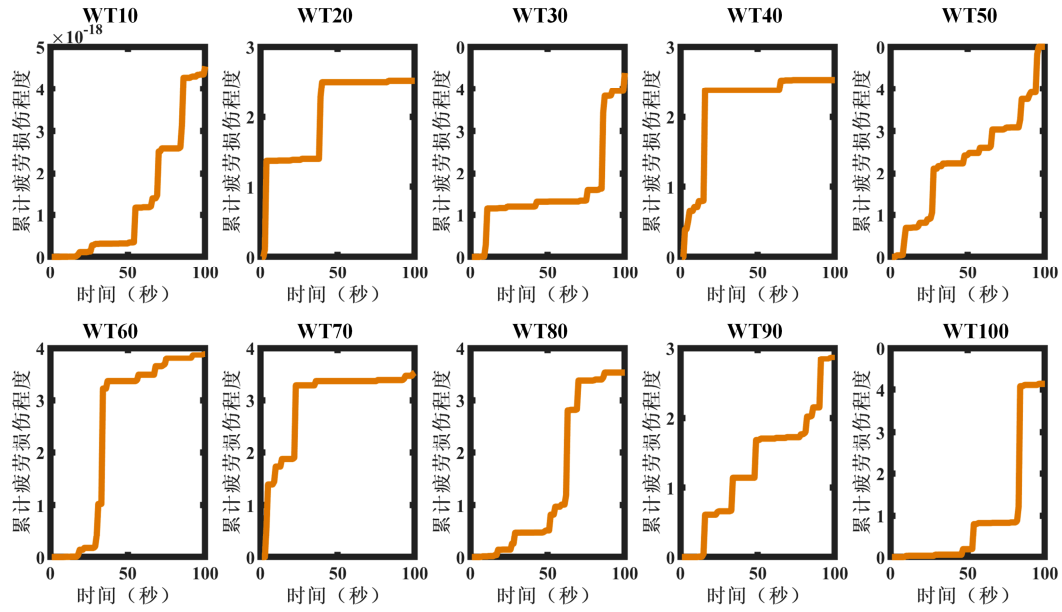


图14 10台风机的塔架累积疲劳损失在100s内的增长过程

3.5. 问题小结

问题一要求克服目前常用的雨流计数法无法在线实时求解不同幅值载荷相应循环次数的问题。本文针对此问题，首先，基于Palmgren-Miner疲劳损伤理论和四点式雨流计数法建立风机的主轴和塔架的累积疲劳损伤程度量化指标计算的数学模型；随后，提出一种改进的可用于实时计算的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方案，并与Rainflow-Miner、Rainflow-Goodman-Miner两种方案在计算时间、误差率等方面进行对比。

通过运行结果可得，所提出的TW-Rainflow-Goodman-Miner累积疲劳损伤程度量化指标计算方法，可以实现根据单台风机的主轴扭矩、塔架推力到风机疲劳损伤程度的快速计算，且计算结果相似度良好，满足风机主轴和塔架任意时段累积疲劳损伤量化指标的实时计算的速度和精度要求。

4. 问题二的模型建立与求解

4.1. 问题分析

问题二要求建立风机风速条件、功率参考值与所承受的应力/扭矩之间的数学模型，实现基于风速和功率参考值对风机的主轴扭矩和塔架推力的估算过程。针对此问题，问题二的详细研究思路流程图如下图所示：

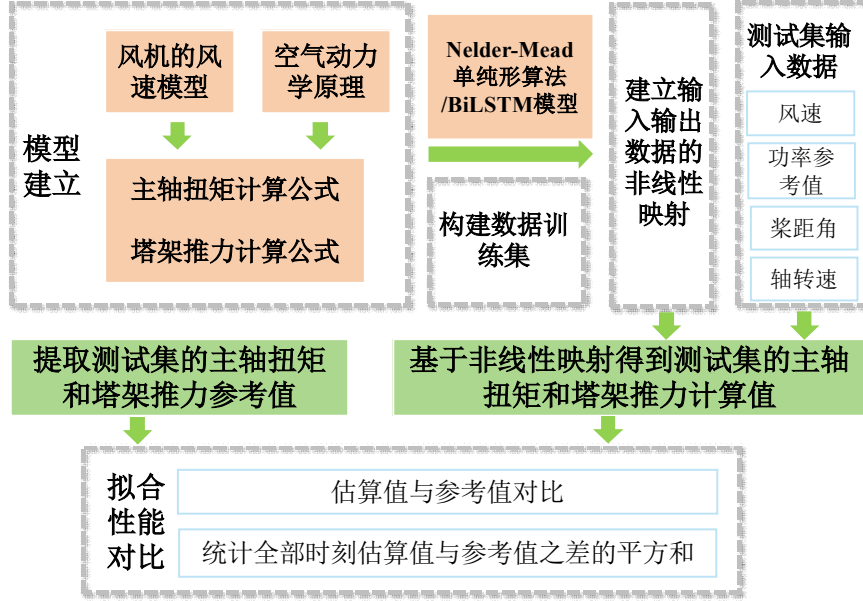


图15 问题二思路流程图

针对问题二，首先，根据风机的风速模型和空气动力学原理，明确风机风速、输出功率与主轴扭矩和塔架推力之间的物理计算公式，建立风机的主轴扭矩和塔架推力的数学计算模型；其次，基于风电场WF1前70%的输入输出数据构建训练集，采用Nelder-Mead单纯形算法、BiLSTM拟合方法描述桨距角、风速、功率参考值等参数与主轴扭矩、塔架推力的非线性映射关系；随后，基于训练好的非线性映射关系，对风电场WF1后30%的输入数据进行风机应力估算；最后，利用本文所训练的风机应力非线性映射关系，计算测试集中的风机在全部时刻的应力，并将数据与给出的参考值进行对比，选取出表现更为优势的拟合方法，并验证本文所建立的风机应力模型和应力估算方法的有效性。

4.2. 主轴扭矩和塔架推力计算模型

本文参考文献[6]建立风速和功率与风机的主轴扭矩、塔架推力之间的关系模型。

风机主轴获得的机械功率与风速的数学关系为：

$$P_m^i = \frac{1}{2} \pi \rho R^2 C_p(\lambda_i, \beta_i) V_w^i{}^3 \quad (15)$$

$$\lambda_i = \frac{w_r^i R}{V_w^i} \quad (16)$$

式中， P_m^i 为风机 i 的机械功率(W)， ρ 为空气密度(kg/m^3)， R 为叶片半径(m)， λ_i 为风机 i 叶尖速比， β_i 为桨距角， V_w^i 为风机 i 轮毂处等效风速(m/s)， w_r^i 为风机 i 转子转速(rad/s)， C_p 为功率系数/风能利用系数。

其中, C_p 与 β_i 和 λ_i 相关, 桨距角 β_i 越大, 遮挡的风能就越小。因此, 风电机组中内部控制器通过改变转子转速 w_r^i 和桨距角 β_i 来调整功率系数 C_p 的大小, 进而调整风电机组从风能中获得的机械功率。

在风机的调节过程中, 风电机组的功率指令发生变化, 塔架所受到的气动推力也会发生动态变化, 塔架的气动推力可以近似表示为:

$$F_t^i = \frac{1}{2} \pi \rho R^2 C_T(\lambda_i, \beta_i) V_w^i{}^2 \quad (17)$$

式中, F_t^i 为风机 i 的塔架所受到的气动推力, C_T 为推力系数。

风机的主轴扭矩可以表示为:

$$T_s^i = \frac{P_{ref}^i}{w_r} \quad (18)$$

式中, T_s^i 为风机 i 的主轴扭矩, P_{ref}^i 为风机 i 需要跟踪的电网功率参考值。由于模型假设中假设风机对发电功率参考值的跟踪过程可以精确实现, 因此在此式中不考虑风机的效率系数。

4.3. 基于数据拟合的主轴扭矩和塔架推力估算方法

风机的塔架推力系数 C_T 与风能利用系数 C_p 可通过二维查表法或者数据拟合方法获得, 因此, 当风能捕获系数 C_p 变化时, C_T 也会间接变化, 即当控制器进行功率调节时, 风电机组所受推力也一定会随之变化, 可能会影响机组的疲劳损伤。

在本文中, 选取Nelder-Mead单纯形算法和BiLSTM模型法两种方法进行数据拟合, 使用附件2中提供的风电场WF1前70%的数据进行数据拟合和模型训练, 形成风机风速、功率参考值和风机的主轴扭矩、塔架推力之间的非线性映射关系; 随后, 基于拟合结果, 结合风电场WF1后30%的风速和功率估算对应的主轴扭矩和塔架推力; 最后, 结合参考值对比两种数据拟合方法的效果, 选取出优势的拟合方法用于风机的主轴扭矩和塔架推力估算。

4.3.1. Nelder-Mead拟合原理

Nelder-Mead单纯形算法是一种直接搜索方法, 用于寻找无约束优化问题的最优解。该算法通过构建一个 n 维的单纯形(简单形状), 并根据目标函数的取值不断调整单纯形的顶点位置, 以找到最小化目标函数的解。其核心思想是通过反射、扩展和收缩等操作, 动态调整单纯形的形状, 直至满足停止条件^[7]。

Nelder-Mead单纯形算法用于数据拟合的具体实现步骤如下:

- 1) 初始化单纯形: 随机生成 $n+1$ 个初始点作为单纯形的顶点, 其中 n 为问题的维度;
- 2) 计算目标函数值: 计算每个顶点位置的目标函数值;
- 3) 排序: 根据目标函数值对顶点进行排序, 确定最好顶点(最小值)、次好顶点和最坏顶点(最大值);
- 4) 计算重心: 计算除最坏顶点外其余顶点的重心位置;
- 5) 反射操作: 根据重心位置进行反射操作, 得到反射点的位置;
- 6) 判断反射点: 比较反射点与次好顶点的目标函数值, 若反射点更好, 则接受反射点, 并进行扩展操作; 否则进行收缩操作; 扩展操作为在反射点的基础上继续扩展, 得到扩展点的位置; 收缩操作为在最坏顶点的方向上进行收缩操作, 得到收缩点的位置;
- 7) 更新顶点: 根据反射、扩展和收缩操作的结果更新单纯形的顶点位置;
- 8) 判断停止条件: 判断是否满足停止条件, 如达到最大迭代次数或目标函数值变化小于设定阈值;

9) 输出最优解：输出最终收敛的解作为最优参数值。

通过不断调整单纯形的顶点位置，Nelder-Mead单纯形算法可以高效地搜索最优数据拟合参数。在实际应用中，需要根据具体问题的特点和目标函数的性质来选择合适的初始化方式、停止条件和参数调整策略，以确保算法的有效性和收敛性。

4.3.2. BiLSTM拟合原理

BiLSTM（双向长短期记忆网络）是一种循环神经网络（RNN）的变体，常用于序列数据的建模和预测。LSTM（长短期记忆网络）作为一种特殊的RNN，通过门控机制来有效解决长序列数据训练过程中的梯度消失或梯度爆炸问题，同时能够捕捉长期依赖关系。BiLSTM是双向的，通过同时在序列的两个方向上运行两个独立的LSTM网络（一个前向，一个后向），从而更好地捕捉序列数据中的上下文信息。在数据拟合过程中，BiLSTM首先通过前向传播和后向传播分别处理输入序列，得到前向和后向的隐藏状态序列。然后将前向和后向的隐藏状态拼接在一起，得到每个时间步的综合隐藏状态表示。最终，通过全连接层等操作，将综合隐藏状态映射为输出结果，用于数据的预测或分类。总的来说，BiLSTM通过双向的方式，结合前向和后向的信息，能够更好地建模序列数据的上下文关系，从而提高数据拟合的效果^[8]。

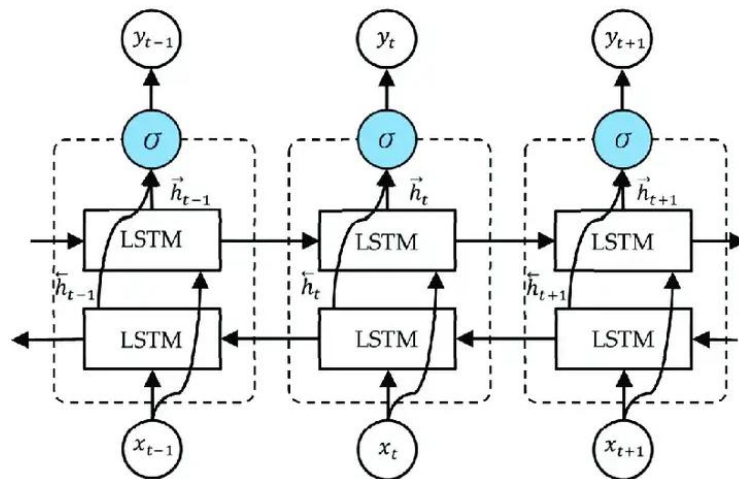


图16 BiLSTM原理图

BiLSTM应用于数据拟合通常包括以下步骤：

1) 数据准备：首先需要准备数据集，包括输入数据和对应的标签（或目标值）。数据可以是序列数据，文本数据或其他具有序列性质的数据；

2) 数据预处理：对数据进行预处理，包括数据清洗、标准化、分割训练集和测试集等工作，确保数据格式符合BiLSTM网络的输入要求；

3) 构建BiLSTM模型：构建一个BiLSTM模型，可以使用深度学习框架如TensorFlow、PyTorch等来构建模型，在构建模型时需要指定网络的结构，包括输入层、BiLSTM层、全连接层等；

4) 编译模型：在构建模型后，需要编译模型，指定损失函数、优化器和评估指标。损失函数可以根据具体问题选择，优化器一般选择Adam、SGD等，评估指标可以是准确率、均方误差等；

5) 模型训练：使用训练集对模型进行训练，通过反向传播算法不断更新模型参数，使模型逐渐收敛到最优解。

6) 模型评估：使用测试集对训练好的模型进行评估，计算模型在测试集上的准确率或其他评估指标；

7) 模型应用：使用训练好的BiLSTM模型对新数据进行预测或数据拟合，可以通过输入数据序列，获取模型输出的预测结果；

具体的实现过程还会根据具体问题和数据集的特点而有所不同；在实际应用中，可通过调参、模型优化达到更好的数据拟合效果。

4.3.3. 主轴扭矩和塔架推力估算方法

基于4.2节中建立的风机主轴扭矩和塔架推力计算模型和附件所提供的风电场运行数据，可结合Nelder-Mead单纯形算法、BiLSTM模型等数据拟合方法，形成风机主轴扭矩和塔架推力的估算方法，具体实现步骤如下：

- 1) 数据预处理：对附件中提供的风电场WF1的数据进行整理和提取；
- 2) 数据训练集构建：提取附件数据中的风电场WF1的前70%数据作为训练集，提取风电场WF1的后30%数据作为测试集，对应分别构建训练集和测试集的数据序列；
- 3) 分别采用Nelder-Mead单纯形算法、BiLSTM模型对数据训练集进行数据拟合，建立风机风速、功率参考值等输入数据与主轴转矩、塔架推力之间的非线性映射关系；
- 4) 基于训练集所得的非线性映射关系，分别估算测试集的主轴扭矩和塔架推力，对比两种拟合方法的估算效果，选取估算效果更优势的拟合方法；
- 5) 对比基于测试集输入数据和非线性映射关系得到的测试集的风机主轴扭矩和塔基推力的估算值与参考值，分析所提出的估算方法的有效性和估算效果。

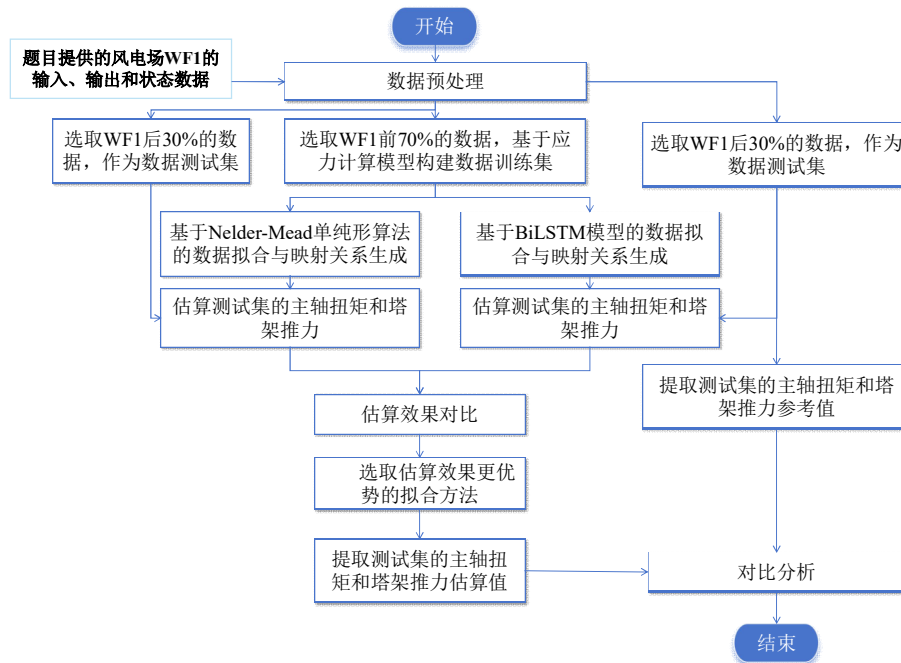


图17 基于数据拟合的主轴扭矩和塔架推力估算方法程序流程图

4.3.4. 误差评价方法

对风电场WF1后30%的主轴扭矩和塔架推力估算值与参考值的差异采用以下方法进行量化：

(1) 平均绝对误差 (MAE)

$$RSS = \sum_{k=1}^N \left(F_{t,cal}^i(k) - F_{t,ref}^i(k) \right)^2 \quad (19)$$

平均绝对误差取值范围 $[0, +\infty)$ ，当预测值与真实值完全吻合时等于0，即完美模型；误差越大，该值越大。

(2) 平均绝对百分比误差 (MAPE)

$$MAPE = \frac{1}{N} \sum_{k=1}^N \left(\frac{F_{t,cal}^i(k) - F_{t,ref}^i(k)}{F_{t,ref}^i(k)} \right)^2 \quad (20)$$

平均绝对百分比误差的取值范围为 $[0, +\infty)$ ，MAPE 为0%表示完美模型，MAPE大于100 %则表示劣质模型。

(3) 均方误差 (MSE)

$$MSE = \frac{1}{N} \left(\sum_{k=1}^N \left(\frac{F_{t,cal}^i(k) - F_{t,ref}^i(k)}{F_{t,ref}^i(k)} \right) \times 100\% \right)^2 \quad (21)$$

均方误差取值范围为 $[0, +\infty)$ ，当预测值与真实值完全吻合时等于0，即完美模型；误差越大，该值越大。

(4) 决定系数 (R^2)

$$R^2 = 1 - \frac{\sum_{k=1}^N (F_{t,cal}^i(k) - F_{t,ref}^i(k))^2}{\sum_{k=1}^N (F_{t,ref}^i(k))^2} \quad (22)$$

决定系数用于衡量自变量对因变量变异的解释程度。具体来说：当 $R^2=1$ 时，表示模型完美地解释了所有的数据变异；当 R^2 越小，表明模型解释的变异比较小，即模型的拟合效果不佳。

(5) 均方根误差 (RMSE)

$$RMSE = \sqrt{\frac{1}{N} \sum_{k=1}^N (F_{t,cal}^i(k) - F_{t,ref}^i(k))^2} \quad (23)$$

均方根误差的取值范围为 $[0, +\infty)$ ，当预测值与真实值完全吻合时等于0，即完美模型；误差越大，该值越大。

(6) 残差平方和 (RSS)

在统计学中，残差平方和是一个重要的概念，特别是在线性回归分析中。它表示数据点与在回归直线上对应位置的差异，即实际值与预测值之间的偏差量。利用这种偏差的平方和来评估模型对数据的拟合程度。简而言之，残差平方和越小，表示模型的拟合效果越好，因为这意味着实际值与预测值之间的差异较小，随机误差的影响也相应较小。

$$RSS = \sum_{k=1}^N (F_{t,cal}^i(k) - F_{t,ref}^i(k))^2 \quad (24)$$

4.4. 结果分析

基于MATLAB进行程序编写和执行，对比所提出的两种拟合方法用于数据集训练和应力计算的应用效果，验证了所提出的基于数据拟合的风机主轴扭矩和塔架推力估算方法的有效性。

4.4.1. 估算说明

首先，基于附件2提供的风电场WF1前70%的输入输出数据，进行功率系数和推力系数的数据拟合，得到风速和功率参考值与风机的主轴扭矩和塔架推力的映射关系；随后，基于映射关系和附件2所提供的风电场WF1后30%的数据中的风速和功率参考值，估算各个风机任意时刻的应力/扭矩值，并与数据中给出的参考值进行对比。

表2 数据集构建说明

输入特征	输出	数据集时长	风机数	训练集占比	测试集占比
尖速比（风速）	扭矩	2000s	100	WF1前70%	WF1后30%
桨距角					
输出功率					

4.4.2. 基于BiLSTM拟合的估算结果

（1）主轴扭矩训练与估算效果展示

图18为基于BiLSTM方法的主轴扭矩训练结果图，可以看出可以看到基于BiLSTM方法的主轴扭矩数据拟合的效果很好，估算的扭矩结果和给定的数据重叠率很高，能够实现高精度的数据估算。图19为训练集和测试集结果中估算扭矩值和给定实际扭矩值的散点图展示图，可以看出样本点大多都聚集在 $y=x$ 的曲线附近，表示数据的拟合效果好，拟合结果的偏差较小。

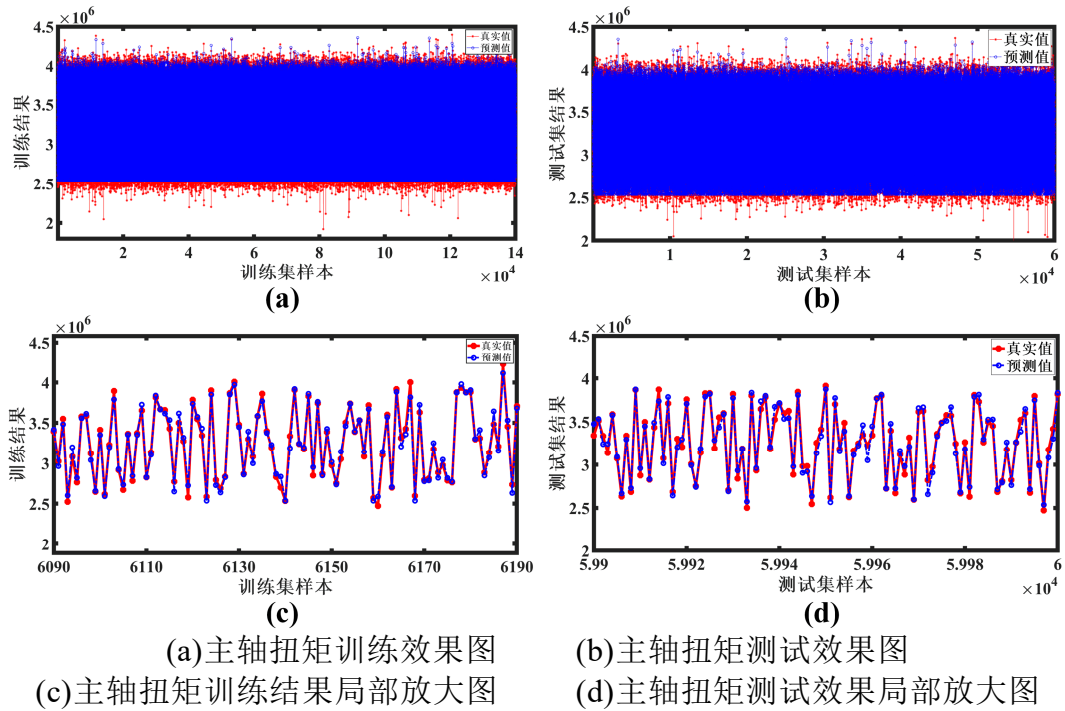


图18 基于BiLSTM方法的主轴扭矩训练结果图

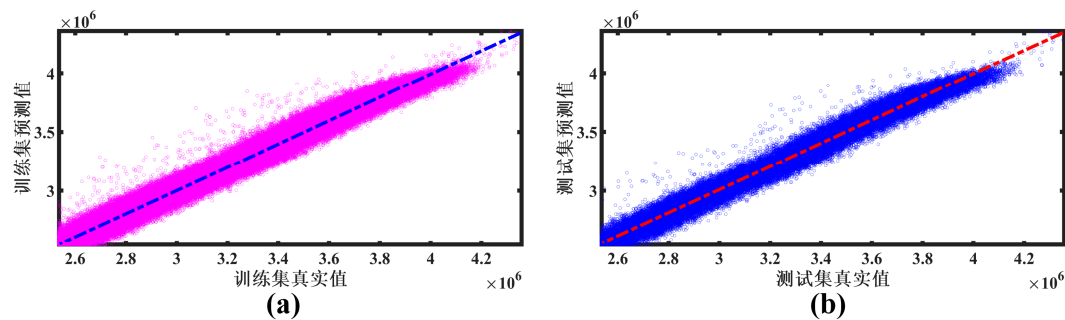


图19 基于BiLSTM方法的训练集和测试集中估算扭矩值和给定实际扭矩值的散点图

图20为BiLSTM训练主轴扭矩过程示意图，左侧是训练进度的动态展示区，其中蓝色曲线表示误差曲线，橙色曲线表示损失值；右侧展示了训练的时间，训练周期、迭代次数等状态。

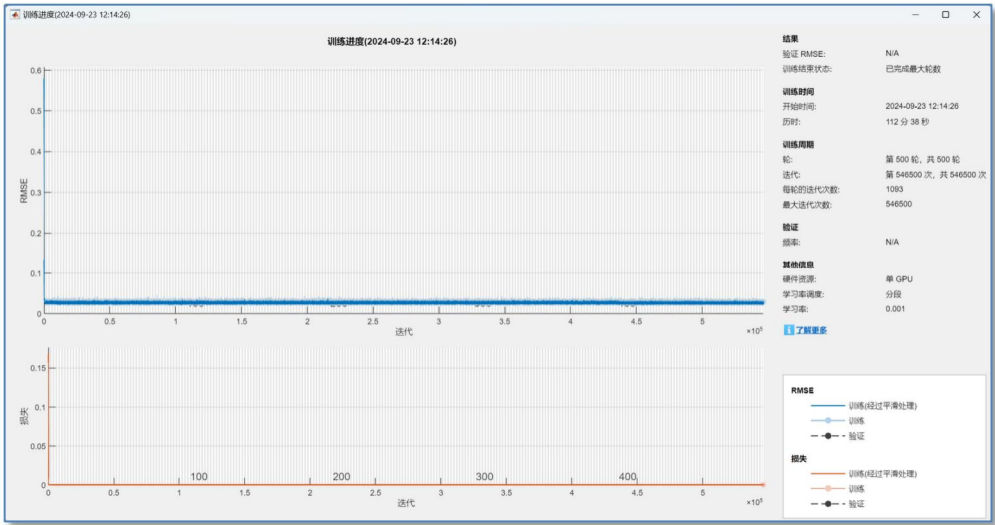
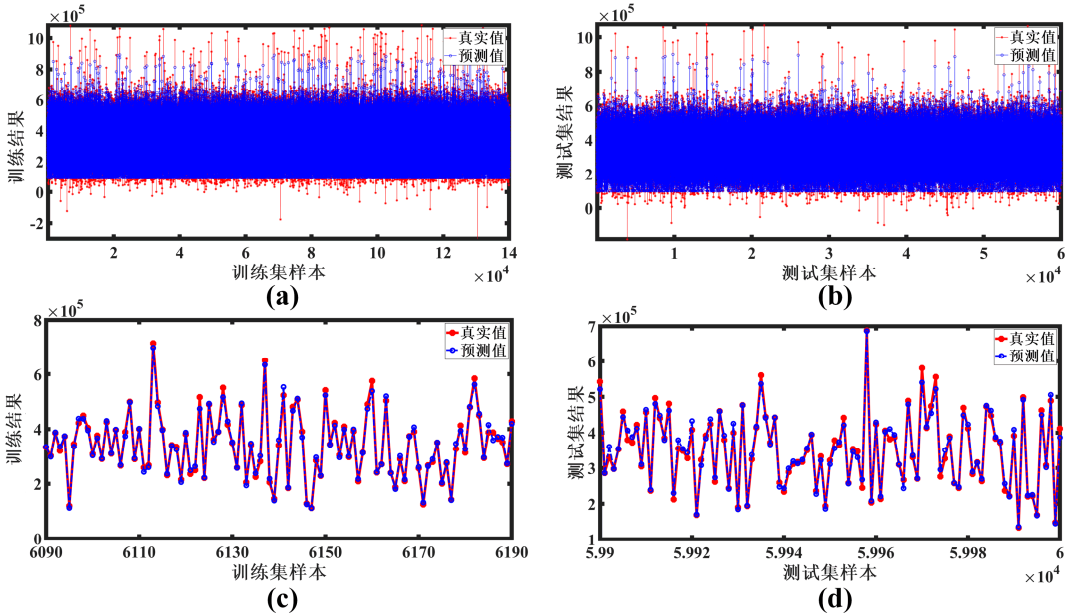


图20 基于BiLSTM方法的主轴扭矩训练过程示意图

（2）塔架推力训练与估算效果展示

图21为塔架推力训练集和测试集数据结果，可以看到基于BiLSTM方法的塔架推力拟合的效果很好，估算的推力结果和给定的数据重叠率很高。图22为训练集和测试集结果中推力估算值和给定实际推力值的散点图展示，可以看到样本点大多都聚集在 $y=x$ 的曲线附近，表示数据的拟合效果好，结果偏差较小。图23为基于BiLSTM方法的塔架推力训练过程示意图，左侧是训练进度的动态展示区，其中蓝色曲线表示误差曲线，橙色曲线表示损失值；右侧展示了训练的时间，训练周期、迭代次数等状态。



(a)塔架推力训练效果图 (b)塔架推力测试效果图
(c)塔架推力训练结果局部放大图 (d)塔架推力测试效果局部放大图

图21 基于BiLSTM方法的塔架推力训练结果图

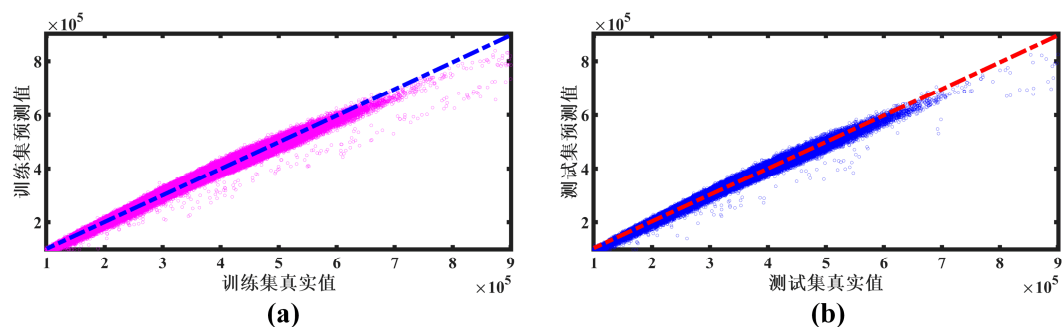


图22 基于BiLSTM方法的训练集和测试集中估算推力值和给定实际推力值的散点图

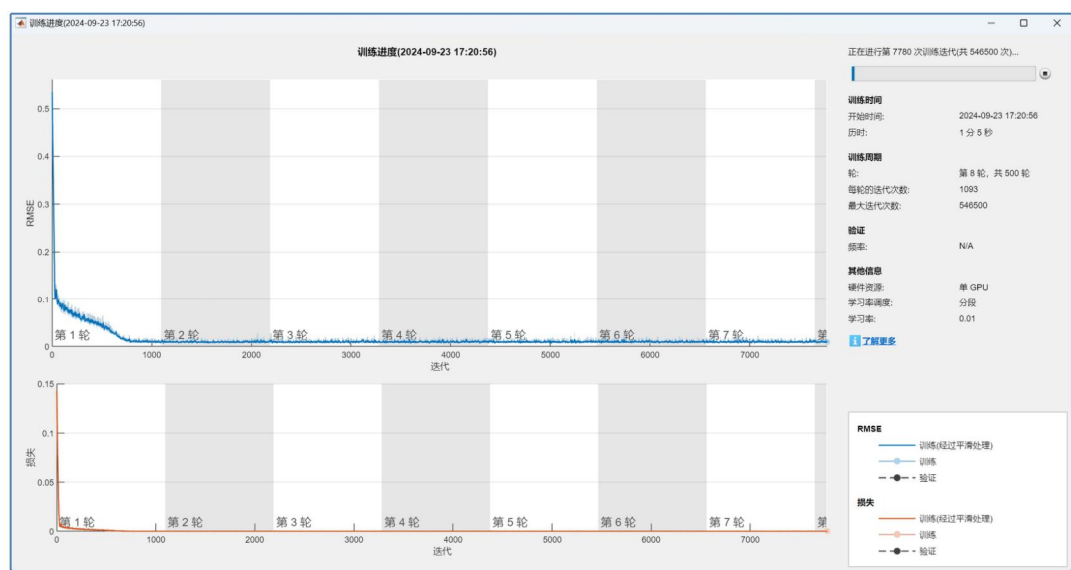


图23 基于BiLSTM方法的塔架推力训练过程示意图

4.4.3. 基于Nelder-Mead拟合的估算结果

图24和图25分别为基于Nelder-Mead方法进行数据拟合的主轴扭矩和塔架推力在最后100s的拟合效果，可以看出，相比参考值来说，基于Nelder-Mead拟合的估算结果的误差较大。

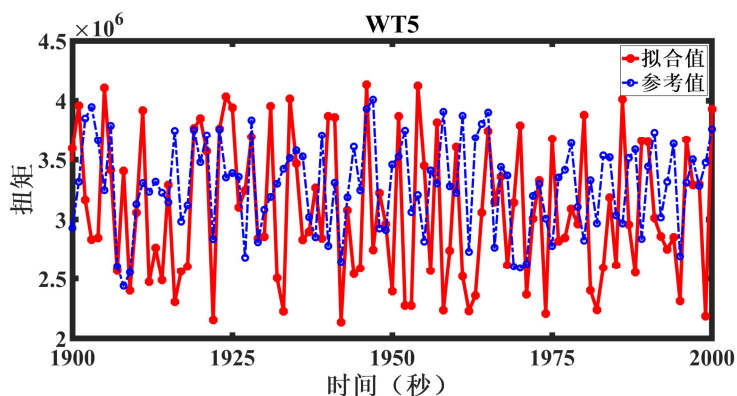


图24 基于Nelder-Mead方法的主轴扭矩拟合效果

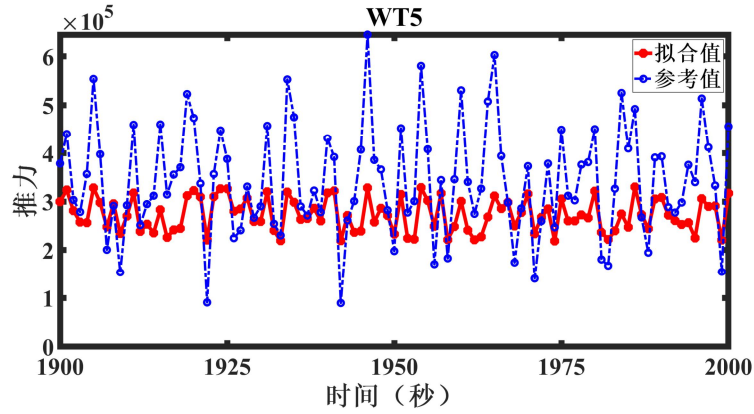


图25 基于Nelder-Mead方法的塔架推力拟合效果

4.4.4. 基于两种拟合方法的估算值与参考值对比

基于两种拟合方法的估算值与参考值对比结果如下表所示3，基于MAE、MAPE、MSE、 R^2 、RMSE等误差评价指标的计算和对比可得，所有的指标都显示BiLSTM结果相比拟合结果来说好得多，其中，MSE在扭矩中相比拟合结果降低了99.05%，MSE在推力中相比拟合结果降低了69.35%。

表3 基于两种拟合方法的估算值与参考值对比

对象	方法	MAE	MAPE	MSE	R^2	RMSE
扭矩	Nelder-Mead	5.7934×10^5	3.5507×10^2	0.0456	0.0039	5.7934×10^5
	BiLSTM	5.1988×10^5	0.0163	4.4436×10^{-4}	0.9742	6.5917×10^4
推力	Nelder-Mead	1.1940×10^5	6.8503×10^2	0.2320	0.0033	1.1940×10^5
	BiLSTM	9.4684×10^3	0.0343	0.0711	0.9854	1.3112×10^4

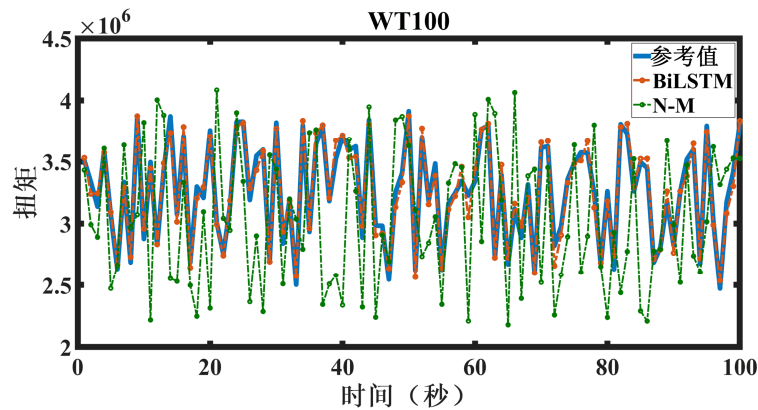


图26 基于两种拟合方法的主轴扭矩拟合效果对比图

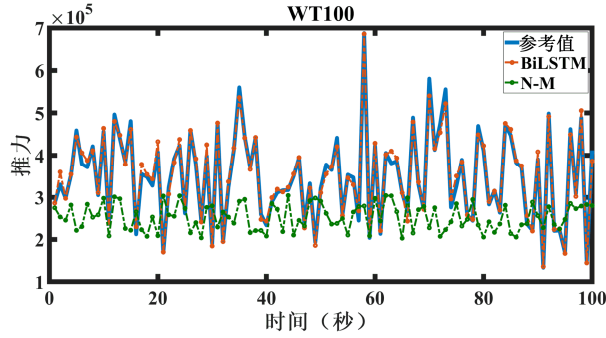


图27 基于两种拟合方法的塔架推力拟合效果对比图

图26为风机WT100在最后100s时，两种方法估计的估算扭矩和参考扭矩的曲线对比图；图27为风机WT100在最后100s时，两种方法估计的估算推力和参考推力的曲线对比图。基于图26和图27的对比结果可得，BiLSTM的重合度更高，Nelder-Mead的拟合效果不佳，即基于BiLSTM拟合的主轴扭矩和塔架推力估算方法为更优势的计算方法。

特别说明，对于估算结果，已将100台风机在全部时刻的应力/扭矩计算结果列入附件6的表格中，100台风机的全部时刻估算值与参照值之差的平方和（RSS指标）也列入了附件6的表格中，在报告中不再重复展示。

4.5. 问题小结

考虑到风速过高时，风机的主轴和塔架将承受过大的扭矩或推力，从而增加了风机的累积疲劳损伤程度。为降低风机运行过程中的损伤程度，需考虑风速对于风机的主轴和塔架的累积疲劳损伤影响。为此，问题二要求建立风机风速条件、功率参考值与所承受的应力/扭矩之间的数学模型，实现基于风速和功率参考值对风机的主轴扭矩和塔架推力的估算过程。

本文针对此问题，首先，根据风机的风速模型和空气动力学原理，明确风机风速、输出功率与主轴扭矩和塔架推力之间的物理计算公式，建立风机的主轴扭矩和塔架推力的数学计算模型；其次，基于风电场WF1前70%的输入输出数据构建训练集，采用Nelder-Mead单纯形算法、BiLSTM拟合方法描述桨距角、风速、功率参考值等参数与主轴扭矩、塔架推力之间的非线性映射关系；随后，基于训练好的非线性映射关系，对风电场WF1后30%的输入数据进行应力估算，估算该风电场每个时刻的所有风机的主轴扭矩和塔架应力；最后，对比两种拟合方法的应用效果，选取更为优势的拟合方法，将估算结果与参考值进行比较，验证所建模型和所提方法的有效性。

通过所提方法估算风电场WF1后30%的主轴扭矩和塔架推力，通过结果对比，验证了使用BiLSTM模型相较于Nelder-Mead单纯形算法的拟合效果更好；将基于BiLSTM数据拟合得到的估算值与参考值进行对比，估算值的误差位于较小范围，验证了所提方法能够实现对于风机的主轴扭矩和塔架推力的高精度估算。上述过程验证了本文所提方法的有效性，能够为后续优化风机集群的联合调度问题提供了计算基础。

5. 问题三的模型建立与求解

5.1. 问题分析

问题三为有功调度优化问题构建与实时求解，要求基于题目所述的优化目标和约束条件，建立优化模型并进行实时求解，得到风电场内的最优有功功率分配策略。

问题三的具体思路流程图如下所示：

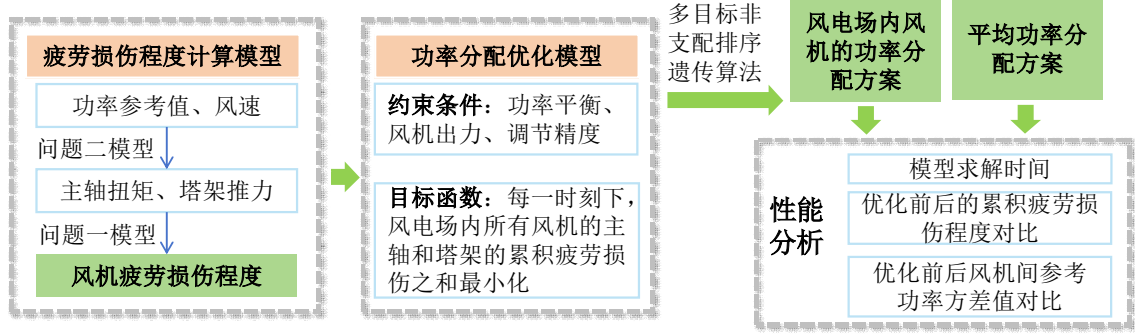


图28 问题三思路流程图

在问题一和问题二的基础上，首先，构建风机疲劳损伤程度计算模型，在每个运行周期内基于风速和功率参考值实时计算风电场内所有风机的累积疲劳损伤程度；随后，以当前运行周期内所有风机的主轴和塔架的总体累积疲劳损伤程度最小为目标函数，考虑功率平衡、风机出力和调节精度等约束条件，建立风电场有功功率优化分配模型；然后，通过多目标非支配排序遗传算法进行模型求解，实现风电场内各风机有功功率的优化调度和实时更新，保证各个风机的疲劳损伤程度和功率分配的合理性；最后，将所求功率优化分配方案的运行效果与平均功率分配方案进行对比，验证建立的优化模型和模型求解方法的有效性。

5.2. 基于功率参考值的风机累积疲劳损伤程度计算模型

基于问题二的解决方案，可形成如4.2节所示的风机运行状态参数、功率参考值等输入数据与风机的主轴扭矩和塔架推力之间的数学模型；基于问题一的解决方案，可构建风机疲劳损伤程度量化指标计算低复杂度模型，形成风机主轴扭矩和塔架推力与对应的元件疲劳损伤程度量化指标的数学模型。综合问题一和问题二，可得到基于功率参考值的风机主轴与塔架累积疲劳损伤程度计算模型如下：

$$F_t^i = \frac{1}{2} \pi \rho R^2 C_T(\lambda_i, \beta_i) V_w^i{}^2 \quad (25)$$

$$T_s^i = \frac{P_{ref}^i}{w_r} \quad (26)$$

$$S_T^i = k_T T_s^i \quad (27)$$

$$S_F^i = k_F F_t^i \quad (28)$$

$$D_T^i = \sum_{j=1}^n \frac{n_F^i(s_T^j)}{N_F^i(s_T^j)} \quad (29)$$

$$D_F^i = \sum_{j=1}^n \frac{n_F^i(S_F^j)}{N_F^i(S_F^j)} \quad (30)$$

$$D_{T_total}^i(T_t) = \sum_{k=1}^{T_t} D_T^i(k) \quad (31)$$

$$D_{F_total}^i(T_t) = \sum_{k=1}^{T_t} D_F^i(k) \quad (32)$$

式中， S_T^i 和 S_F^i 分别为风机 i 的主轴和塔架的应力幅值； $D_T^i(k)$ 和 $D_F^i(k)$ 分别为风机 i 的主轴和塔架在 k 时刻的疲劳损伤程度量化指标； $D_{T_total}^i(T_t)$ 和 $D_{F_total}^i(T_t)$ 分别为风机 i 的主轴和塔架在 T_t 时刻的累积疲劳损伤程度量化指标。

5.3. 考虑疲劳损伤的风机有功功率优化分配模型

为降低风电场的运维成本和风机故障率，需尽可能地降低风电场内所有风机的总体疲劳损伤程度，单台风机的疲劳损伤包括主轴的疲劳损伤与塔架疲劳损伤两部分。

因此，在风电场内多台风机的有功功率优化分配过程中，需要在每秒钟的有功功率的优化分配计算过程中，根据多台风机的疲劳损伤情况合理进行功率的优化分配，在保证风机集群整体可有效、快速响应电网调度指令的同时，实现各台风机的疲劳损伤程度整体削减与优化。为此，以当前运行周期内所有风机的主轴和塔架的总体疲劳损伤程度最小为目标函数，考虑功率平衡、风机出力和调节精度等约束条件，构建考虑主轴和塔架的疲劳损伤程度的风机有功功率优化分配模型，目标函数和约束条件详述如下。

5.3.1. 目标函数

分别将风电场内所有风机的主轴和塔架在每一时刻的累积疲劳损伤量化指标之和最小定义为两个目标函数：

$$\min F_1 = (\sum_{i=1}^{W_t} D_{T_total}^i(k)) \quad (33)$$

$$\min F_2 = (\sum_{i=1}^{W_t} D_{F_total}^i(k)) \quad (34)$$

式中， W_t 为风电场风机的数量， $D_{T_total}^i(k)$ 和 $D_{F_total}^i(k)$ 分别为风机 i 的主轴和塔架在 k 时刻的疲劳损伤程度量化指标。

5.3.2. 约束条件

优化模型的约束条件主要包含功率平衡、风机出力和调节精度三个方面。

在功率平衡方面，需保证所有风机有功参考值之和等于电网调度指令：

$$\sum_{i=1}^{W_t} P_{ref}^i = P_t \quad (35)$$

式中， P_t 为电网下达给风电场的调度指令要求功率， P_{ref}^i 为风电场分配到风机 i 的功率参考值，是该风机需要跟随的功率目标。

在风机出力方面，需保证各风机有功参考值不大于风机有功功率额定值（5MW）：

$$0 \leq P_{ref}^i \leq 5\text{MW} \quad (36)$$

在调节精度方面，需保证各风机有功优化分配值与平均分配方法结果差值不超过1MW：

$$\left| P_{ref}^i - \frac{P_t}{W_t} \right| \leq 1\text{MW} \quad (37)$$

5.4. 基于NSGA-II算法的模型求解方法

多目标非支配排序遗传算法（Multi-Objective Non-Dominated Sorting Genetic Algorithm, 简称NSGA-II）是一种用于解决多目标优化问题的进化算法。其基本原理是通过遗传算法的进化过程来搜索解空间中的非支配解集合，以实现在多个冲突目标之间找到Pareto最优解集合^[9]。

NSGA-II算法具有高效的搜索性能和较好的收敛性，被广泛应用于多目标优化领域。NSGA-II算法应用于求解多目标优化模型的具体步骤如下：

- 1) 初始化种群：随机生成初始种群，包括一定数量的个体，每个个体代表问题的一个解，并且要符合问题的约束条件；
- 2) 评估适应度：计算每个个体的适应度值；
- 3) 非支配排序：根据个体之间的支配关系，将种群中的个体划分为不同的非支配层次；
- 4) 拥挤度计算：对每个非支配层次中的个体进行拥挤度计算，用于选择较优个体；
- 5) 选择操作：根据非支配排序和拥挤度计算，选择新一代种群的个体；
- 6) 交叉操作：对选择出的个体进行交叉操作，生成子代个体；
- 7) 变异操作：对子代个体进行变异操作，引入随机扰动，增加种群的多样性；
- 8) 合并种群：将父代种群和子代种群进行合并；
- 9) 非支配排序和拥挤度计算：对合并后的种群进行非支配排序和拥挤度计算；
- 10) 选择新种群：根据非支配排序和拥挤度计算，选择下一代种群的个体；
- 11) 迭代：重复执行第6至第10步，直到满足停止条件（如达到最大迭代次数）；
- 12) 输出结果：输出最终的Pareto最优解集合，即所有非支配解集合中的最优解。

本文针对问题三的多目标有功功率优化分配模型，应用NSGA-II算法求解模型的具体流程如下图所示：

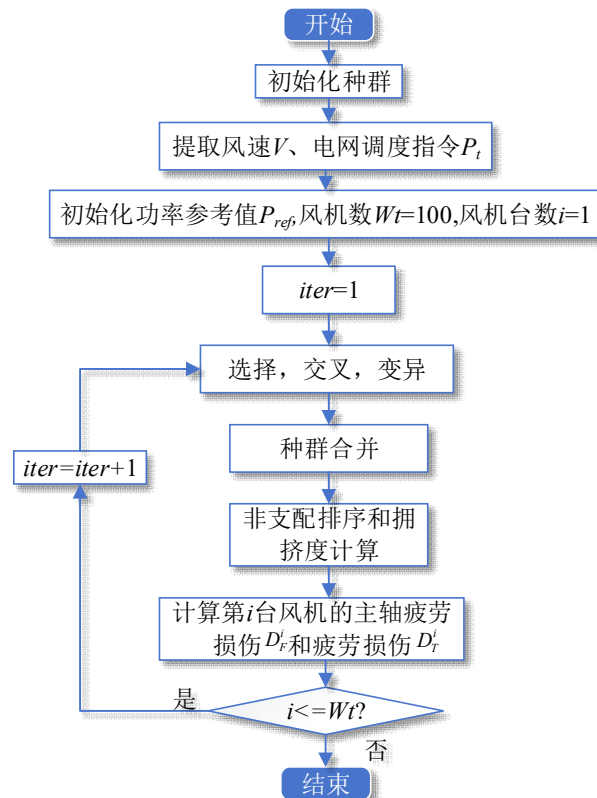


图29 有功功率优化分配模型求解流程图

5.5. 优化结果分析

5.5.1. 计算实时性展示

基于MATLAB进行程序编写和执行，对程序的执行时间进行100s功率优化分配计算的运行过程的动图记录，展示计算结果的实时性。由于文档对于动图的展示限制，仅在文档中展示所录制动图的截图，程序运行时间的动图另附在附件中单独上传。

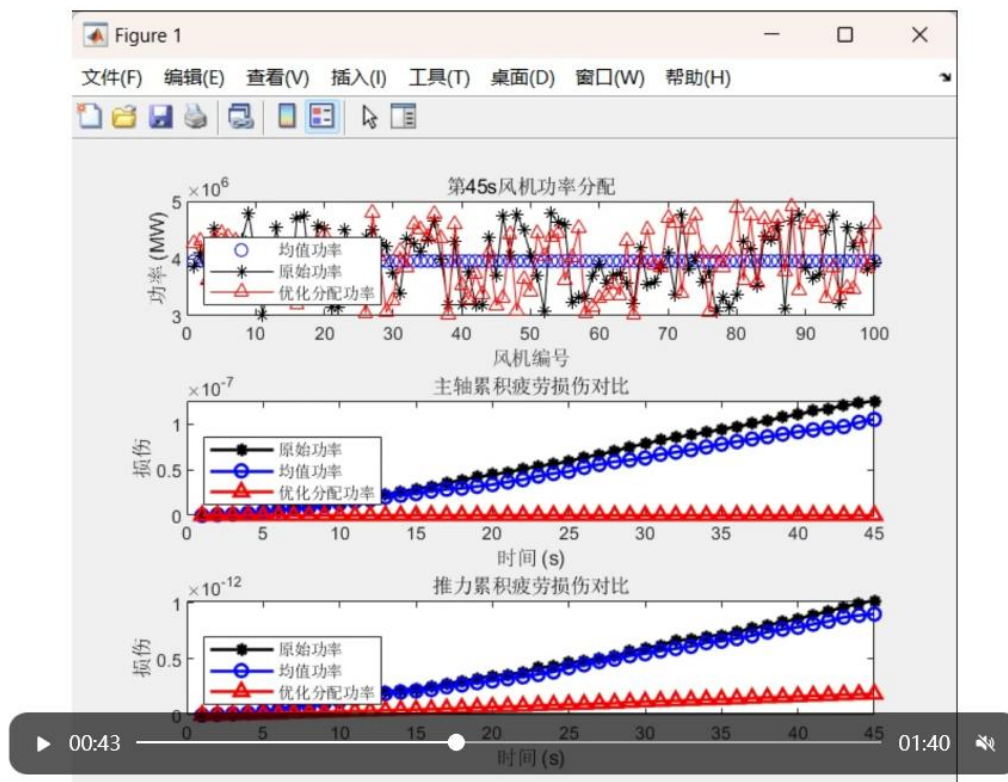


图30 含有计时器的程序执行动图截图

5.5.2. 优化模型约束条件满足情况

优化模型的约束条件包括功率平衡、风机出力和调节精度三个方面。

图31为每时刻风机总输出功率与电网调度功率对比图，由图可知，在1~100s内，调度功率曲线与基于优化模型得到的风机输出功率总和的曲线表现出高度一致性；图32为每时刻调度功率与风机总输出功率之和的误差柱状图，整体误差维持在 10^{-7} 数量级，表明优化过程中的数值偏差极小。

图31和图32证明了基于本文所建立的有功功率优化分配模型求解得到的功率优化分配方案可以保证风电场中所有风机的总输出功率有效、精确地跟踪电网下达指令的功率参考值。图31中所有时刻的功率参考值均小于5MW，即该优化结果可以保证各风机有功参考值不大于风机有功功率额定值（5MW）。验证了所建立的优化模型的求解结果可以满足各风机有功优化分配值与平均分配方法结果差值不超过1MW。图32中所有时刻的风机输出总功率与调度功率差值均不超过 10^{-6} ，即该优化结果可以保证各风机有功优化分配值与平均分配方法结果差值不超过1MW。

综上所述，所求优化结果可以满足所有的约束条件。

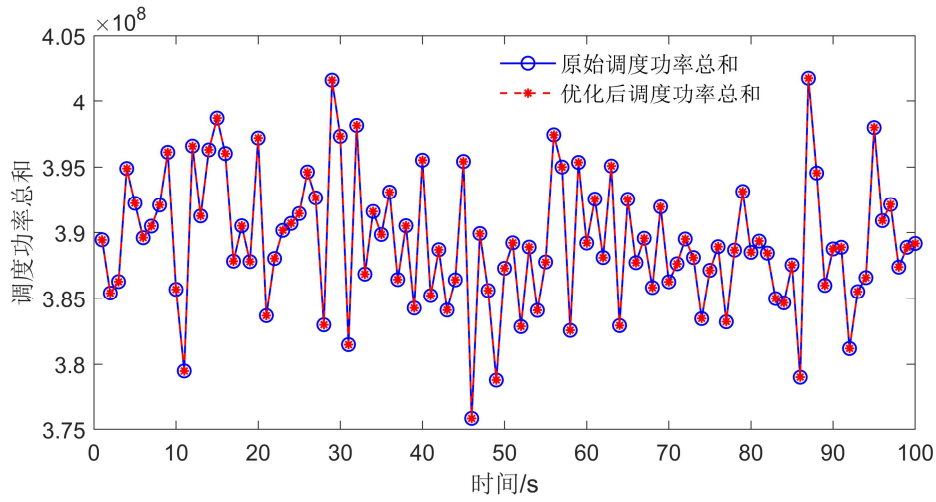


图31 每时刻风机总输出功率与电网调度功率对比图

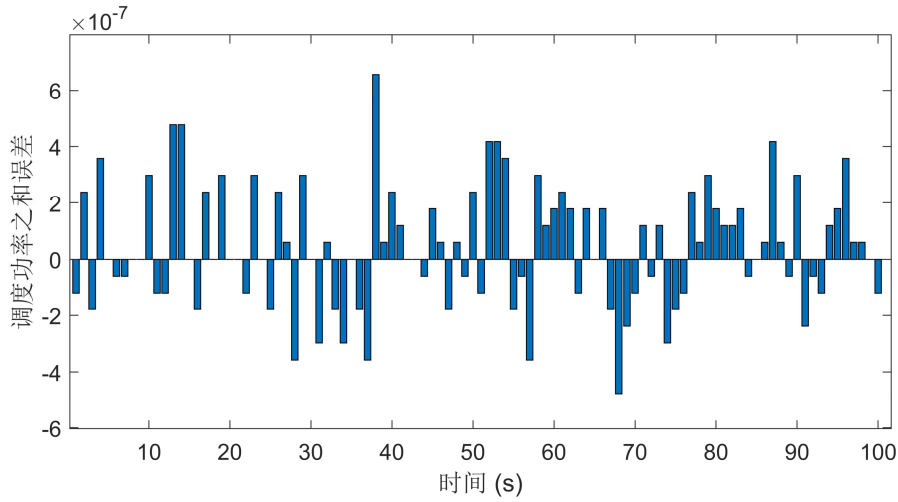


图32 每时刻调度功率与风机总输出功率之和的误差柱状图

5.5.3. 优化结果与平均分配功率结果对比

将基于本文所提出的风机有功功率优化分配模型的求解结果与基于平均分配功率的结果进行对比。

图33为优化前后各个风机的主轴累积疲劳损伤程度对比图，**优化后的功率分配使得主轴的累积疲劳损伤值降至 10^{-13} 数量级**，而原始调度功率曲线及均值调度功率曲线的损伤值则为 10^{-7} 数量级。这一结果清晰地表明了优化算法在疲劳损伤管理上的高效性。均值分配方案相比原始功率调度方案的疲劳损伤值减小，主要是由于均值分配能够更平滑地分配负载，减少了瞬时负荷的剧烈波动。原始调度中，负载的波动性较大，导致主轴在短时间内承受了较高的应力，从而加速了疲劳损伤的积累，而均值调度通过对负载的合理平均与分配，显著降低了峰值负荷，从而有效减小了主轴的损伤风险。优化后的方案更进一步，通过数据分析与调整，提高了负载的分配效率，使得主轴在工作过程中始终处于更安全的应力区间。这不仅延长了主轴的使用寿命，还优化了整体设备的运行稳定性。因此，优化后的功率分配方案不仅在理论上展示了其有效性，在实际应用中同样展现出显著的优势。

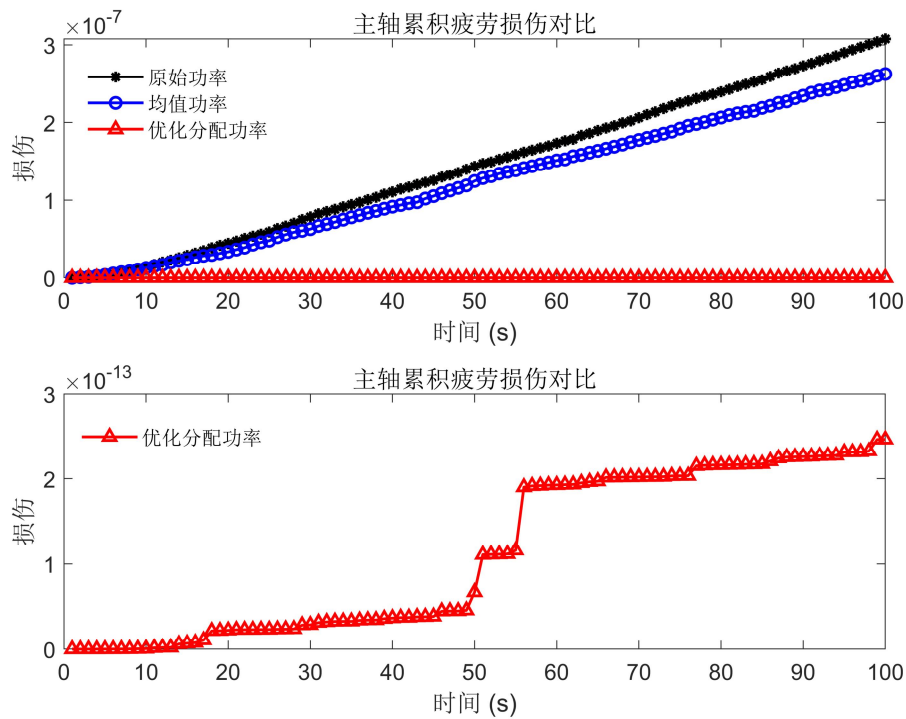


图33 优化前后所有风机的主轴累积疲劳损伤程度对比图

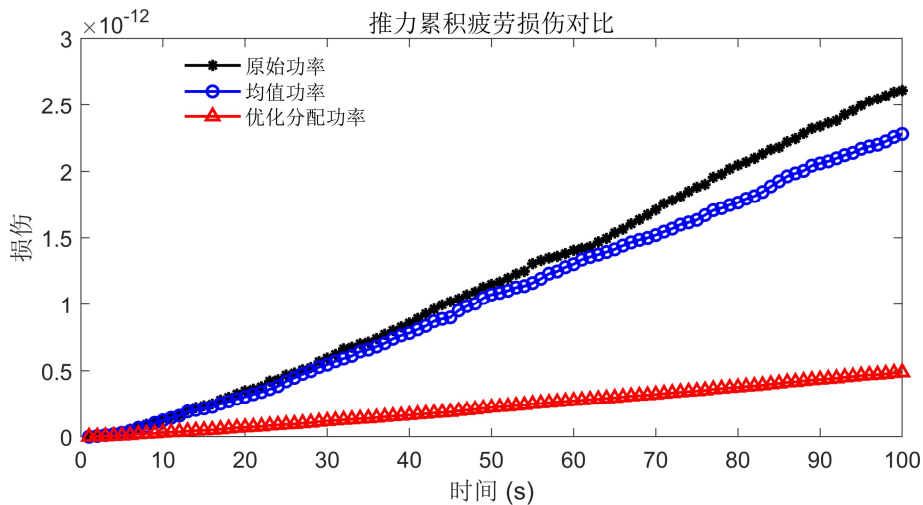


图34 优化前后所有风机的推力累积疲劳损伤程度对比图

图34为优化前后所有风机的推力累积疲劳损伤程度对比图，**优化后的功率分配方案使得推力累积疲劳损伤值降至 10^{-13} 数量级**，而均值调度功率曲线的损伤值为 10^{-12} 数量级，这表明了优化算法在降低疲劳损伤方面的高效性。从推力角度分析，原始调度功率曲线可能存在某些峰值和波动，导致推力的瞬时过载，从而加速材料疲劳，均值调度虽然能一定程度上平衡推力，但仍然未能充分利用优化算法所带来的动态调整能力。优化后的功率分配方案能够更精确地匹配实际需求，避免了不必要的过载和低效运行。此外，优化后方案可能采用了更为细致的功率分配调节策略，根据实时运行状态和外部条件动态调整功率分配，降低了在特定工况下的疲劳损伤风险。这种精准的功率分配管理不仅能够提高系统的整体效率，还能延长设备的使用寿命，减少维护成本。

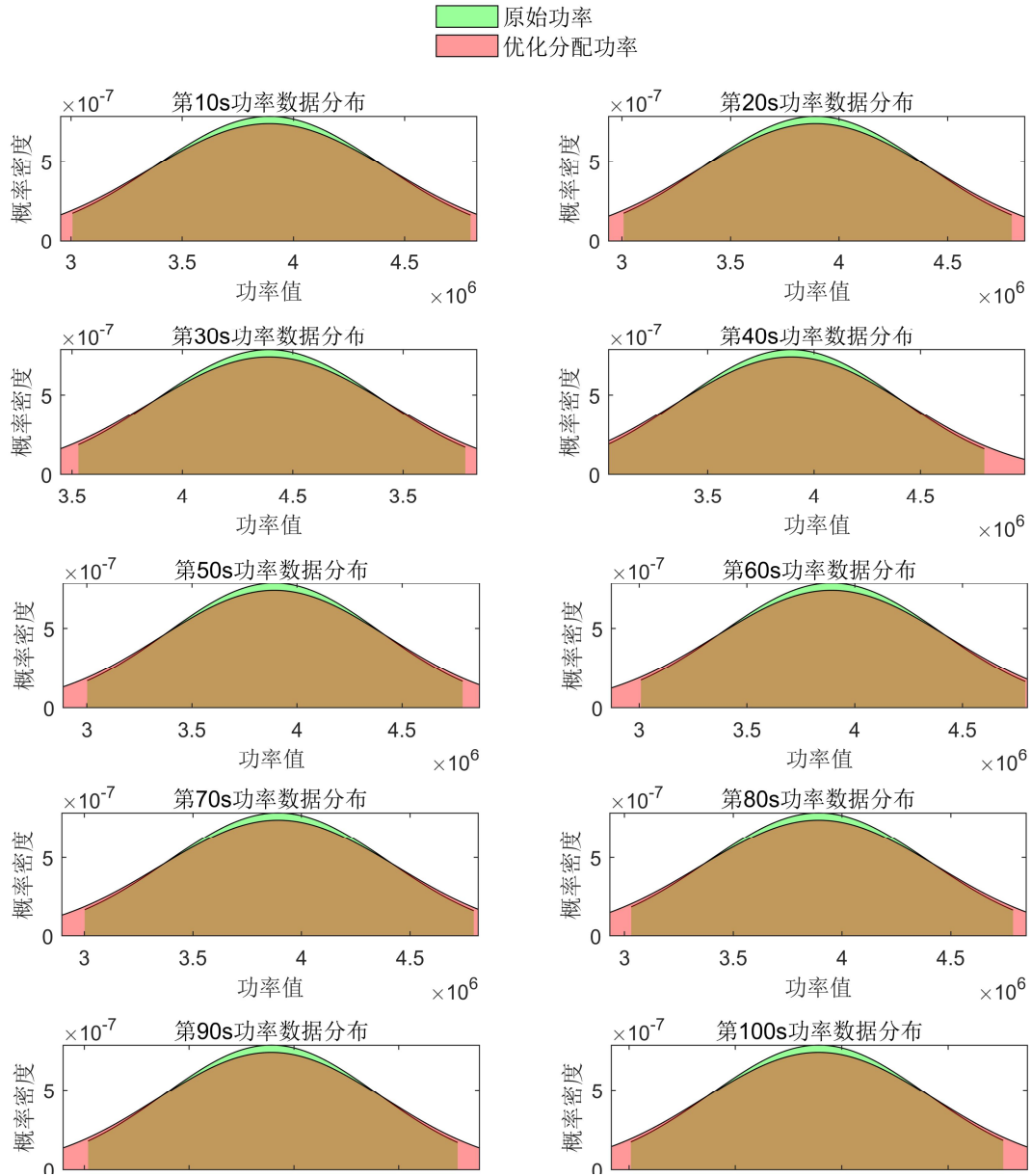


图35 优化前后不同时刻下的功率数据分布对比图

图35为优化前后不同时刻下的功率数据分布对比图，优化后的功率分布范围大于原始分布，这种宽广的功率分布范围允许系统在面临高负载时，迅速响应并提供足够的能量，从而避免因瞬时负载激增而导致的停机或性能下降。

图36为优化前后不同时刻下所有风机的功率分配对比图，**基于优化模型得到的功率分配方案，可保证风机间功率方差显著降低**，这一现象表明风电场整体的功率输出更加稳定，波动性减小，意味着在负载变化或外部干扰时，系统整体能够更好地保持一致的性能。稳定性提升的优势在于降低了设备的磨损和故障风险，提高了整个系统的可靠性。此外，方差的降低使得资源的利用更加高效，能减少因过载或不足带来的能量浪费，进而提升整体能效。

上述结果表明，本文所建立的有功功率优化分配模型和求解方法不仅提高了系统的稳定性与响应能力，还增强了对变动负载的适应性，充分体现了本文所提模型和应用的求解方法的有效性与必要性。

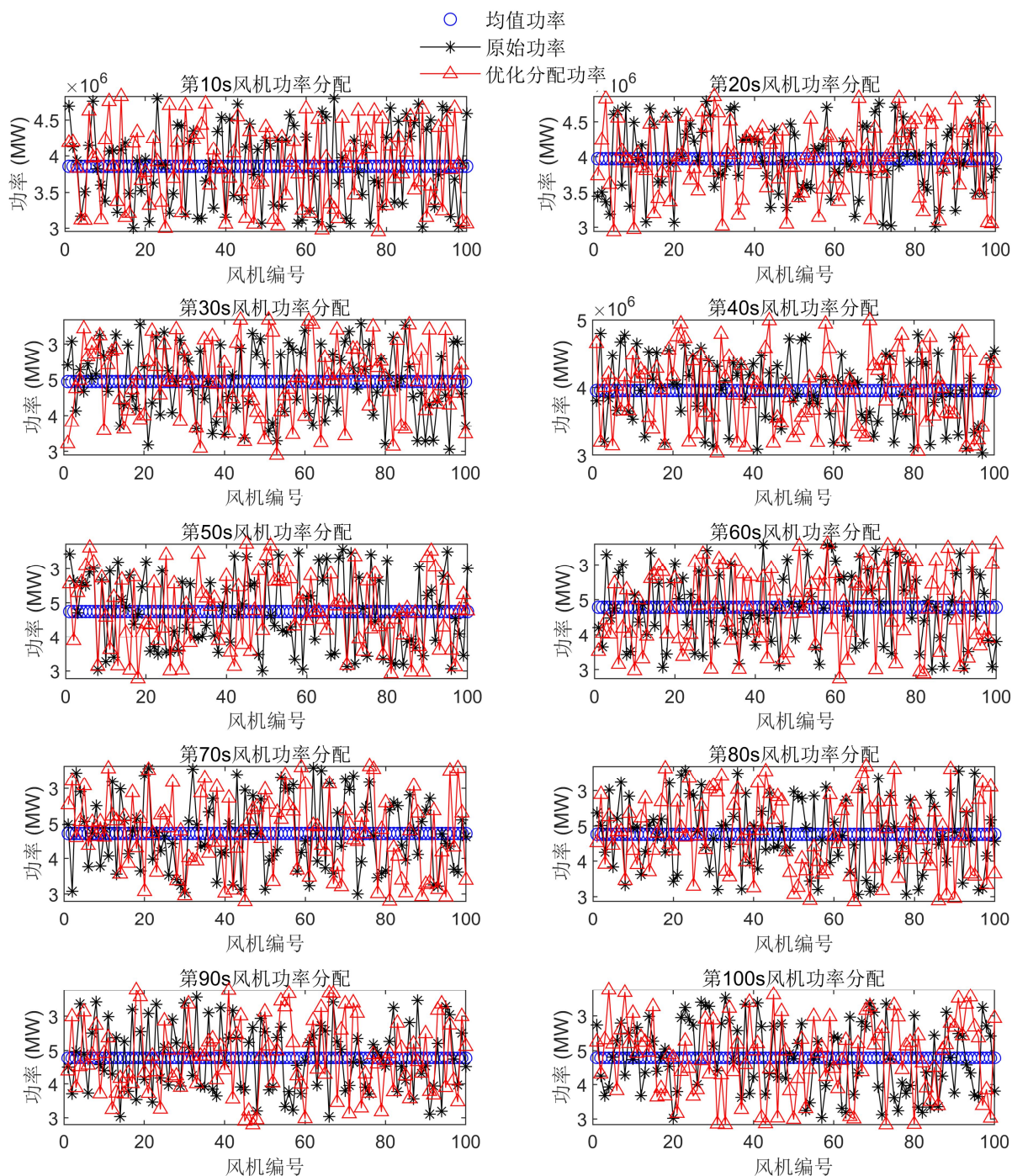


图36 优化前后不同时刻下所有风机的功率分配对比图

5.6. 问题小结

问题三要求基于题目所述的优化目标和约束条件，建立有功调度优化分配模型并进行实时求解，得到风电场内的最优有功功率分配策略。

针对此问题，本文首先在问题一和问题二的基础上构建了风机累积疲劳损伤程度计算模型，实现对每个运行周期内风电场内所有风机的累积疲劳损伤程度计算；随后，以当前运行周期内所有风机的主轴和塔架的总体疲劳损伤程度最小为目标函数，考虑功率平衡、风机出力和调节精度等约束条件，建立风电场有功功率优化分配模型；然后，通过多目标非支配排序遗传算法进行模型求解，实现风电场内各风机有功功率的优化调度。

和实时更新，保证各个风机的疲劳损伤程度和功率分配的合理性；最后，分析了所求功率优化分配方案可以满足计算实时性和约束条件，将所求功率优化分配方案的运行效果与平均功率分配方案进行对比，验证建立的优化模型和模型求解方法的有效性。

6. 问题四的模型建立与求解

6.1. 问题分析

问题四要求随机测量噪声和通信延迟对于模型精度和优化问题最优性的影响，完善问题三中的优化方案。问题四的研究思路流程图如下图所示：

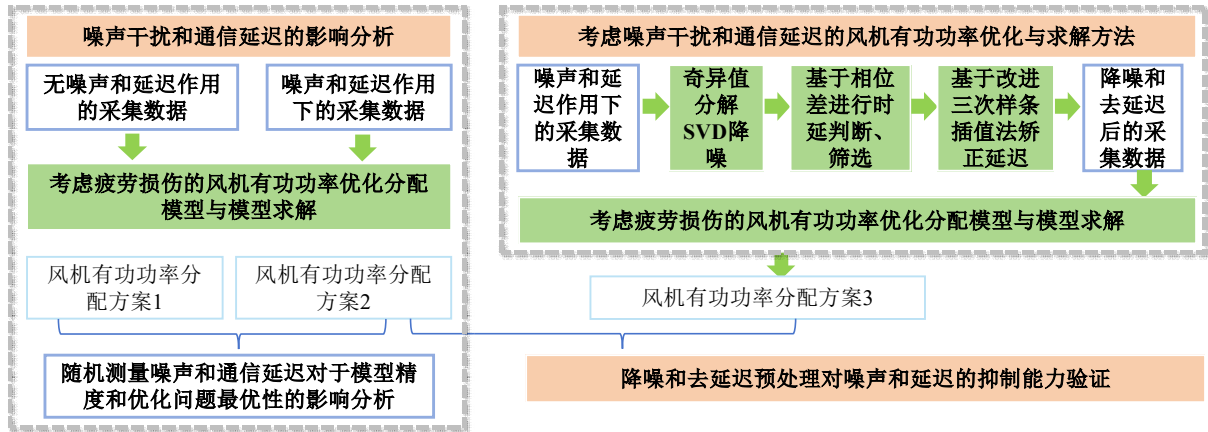


图37 问题四思路流程图

针对问题四，首先，将附件4提供的噪声干扰和通信延迟作用下的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将计算结果与问题三的优化结果进行对比，分析噪声干扰和通信延迟为计算精度和优化效果带来的影响；随后，针对噪声和延迟问题设计优化方案，对采集数据进行合理的预处理，其中，针对噪声干扰问题，采用奇异值分解SVD进行降噪处理，针对通信延迟问题，基于相位差进行时间序列的滞后性判断、筛选，并基于改进三次样条插值法进行缺失数据恢复；最后，将经过预处理后的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将得到的计算结果与基于未处理的采集数据的计算结果、基于无噪声和延迟的采集数据的计算结果进行对比，验证基于降噪和去延迟的预处理环节，可以有效消除噪声干扰和通信延迟为计算精度和优化效果带来的影响。

6.2. 考虑噪声干扰和通信延迟的风机有功功率优化与求解

6.2.1. 基于奇异值分解SVD的噪声干扰处理

奇异值分解（Singular Value Decomposition，简称SVD）是一种常用的矩阵分解方法，将一个矩阵分解为三个矩阵的乘积，分别是一个正交矩阵、一个对角矩阵和另一个正交矩阵的转置^[10]。

$$A = U \Sigma V^T = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T$$

图38 奇异值分解SVD原理图

给定一个矩阵 A ，其SVD分解为 $A = U\Sigma V^T$ ，其中 U 和 V 是正交矩阵， Σ 是对角矩阵，对角线上的元素称为奇异值。奇异值按照大小排列，从大到小排列在对角线上。SVD的主要思想是将原始矩阵 A 在特征值分解的基础上进行推广，将非方阵也能进行分解，进而实现矩阵的降维、压缩、去噪等操作。因此，在图像处理、信号处理、文本处理等领域，SVD常常被应用于数据降维和降噪处理，具有广泛的实际应用价值。

SVD在降噪中的具体实施步骤：

- 1) 将需要进行降噪处理的数据构建成一个矩阵 A （通常是一个数据矩阵，比如图像数据矩阵）；
- 2) 对矩阵 A 进行SVD分解，得到矩阵 U 、对角矩阵 Σ 和矩阵 V^T ；
- 3) 选择一个合适的截断奇异值的数量或阈值，将 Σ 矩阵中较小的奇异值置为0，实现降维；
- 4) 使用截断后的 U 、 Σ 和 V^T 重构原始矩阵，得到一个降噪后的矩阵 A' ；
- 5) 可以根据需要对 A' 进行后续处理，比如进行逆变换、反归一化等。

通过SVD的降维处理，可以有效地去除原始数据中的噪声，得到更加干净、简洁的数据表示。

6.2.2. 基于相位差和改进三次样条插值法的通信时延处理

针对数据的通信延迟问题，首先采用基于相位差的时间序列信号滞后相关性判断进行通信延迟识别、筛选，随后基于改进三次样条插值进行缺失数据恢复，从而对通信延迟进行有效校正，达到去延迟的效果。

(1) 基于相位差的时间序列信号滞后相关性判断

相位差是衡量两个信号之间时间延迟的一种方式，它反映了信号之间的时间差异或超前滞后关系。通过测量相位差的方法，如过零相位检测法、相关函数法(correlation function)和傅里叶变换法，可以计算信号之间的相位差。这种方法适用于对时域信号进行采样，生成时间序列，并通过计算信号的互相关找到滞后时间，进而转换为角度得到信号的相位差^[11]。

使用相位差判断与计算时间序列延时的具体步骤为：

- 1) 获取需要比较的两个时间序列，假设分别为 $x(t)$ 和 $y(t)$ ；
- 2) 对两个时间序列进行傅里叶变换，得到频域表示 $X(f)$ 和 $Y(f)$ ；
- 3) 计算两个信号的复数傅里叶系数之间的相位差： $phase_diff = arg(Y(f)) - arg(X(f))$ 其中， $arg(\cdot)$ 表示求取复数的幅角；
- 4) 将相位差转换为时间延迟： $time_diff = phase_diff / (2\pi f)$ 其中， f 为信号的频率， $2\pi f$ 表示信号的周期；
- 5) 得到两个时间序列之间的时间延迟。



图39 相位差判断与计算时延原理图

(2) 基于改进三次样条插值的缺失数据恢复

改进三次样条插值是一种插值方法，用于在已知数据点之间的区间内插值缺失数据。三次样条插值可以通过构造分段三次多项式来逼近数据点之间的曲线，并保证插值结果在每个区间内具有良好的光滑性和连续性。

改进三次样条插值的缺失数据恢复的原理是在传统的三次样条插值方法基础上，引入更多的信息和约束条件，以提高插值结果的准确性和稳定性。这可以通过引入额外的约束条件，如数据间的相关性、趋势等，或者使用更复杂的插值模型来实现^[12]。

具体实施步骤如下：

1) 数据预处理：首先，需要对缺失数据进行处理，包括查找缺失数据的位置、确定插值区间等。同时，对已有的数据点进行预处理，去除异常值、平滑数据等；

2) 构建插值模型：选择适当的插值模型，可以是改进的三次样条插值方法，也可以是其他插值方法如多项式插值、拉格朗日插值等。根据实际情况，可以在插值模型中引入额外的约束条件或信息；

3) 插值计算：利用已知数据点和构建的插值模型，对缺失数据进行插值计算。根据插值模型的具体形式，可以使用数值方法或优化算法来计算插值结果；

4) 插值结果评估：对插值结果进行评估，可以通过与真实值的比较、误差分析等方法来评估插值结果的准确性和稳定性；

5) 参数调整和优化：根据插值结果的评估情况，对插值模型的参数进行调整和优化，以进一步提高插值结果的质量；

6) 结果验证和应用：验证改进的三次样条插值方法在缺失数据恢复中的效果，并根据实际需求应用插值结果，可以进一步分析插值结果的可靠性和适用性，对实际问题进行进一步的探讨和研究。

通过以上步骤，可以利用改进的三次样条插值方法来恢复缺失数据，提高数据的完整性和可用性。在实际应用中，需要根据具体问题和数据特点来选择合适的插值方法，并不断优化和调整插值模型，以获得更准确和可靠的插值结果。

6.2.3. 考虑噪声干扰和通信延迟的风机有功功率优化方法

本文基于奇异值分解SVD降噪法、相位差判断时延法、改进三次样条插值法，提出考虑噪声干扰和通信延迟的风机有功功率优化方法，具体步骤如下：

1) 数据预处理：提取噪声干扰和通信延迟作用下的采集数据，提取其中的风速、功率参考值等输入数据，形成时间序列；

2) 降噪处理：基于奇异值分解SVD对时间序列进行降噪处理，得到降噪后的时间序列数据；

3) 去延迟处理：对降噪后的时间序列数据，利用相位差的时间序列信号滞后相关性判断进行通信时延的识别、筛选，随后基于改进三次样条插值进行缺失数据恢复，从而对通信延时进行有效校正，得到去延迟后的时间序列数据；

4) 优化方案求解：将降噪、去延迟处理后的时间序列数据代入问题三建立的考虑疲劳损伤的风机有功功率优化分配模型并进行模型求解，得到基于降噪和去延迟处理的风机有功功率优化分配方案；

5) 有效性检验：将得到的优化分配方案与无噪声和延迟作用的采集数据得到的优化方案进行对比，验证所应用的降噪和去延迟方法的有效性；

6) 必要性检验：将得到的优化分配方案与有噪声和延迟作用的采集数据得到的优化方案进行对比，验证所应用的降噪和去延迟方法的必要性。

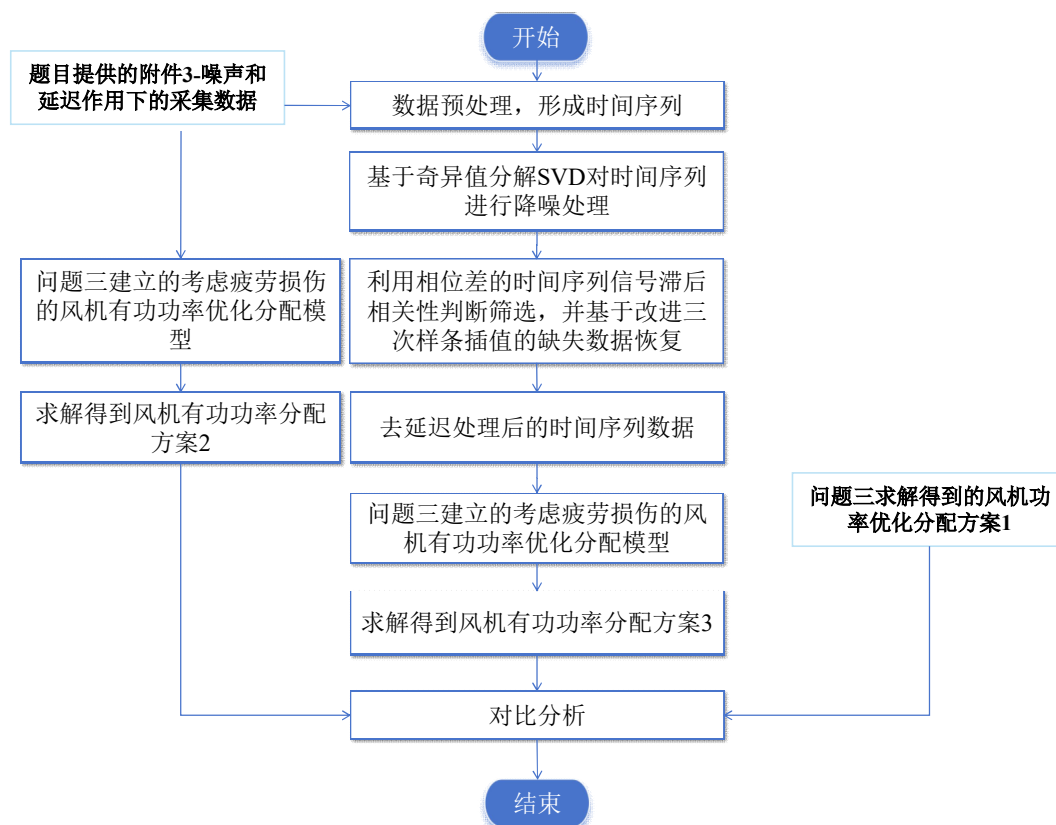


图40 考虑噪声干扰和通信延迟的风机有功功率优化流程图

6.3. 结果分析

6.3.1. 计算结果实时性展示

基于MATLAB进行程序编写和执行，对程序的执行时间进行动图记录，展示计算结果的实时性。由于文档对于动图的展示限制，仅在文档中展示所录制动图的截图，程序运行时间的动图另附在附件中单独上传。图41和图42分别为进行降噪和去延迟处理前后的100s功率优化分配计算的运行过程的动图记录，不难看出，基于经过降噪和去延迟处理后的数据进行优化模型求解，风机的主轴和塔架的疲劳损伤程度增长过程更为平缓，受噪声干扰和通信延迟的影响更小，验证了本文所应用的降噪和去延迟方法的有效性。

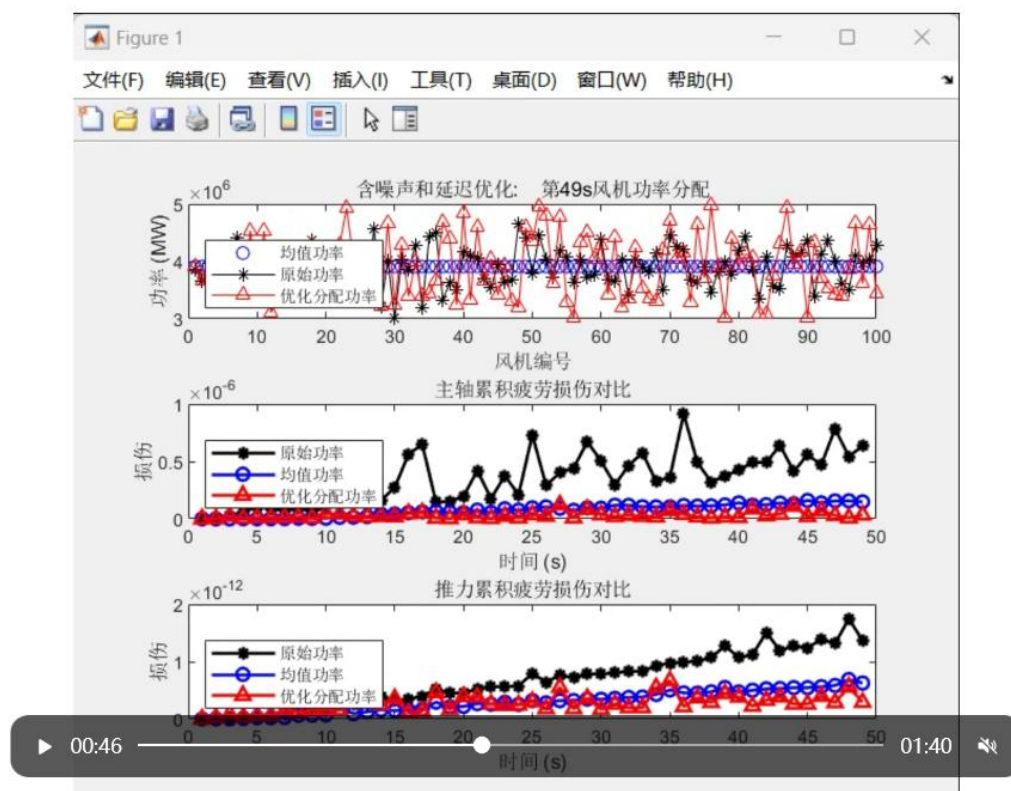


图41 噪声干扰和通信延迟作用下的风机有功功率优化方案求解过程截图

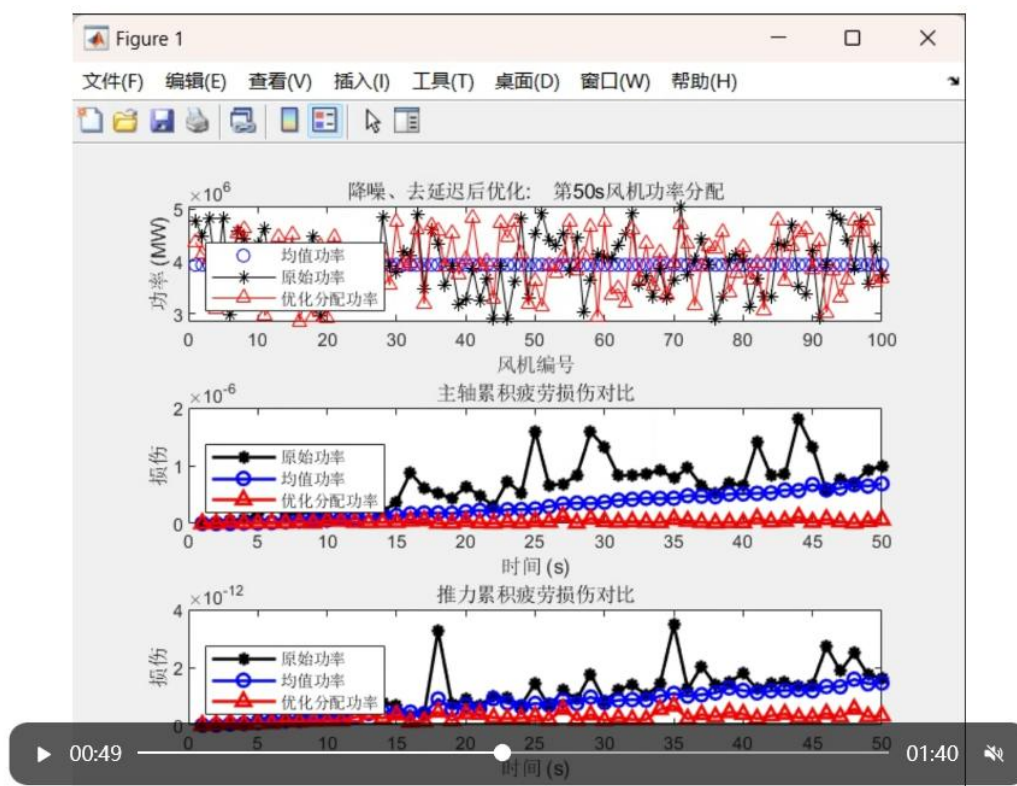


图42 降噪和去延迟处理后的风机有功功率优化方案求解过程截图

6.3.2. 基于奇异值分解SVD的降噪效果

对比风速、功率调度参考值、主轴扭矩、塔架推力、输出功率等输入输出数据在降噪前后的信噪比情况如下表4所示，可以看出，奇异值分解SVD可以有效提升数据的信噪比，且降噪后的信噪比与原始数据误差极小，验证了该方法的有效性。

表4 输入输出数据在降噪前后的信噪比对比

对象	数据集	信噪比 (dB)
风速	原始数据	77.4773
	含噪声数据	74.5945
	降噪后	77.5239
功率调度指令	原始数据	70.5187
	含噪声数据	69.7542
	降噪后	72.6601
扭矩	原始数据	70.9899
	含噪声数据	70.1582
	降噪后	71.0886
推力	原始数据	62.8627
	含噪声数据	62.7110
	降噪后	62.9132
输出功率	原始数据	70.6934
	含噪声数据	69.8678
	降噪后	70.8280

具体而言，以风速的数据为例，通过信噪比比较和降噪前后的数据点对比，对SVD的降噪效果进行分析展示。

图43为SVD降噪处理对于风速数据的信噪比影响展示图，该图对三种场景下的数据信噪比进行比较：不含噪声的原始风速数据、含噪声的风速数据、经过降噪处理之后的风速数据。可以看出，含噪声的风速数据信噪比为74.5945dB，通过SVD降噪处理之后的风速数据信噪比为77.5239dB，SVD降噪处理可以将数据的信噪比提升2.9294dB。

图44为SVD降噪前后的前100s的风速数据与无噪声数据对比图，该图是前100s风速降噪前后的曲线对比图，可以看到降噪之后的曲线和不含噪声的原始曲线趋势更接近，验证了该降噪方法的有效性。

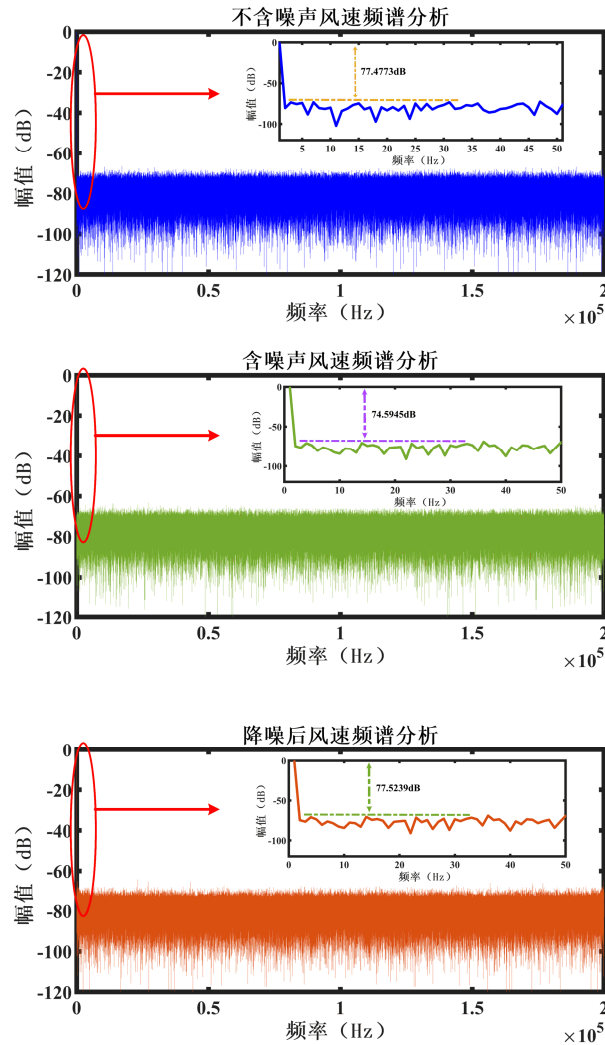


图43 SVD降噪处理对于风速数据的信噪比影响展示图

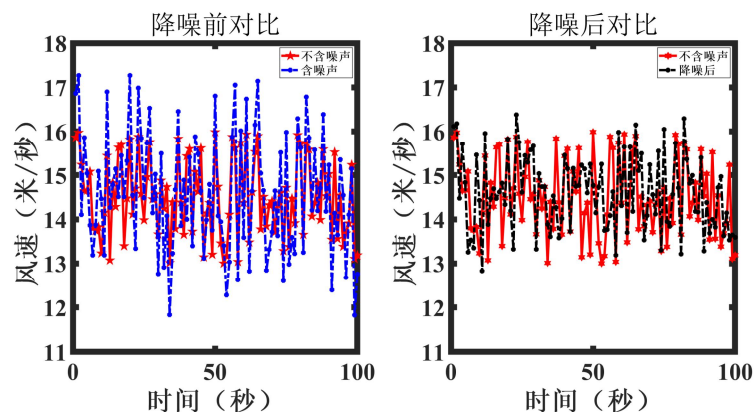


图44 SVD降噪前后的前100s的风速数据与无噪声数据对比图

6.3.3. 基于相位差和改进三次样条插值法的去延迟效果

(1) 基于相位差的数据延迟筛选

图45是相位差的直方图分布情况，风机WT1的2000s扭矩数据为例，看到大部分数据是没有滞后现象的，有少部分数据存在滞后的情况。如表5所示，在2000个数据中选取滞

后性最严重的97个数据进行分析，占比3.4%，计算该97个数据的滞后时间，并将其与延迟前的数据进行对比，发现综合差异度为15.10%。

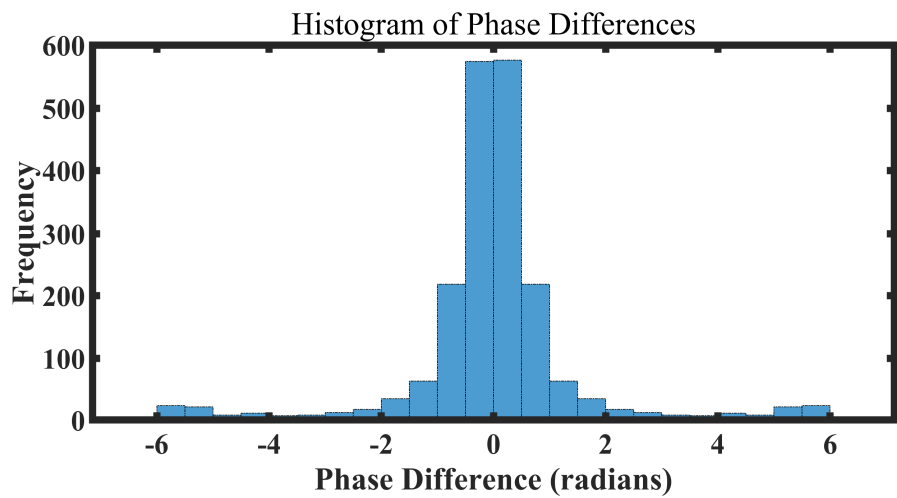


图45 相位差的直方图

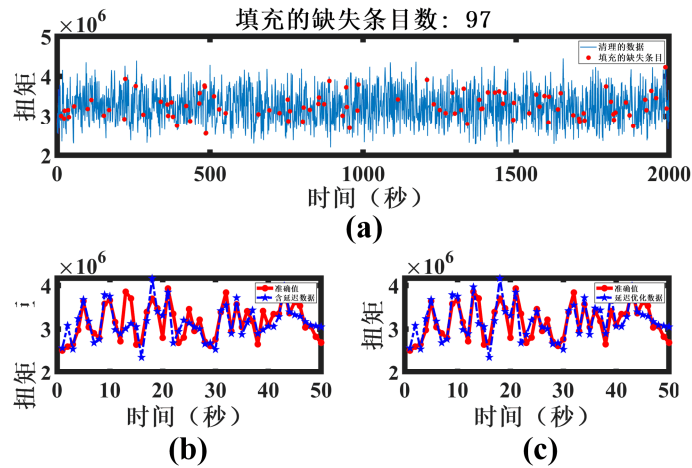
表5 基于相位差的时间序列延迟筛选列表

时间（s）	滞后性时间（s）	与延迟前数据差异度（×100%）
34	4.90	0.16
166	5.14	0.24
170	4.58	0.01
230	4.88	0.40
247	4.18	0.13
289	4.64	0.25
299	4.44	0.15
322	4.76	0.04
341	3.76	0.17
455	4.42	0.47
500	4.54	0.10
526	3.61	0.09
543	3.62	0.12
565	3.69	0.10
571	3.73	0.08
582	4.89	0.13
583	4.54	0.13
599	4.70	0.09
...	...	...
1919	3.49	0.19
1925	4.34	0.30
1954	4.92	0.10
1974	4.63	0.12
1998	4.51	0.28

（2）基于改进三次样条插值法的缺失数据恢复

图46是基于风机1数据筛选出来的主轴扭矩延迟数据基于改进三次样条插值法进行去延迟校正前后的对比图。其中，图(a)是时间总长2000s的曲线图，红色的点表示发生延迟的位置；图(b)和(c)分别是含延迟数据和去延迟处理后的数据与准确数据的对比曲线，可以看到优化后的曲线重叠度更高。图47为含时延的扭矩数据进行去延迟优化前后的评价指标对比，将其与准确值进行对比，分别得到MAPE、MSE、 R^2 的结果，可以看出，经去延迟优化后的三种评价指标均比优化前要好，证明了优化模型的有效性

综上所述，改进三次样条插值法可有效恢复数据延时导致的缺失数据。



(a)2000s内延迟条目标注图

(b)含延迟数据与准确数据的对比曲线

(c)去延迟处理后的数据与准确数据的对比曲线

图46 基于改进三次样条插值法对主轴扭矩延迟数据进行去延迟校正前后的对比图

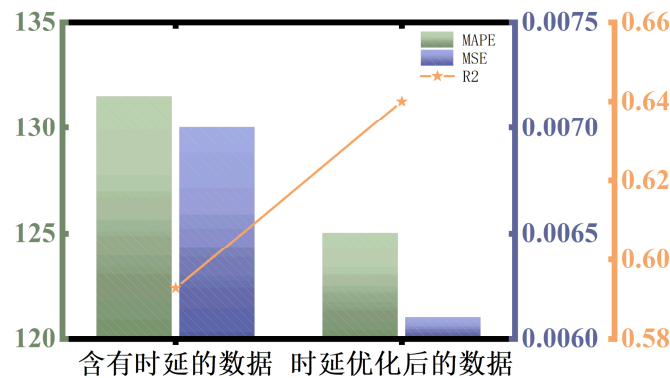


图47 基于改进三次样条插值法据进行去延迟校正前后的评价指标对比图

6.3.4. 噪声干扰与通信延迟对功率分配优化结果的影响分析

含噪声和延迟优化风机分配功率总和检验如图48所示，图中为原始调度功率总和与优化分配功率总和的折线图。从图中可以看出两条折线几乎重合，说明优化方法在噪声与延迟的影响下，依然能够保持功率的稳定性。这表明，通过优化分配算法，有效减小了因信号延迟和噪声带来的误差，使得调度的可靠性显著提高。第二个子图展示了两者的误差在 10^{-7} 次方，说明优化分配结果基本满足要求。

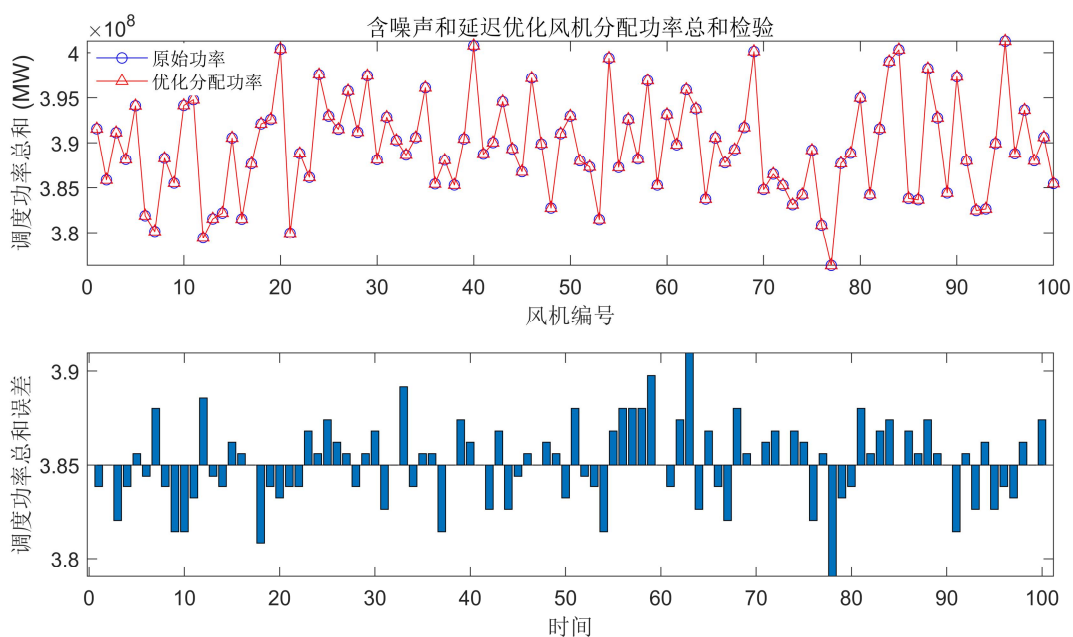


图48 含噪声和延迟优化风机分配功率总和和检验

含噪声与延迟下主轴累积疲劳损伤对比如图49所示，结果显示原始功率下的主轴累积疲劳损伤值随时间增长最快，均值功率次之，而优化分配功率的增长速度最慢。这种现象可以归因于噪声与延迟对功率波动的影响，在原始功率分配中，因缺乏有效的应对策略，风机可能遭受更大的负荷波动，从而导致疲劳损伤加剧。均值功率虽然在某种程度上平滑了负荷，但依旧无法有效应对突发的噪声和延迟。相对而言，优化分配方法通过智能调度，能够在不同风况下实时调整功率分配，减少极端负荷，从而显著降低疲劳损伤。

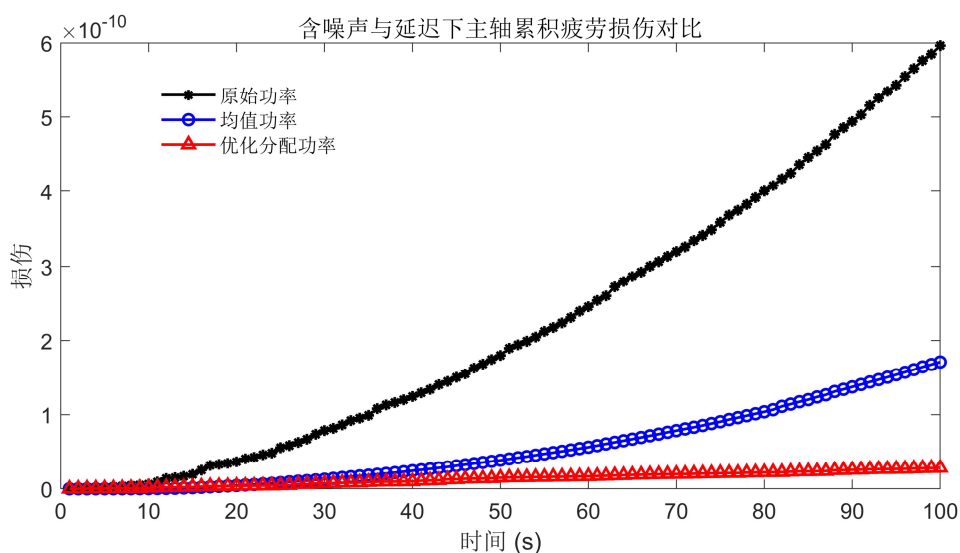


图49 含噪声与延迟下主轴累积疲劳损伤对比

图50同样展示了推力累积疲劳损伤的对比，原始功率的损伤增长较快，但均值功率在前期损伤较低，后期逐渐增大，这可能是因为均值功率在初期能较好地应对突发噪声，但长期看仍难以持续优化。而优化分配功率始终保持最低的损伤值，表明其在应对推力波动时具有更优的调节能力，这进一步证明了优化方法在面对噪声与延迟时的优势。

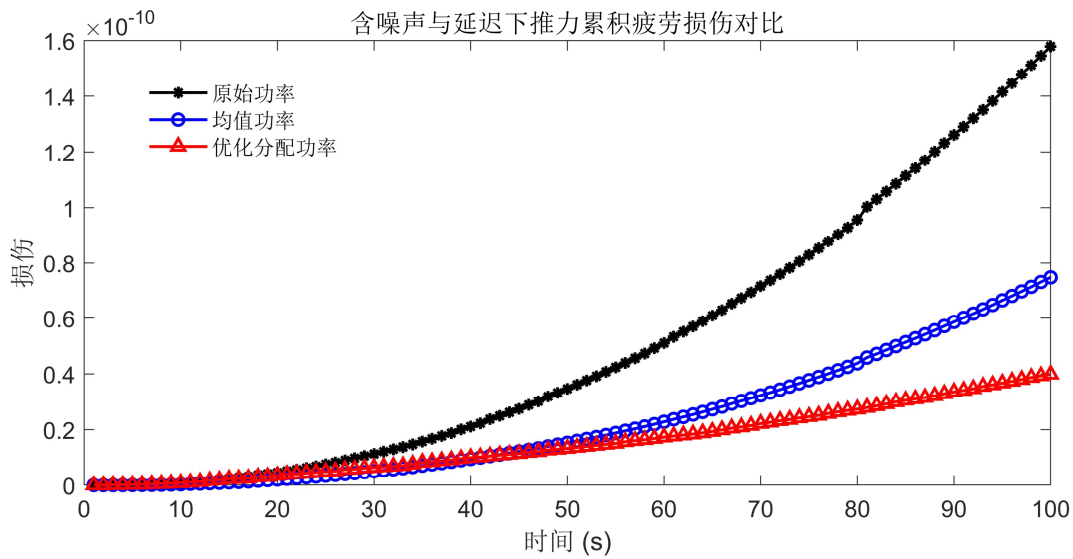


图50 含噪声与延迟下推力累积疲劳损伤对比

在风机调度功率优化中，考虑到含噪声与信号延迟的影响，图51展示了原始功率与优化分配功率的方差特性。原始功率的方差较低，表明其输出较为稳定，且受噪声影响较小。而优化分配功率的方差显著高于原始功率一个数量级，这主要是由于优化过程中对系统动态特性的敏感性增强。优化算法在处理实时数据时，受噪声干扰可能导致功率分配的剧烈波动，且信号延迟使得系统对变化的响应不断调整。这样的反馈机制在优化过程中可能导致过度调整，从而增加功率分配的方差。因此，在实际应用中，需平衡优化策略与系统稳定性，以减少噪声与延迟对功率分配的负面影响。

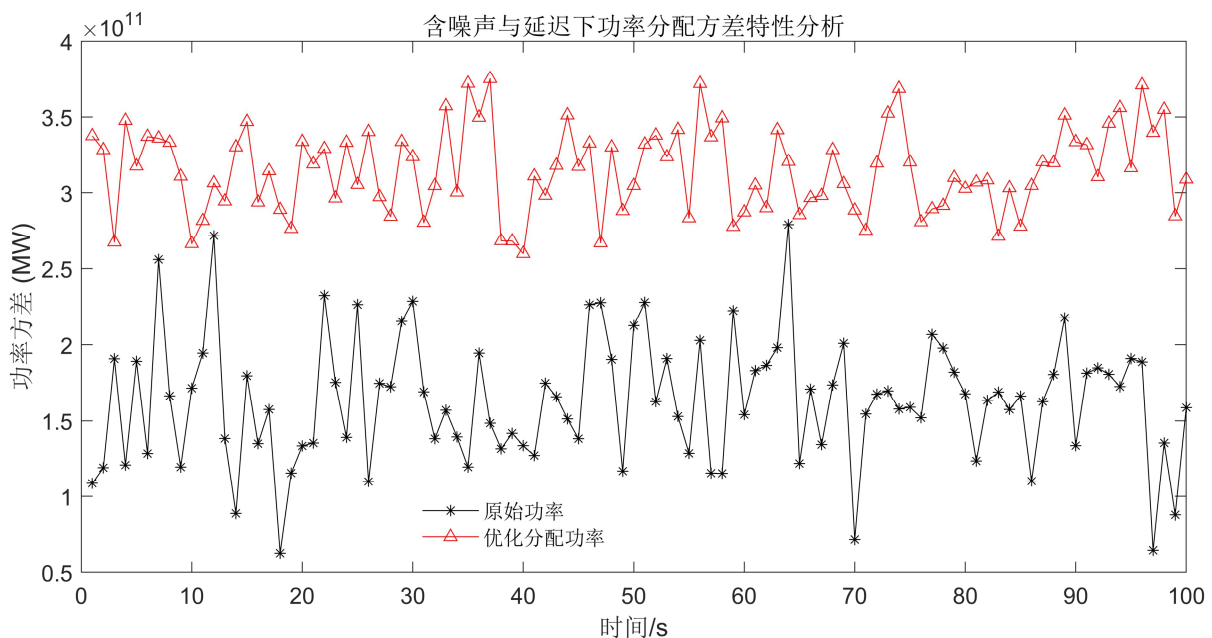


图51 含噪声与延迟下功率分配方差特性分析

6.3.5. 降噪和去延迟前后的功率分配优化结果对比

在风机调度功率优化的研究中，降噪和去延迟的效果显著提升了系统的性能。以下是具体的结果分析及其原因探讨。

图52展示了降噪、去延迟后优化风机分配功率的总和检验。其两个子图阐明了原始调度功率与优化分配功率之间的一致性。第一子图中，原始功率和优化功率的折线图吻合，说明优化手段在没有显著改变总功率的情况下，有效提升了调度的精度和稳定性。第二子图的差值在 7×10^{-7} 以内，进一步反映了优化分配对功率调度的精细化管理，使得系统在噪声环境下保持较高的稳定性，降低了因外部干扰引起的误差。

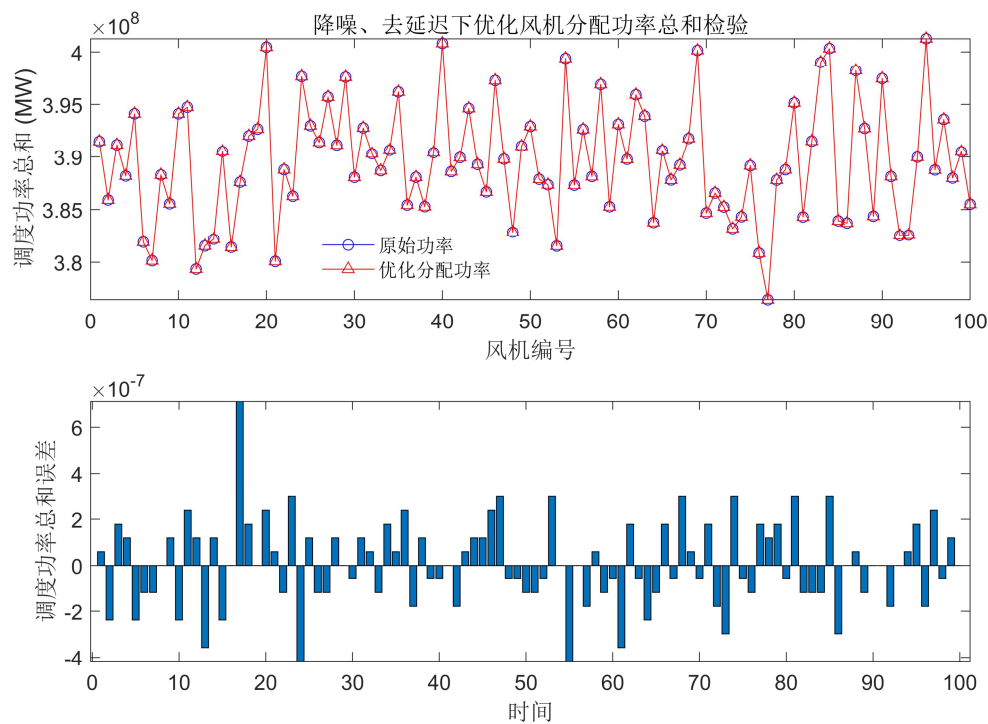


图52 降噪、去延迟下优化风机分配功率总和检验

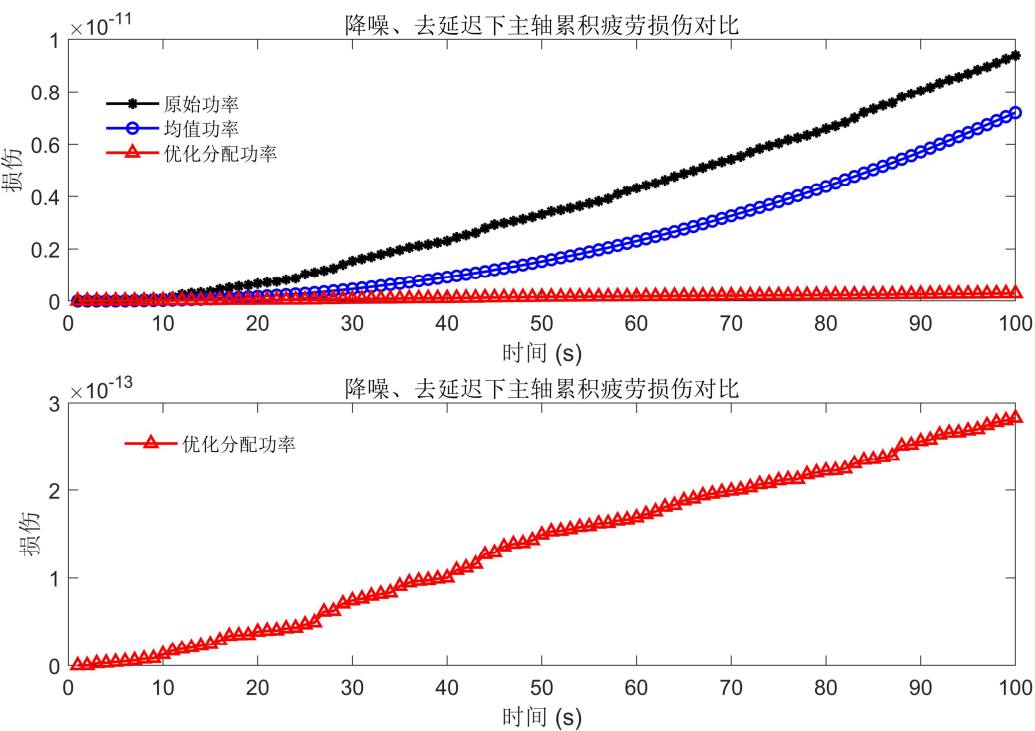


图53 降噪、去延迟下主轴累积疲劳损伤对比

图53则关注主轴的累积疲劳损伤随时间变化的对比。三条曲线初期趋势一致，疲劳损伤值相近，反映了在初始阶段，所有功率分配方法对主轴的影响相似。但随着时间推移，优化分配功率的疲劳增长速度显著减缓，显示出优化方法在降低机械磨损方面的优势。此现象表明，优化分配不仅有效降低了功率波动，还在一定程度上分散了负荷，减少了主轴的疲劳积累。

在图54中，推力累积疲劳损伤的对比分析揭示了均值功率在前期的疲劳损伤较低，但后期增长显著，而优化分配功率始终保持最小的损伤值。此现象说明在噪声与延迟影响下，均值功率未能有效应对负荷波动，而优化分配通过动态调整功率分配，成功延缓了疲劳损伤的累积，展现出更优的抗干扰能力。

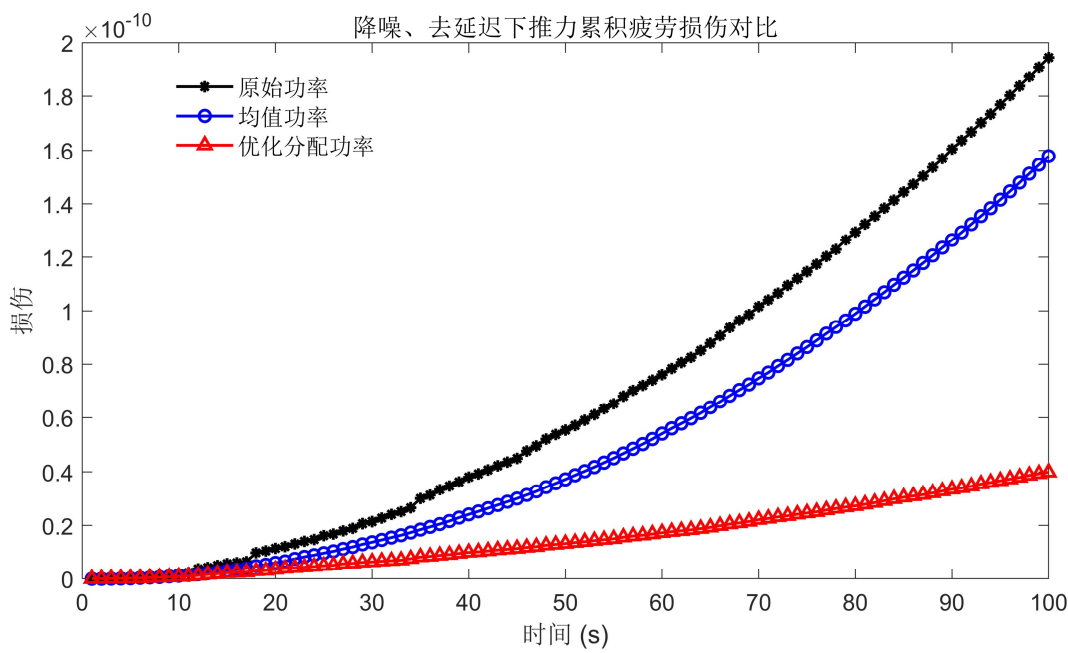


图54 降噪、去延迟下推力累积疲劳损伤对比

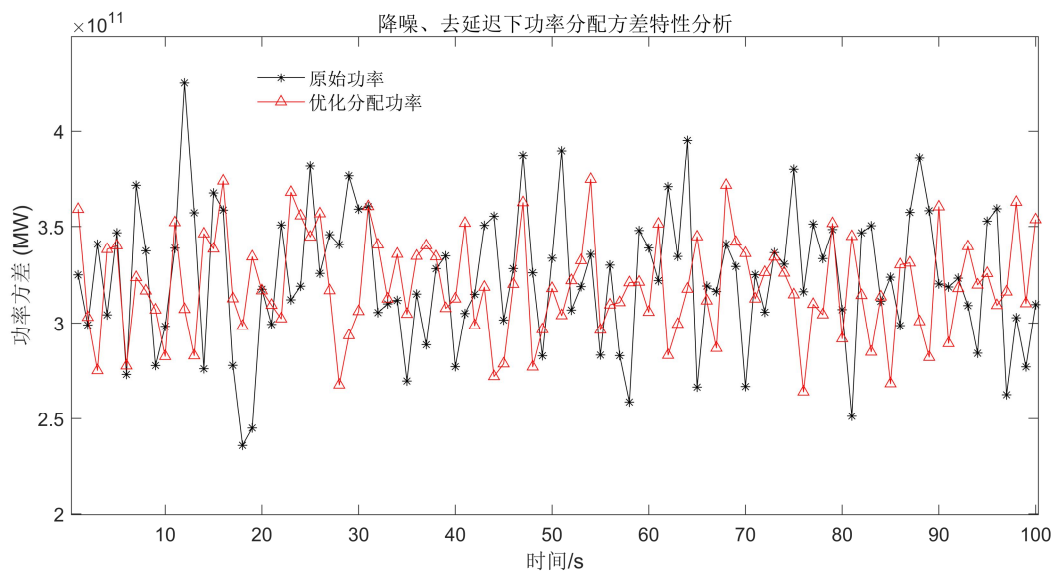


图55 含噪声与延迟下功率分配方差特性分析

图55分析了含噪声与延迟下的功率分配方差特性。原始功率与优化分配功率的方差曲线相近，但优化分配的方差波动幅度更小，反映出优化方法在功率分配上实现了更高的一致性和稳定性。较小的方差意味着在噪声条件下，优化分配能够更有效地控制功率的波动，降低系统的不确定性，提升整体运行的可靠性。

6.3.6. 模型鲁棒性检验

300s内十个风电场含噪声与延迟的功率总和检验为图56所示，第一个子图展示了原始调度功率总和与优化分配功率总和的折线图，两者曲线相吻合，表明在优化后，调度功率总和依旧维持在合理范围。第二个子图则显示了两者的差异，整体误差在 10^{-5} 次方，说明优化分配在处理噪声和延迟的情况下，能够有效降低功率分配的不确定性。这一现象表明优化分配方法在面对信号延迟和噪声时，能够更好地保持功率的稳定性。

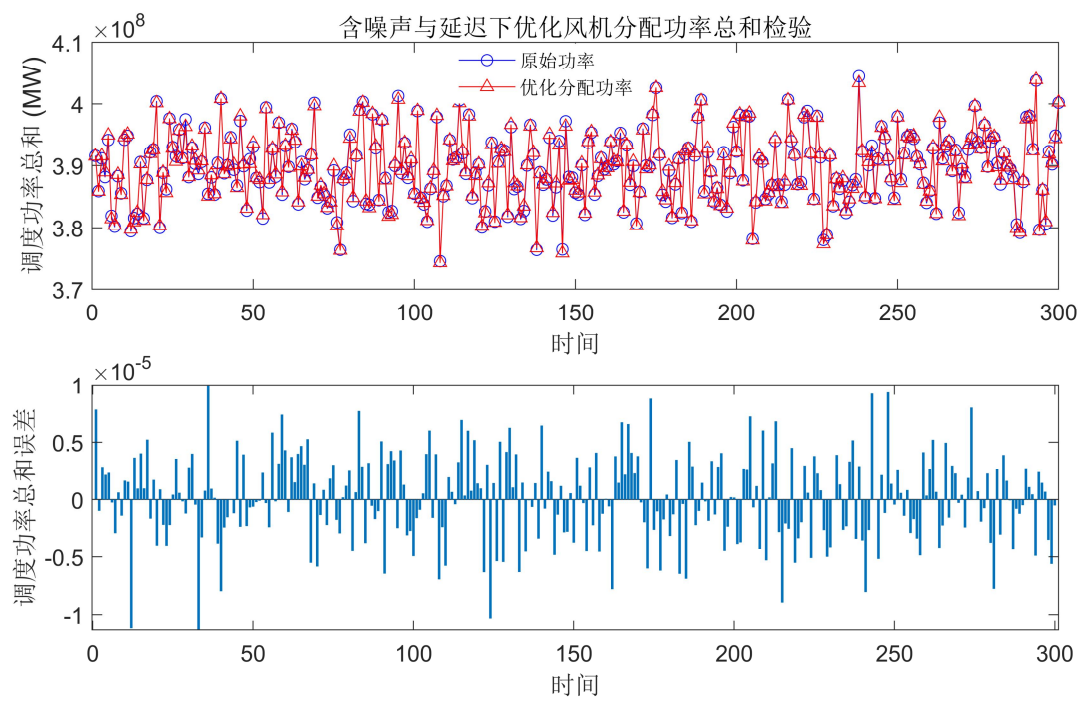


图56 含噪声和延迟优化风机分配功率总和检验

主轴累积疲劳损伤对比如图57所示，三条线分别代表原始功率、均值分配功率和优化分配功率的主轴累积疲劳损伤值。结果显示，在50秒之前，三者趋势较为接近，但随着时间推移，原始功率下的疲劳损伤值增长最快。这是因为原始功率在噪声与延迟下，可能导致瞬间过载，从而加速疲劳损伤的积累。均值功率虽有所改善，但仍难以应对突发变化。而优化分配功率则通过动态调整，减少了瞬时负荷的波动，因此增长速度最慢，有效控制了主轴的疲劳损伤。

推力累积疲劳损伤对比如图58所示，与图57类似，推力的累积疲劳损伤显示了同样的趋势，进一步确认了优化分配在平稳功率输出方面的优势。噪声和延迟使得推力的瞬时变化更为剧烈，优化分配通过合理调度，降低了整体推力的波动，从而减缓了疲劳损伤的积累速度。

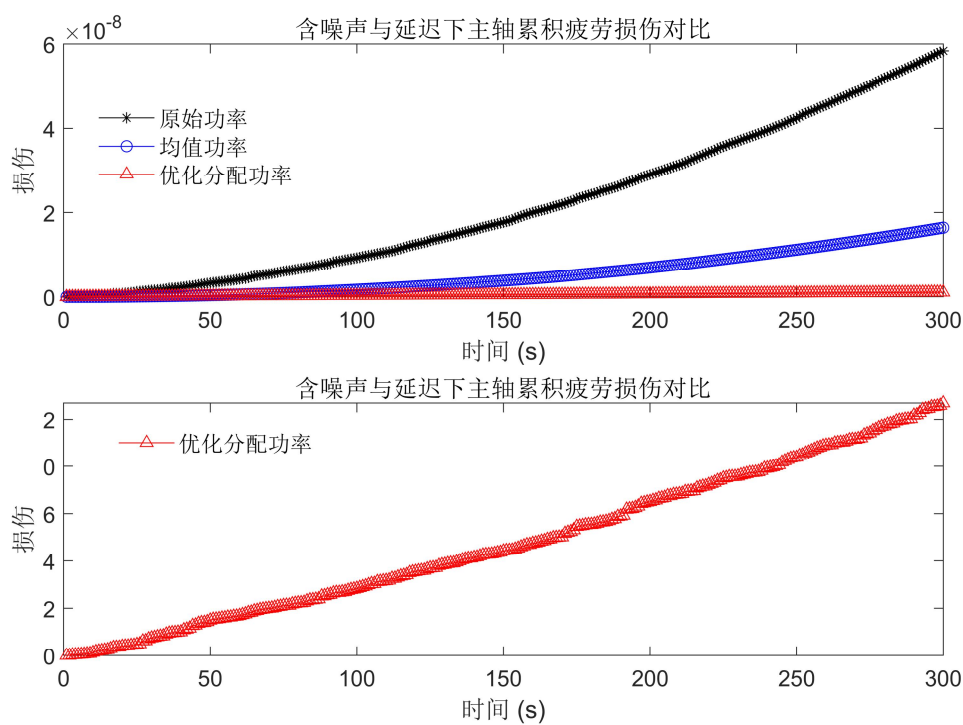


图57 含噪声与延迟下主轴累积疲劳损伤对比

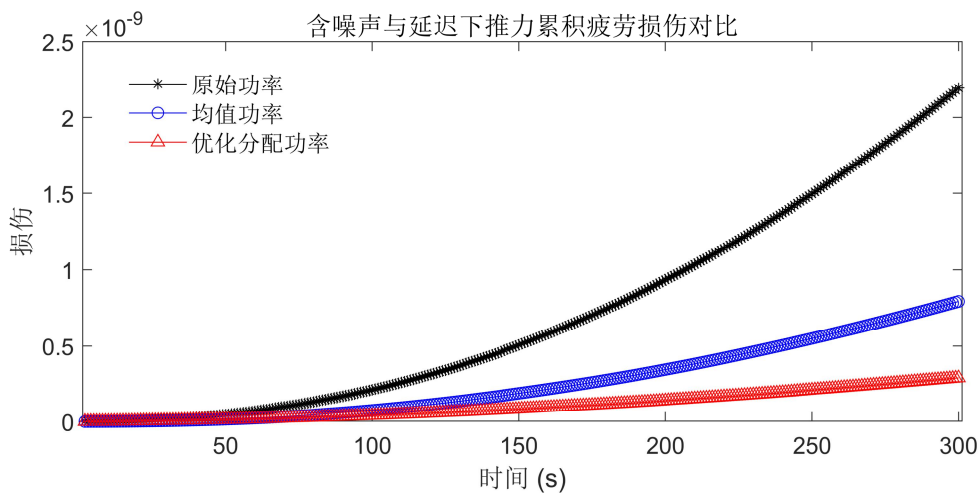


图58 含噪声与延迟下推力累积疲劳损伤对比

功率分配方差特性分析如图59所示,在该图中,优化分配功率的方差明显低于原始功率,显示了优化方法在降低功率分配不确定性上的显著优势。较低的方差意味着在含噪声与延迟的环境中,优化分配能够提供更稳定的输出,这对于风机的长期运行和可靠性至关重要。

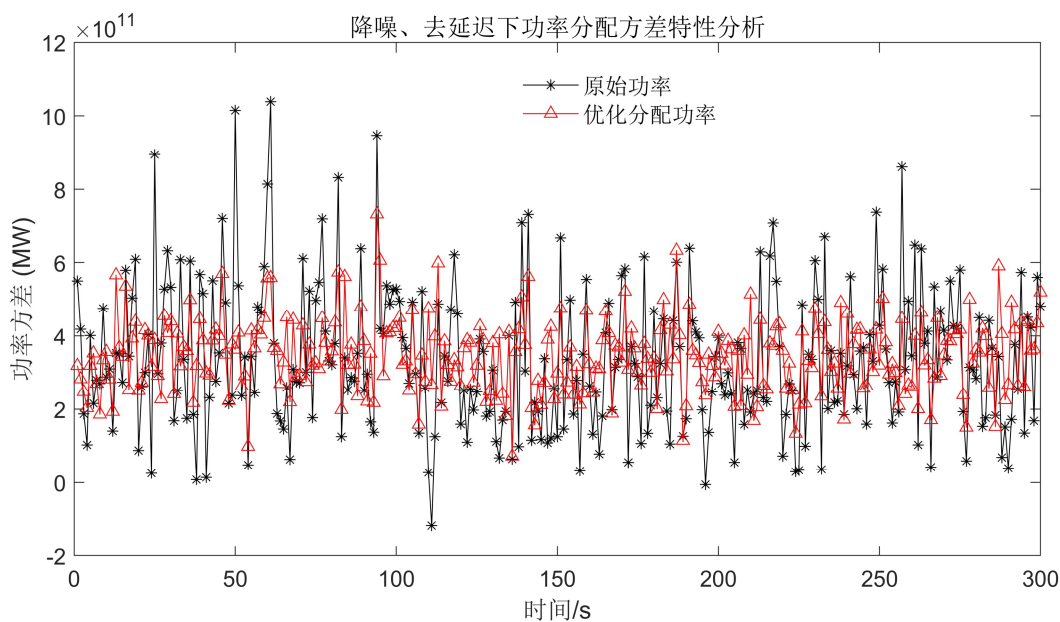


图59 含噪声与延迟下功率分配方差特性分析

降噪、去延迟结果对比:

图60展示了原始调度功率与优化分配功率的总和折线图，两个折线的吻合表明优化后的分配方式在总体功率的保持上具有一致性。这是因为优化策略在实现降噪的同时，确保了功率的平衡与调配的合理性。第二个子图显示了两条折线的差异，其总体误差小于 1.5×10^{-6} ，进一步证明了优化方法在降噪效果上的显著性，说明在减少干扰的过程中，能有效保持功率的稳定性。

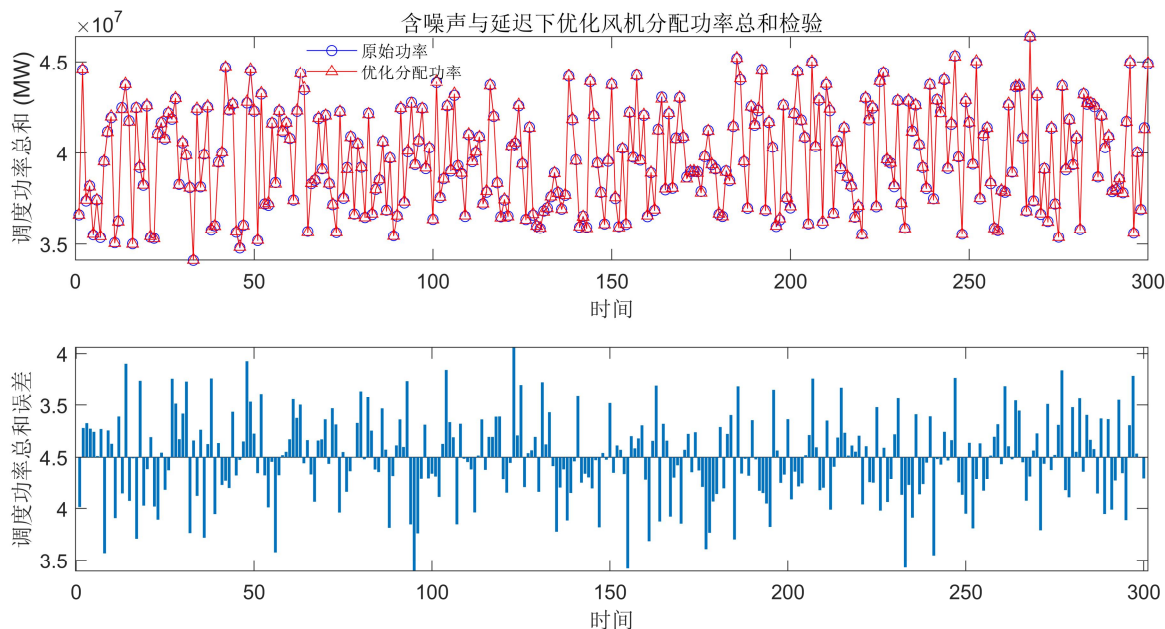


图60 降噪、去延迟下优化风机分配功率总和检验

图61则聚焦于主轴累积疲劳损伤的对比。原始功率、均值功率和优化分配功率的三条线初期趋势相似，这表示在短时间内，三者的影响相对均衡。然而，随着时间的推移，优化分配功率的疲劳损伤增长速度明显减缓，达到 10^{-14} 数量级。这一现象反映出，优化分

配策略在去延迟方面的优势，使得风机在运行过程中承受的负荷更为合理，从而有效延缓了疲劳损伤的累积。

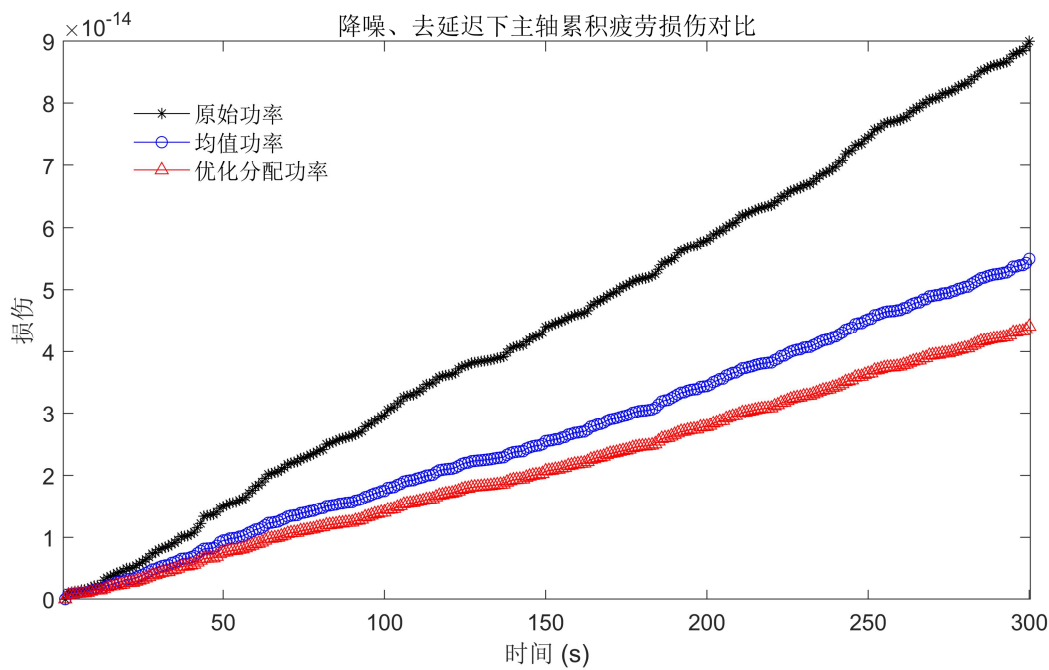


图61 降噪、去延迟下主轴累积疲劳损伤对比

图62进一步显示了降噪与去延迟对推力累积疲劳损伤的影响。在前期，均值功率与原始功率的推力损伤一致，表明两者在初始阶段的负荷分配相似。然而，随着时间的推移，均值功率逐渐低于原始功率，显示出优化后的功率分配在长期运行中的优势。优化分配功率的推力累积疲劳损伤值最小（数量级为 10^{-12} ），表明其在降低风机疲劳损伤方面的有效性。

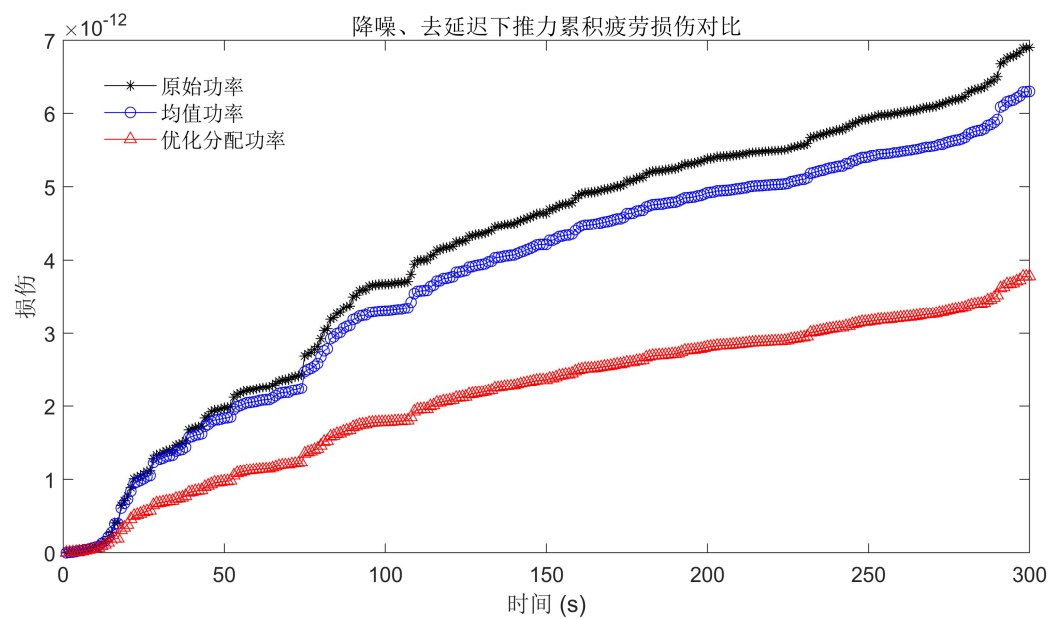


图62 降噪、去延迟下推力累积疲劳损伤对比

图63分析了功率分配的方差特性，显示出原始功率与优化分配功率的方差差异不大，但优化分配功率的方差波动范围更小。这意味着优化后的功率分配更加稳定，能够有效降低因风速变化或其他干扰因素引起的功率波动，从而实现更为平稳的运行状态。

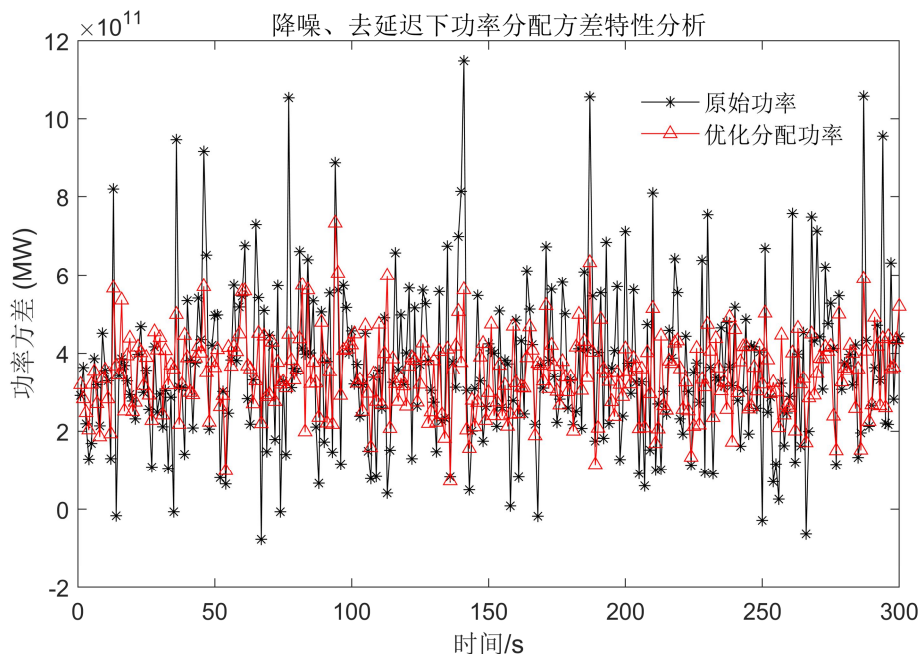


图63 降噪、去延迟下功率分配方差特性分析

综合以上结果，降噪与去延迟的优化方法显著提高了风机调度的安全性和效率，从时间长短的比较（300s与100s），可以看出在更长的跨时段内，优化方法展现出更为明显的优势，确保了风机在复杂环境下的稳定运行。通过这些分析，我们可以得出结论：优化的分配方法在提升风机工作效能、延长使用寿命方面具有重要意义，值得在实际应用中推广。

6.4. 问题小结

问题四要求随机测量噪声和通信延迟对于模型精度和优化问题最优性的影响，完善问题三中的优化方案。

针对此问题，本节首先将附件4提供的噪声干扰和通信延迟作用下的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将计算结果与问题三的优化结果进行对比，分析噪声干扰和通信延迟为计算精度和优化效果带来的影响；随后，针对噪声和延迟问题设计优化方案，对采集数据进行合理的预处理，其中，针对噪声干扰问题，采用奇异值分解SVD进行降噪处理，针对通信延迟问题，基于相位差进行时间序列的滞后性判断、筛选，并基于改进三次样条插值法进行缺失数据恢复；最后，将经过预处理后的采集数据输入问题三所建立的有功功率优化分配模型进行优化计算，将得到的计算结果与基于未处理的采集数据的计算结果、基于无噪声和延迟的采集数据的计算结果进行对比，验证基于降噪和去延迟的预处理环节的有效性和必要性。

将预处理后的采集数据作为有功功率优化分配模型的输入参数，所得优化结果与无噪声干扰和通信延迟的优化结果一致，证明了所提方法可以有效消除噪声干扰和通信延迟为计算精度和优化效果带来的影响，验证了本文所提出的基于奇异值分解SVD降噪和基于相位差判断处理时延的风机有功功率优化与求解方法的有效性。

7. 模型总结

7.1. 模型优点

(1) 本文在充分阅读问题的基础上, 针对每个子问题, 建立了对应的数学模型, 并对每个模型进行了充分的对比检验, 选择效果更好的方法进行结果处理和响应问题诉求, 所建立的模型可用于解决考虑疲劳损伤程度的风电场的有功功率优化分配问题, 具有现实意义, 所建立的模型具有极强的可解释性和可应用性;

(2) 应用了滑动时间窗提升了风机疲劳损伤程度量化指标的快速计算, 并取得了理想的结果, 对于后续模型的实时计算具有重要作用;

(3) 应用N-M、BiLSTM等多种拟合方法描述风机运行状态与元件应力之间的非线性映射关系, 并优选出估算精度更高的拟合方法用于应力估算, 具有较强的可应用性;

(4) 在问题一和问题二所建立的数学模型的基础上, 构建问题三的有功功率优化分配模型, 进而考虑奇异值分解SVD降噪和相位差判断处理时延, 在问题四中完善问题三的优化模型, 全文各个子问题的解决方案和数学模型联系紧密、环环相扣、层层递进;

(5) 尽可能多种形式的绘图及表格来展示计算结果和对比分析, 使得模型的应用效果展示更加生动直观。

7.2. 模型缺点与改进方向

(1) 由于时间原因, 对于部分问题的回答不够细致, 结果分析的角度、展示的指标不够全面;

(2) 由于时间原因, 对部分问题中理论方法的应用过程描述不够详细具体, 大多使用流程图进行展示, 后续改进可使用更为直观的示意图或更具有普适性的数学公式进行阐述。

参考文献

- [1] 薛帆, 考虑机组疲劳载荷的风电场有功功率控制策略研究, 沈阳工业大学, 2022。
- [2] 武晓冬, 席鹏辉, 赵锟, 等, 基于疲劳分布的风电场有功优化调度策略, 科学技术与工程, 24(22):9400-9407, 2024。
- [3] 景少伟, 魏书荣, 考虑疲劳载荷分布的海上风电机组出力优化控制, 上海电力大学学报, 40(01):10-16+33, 2024。
- [4] 刘颖明, 赵哲, 王晓东, 等, 考虑机组疲劳载荷的风电集群有功功率分配方法, 太阳能学报, 42(05):430-436, 2021。
- [5] 一直流浪, 限流算法——时间滑动窗口, <https://zhuanlan.zhihu.com/p/689922080>, 2024年9月22日。
- [6] 潘沈恺, 高丙团, 毛永恒, 等, 考虑机组疲劳载荷的风电场快速有功功率分配方法, 电力系统自动化, 48(15):112-121, 2024。
- [7] 画船听暮雨, Nelder-Mead 算法 (智能优化下山单纯形法), https://blog.csdn.net/weixin_52732185/article/details/129105796, 2024年9月23日。
- [8] xiaoK, 时间序列预测: CNN-BiLSTM模型实践, <https://zhuanlan.zhihu.com/p/695255383>, 2024年9月23日。
- [9] 木木松, 非支配排序遗传算法2 (NSGA-II), https://zhuanlan.zhihu.com/p/57482087?utm_source=qq, 2024年9月23日。
- [10] Cyrus, 奇异值分解 (SVD), <https://zhuanlan.zhihu.com/p/676809666>, 2024年9月24日。
- [11] 小飞6, 相位差计算总结, <https://zhuanlan.zhihu.com/p/667442596>, 2024年9月24日。
- [12] 周坤, 陈文杰, 陈伟海, 等, 基于三次样条插值的扩展谱减语音增强算法, 北京航空航天大学学报, 49(10):2826-2834, 2023。

附录

各问题部分MATLAB代码

问题一代码: #####MATLAB#####

```
function [D,L_N]=problem1(F,t)
% F是时间×100的矩阵
% t是需要计算第t秒的损失值
% D_total是t秒的疲劳损伤值 1×100风机
% L_N是t秒的等效疲劳载荷 1×100风机
max_load = 50000000;
N_l=42565440.4361;
m=10;
C=9.77*10^70;
W=F(1:t,:);
if t==1
D(1,:)=0;
L_N(1,:) = (sum(W(t,:).^m) / N_l)^(1/m);
elseif t==2
s_amp=abs(W(2,:)-W(1,:));
s_N = (C ./ s_amp.^m);
D(1,:)=1./s_N;
L_N(1,:) = (sum(s_amp.^m) / N_l)^(1/m);
else
[D(1,:),L_N(1,:)] = cacul_damage(W, max_load, N_l, m, C);
end
end
```

```
function [D, L] = cacul_damage(x, max_load, N_life, m, C)
for q=1:100
data=x(:,q);
[c] = rainflow(data);
count=c(:,1);
cycles=c(:,2);
means=c(:,3);

% 记录载荷幅值和循环次数
n_cycles = size(cycles, 1); % 循环次数
S_amplitudes = abs(cycles(:, 1)); % 应力幅值
S_means = means; % 应力均值
S_corrected = zeros(n_cycles, 1);
for i = 1:n_cycles
S_ai = S_amplitudes(i); % 幅值
S_mi = S_means(i); % 均值
S_corrected(i) = S_ai / (1 - S_mi / max_load); % Goodman修正
end
```

```

D_total = 0;
for i = 1:n_cycles
    S_i = S_corrected(i);
    N_F = (C / S_i^m);
    n_F = count(i);
    D_F = n_F / N_F;
    D_total = D_total + D_F;
end

L_N = (sum(S_corrected.^m) / N_life)^(1/m);
D(1,q)=D_total;
L(1,q)=L_N;
end

```

问题二代码：#####MATLAB#####

```

%% 清空环境变量
warning off          % 关闭报警信息
close all           % 关闭开启的图窗
clear               % 清空变量
clc                 % 清空命令行

%% 导入数据
res = xlsread('数据集.xlsx');

R= 40; % 叶片半径(m——根据高速轴转速/pi获得
lamda1 = 2*pi*(all_omega_r/60)*R./all_V;

res=[all_pitch_angle lamda1 all_P_out_real all_T_shaft_real]; %扭矩
res=[all_pitch_angle lamda1 all_F_t_real]; %推力

res=resnew_niuju;
res=resnew_tuili;

%% 数据分析
num_size = 0.7;          % 训练集占数据集比例
outdim = 1;              % 最后一列为输出
num_samples = size(res, 1); % 样本个数
res = res(randperm(num_samples), :); % 打乱数据集
num_train_s = round(num_size * num_samples); % 训练集样本个数
f_ = size(res, 2) - outdim; % 输入特征维度

%% 划分训练集和测试集
P_train = res(1: num_train_s, 1: f_);
T_train = res(1: num_train_s, f_ + 1: end);
M = size(P_train, 2);

```

```

P_test = res(num_train_s + 1: end, 1: f_);
T_test = res(num_train_s + 1: end, f_ + 1: end);
N = size(P_test, 2);

%% 数据归一化
[P_train, ps_input] = mapminmax(P_train, 0, 1);
P_test = mapminmax('apply', P_test, ps_input);

[t_train, ps_output] = mapminmax(T_train, 0, 1);
t_test = mapminmax('apply', T_test, ps_output);

%% 数据平铺
% 将数据平铺成1维数据只是一种处理方式
% 也可以平铺成2维数据，以及3维数据，需要修改对应模型结构
% 但是应该始终和输入层数据结构保持一致
P_train = double(reshape(P_train, f_, 1, 1, M));
P_test = double(reshape(P_test, f_, 1, 1, N));

t_train = t_train';
t_test = t_test';

%% 数据格式转换
for i = 1 : M
    p_train{i, 1} = P_train(:, :, 1, i);
end

for i = 1 : N
    p_test{i, 1} = P_test(:, :, 1, i);
end

%% 建立模型
layers = [
    sequenceInputLayer(f_) % 输入层

    bilstmLayer(4, 'OutputMode', 'last') % biLSTM层
    reluLayer % Relu激活层

    fullyConnectedLayer(outdim) % 输出回归层
    regressionLayer];

%% 参数设置
options = trainingOptions('adam', ... % Adam 梯度下降算法
    'MaxEpochs', 500, ... % 最大训练次数 500
    'InitialLearnRate', 0.01, ... % 初始学习率 best_lr
    'LearnRateSchedule', 'piecewise', ... % 学习率下降
    'LearnRateDropFactor', 0.1, ... % 学习率下降因子 0.1

```

```

'LearnRateDropPeriod', 400, ...    % 经过 400 次训练后 学习率为 best_lr * 0.1
'Shuffle', 'every-epoch', ...      % 每次训练打乱数据集
'ValidationPatience', Inf, ...     % 关闭验证
'L2Regularization', 1e-4, ...      % 正则化参数
'Plots', 'training-progress', ...   % 画出曲线
'Verbose', false);

%% 训练模型
net = trainNetwork(p_train, t_train, layers, options);

%% 仿真验证
t_sim1 = predict(net, p_train);
t_sim2 = predict(net, p_test );

%% 数据反归一化
T_sim1 = mapminmax('reverse', t_sim1, ps_output);
T_sim2 = mapminmax('reverse', t_sim2, ps_output);

%% 均方根误差
rmse1 = sqrt(sum((T_sim1 - T_train).^2) ./ M);
rmse2 = sqrt(sum((T_sim2 - T_test ).^2) ./ N);

%% 查看网络结构
analyzeNetwork(net)

%% 绘图
figure
plot(1: M, T_train, 'r-*', 1: M, T_sim1, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值')
xlabel('预测样本')
ylabel('预测结果')
string = {'训练集预测结果对比'; ['RMSE=' num2str(rmse1)]};
title(string)
xlim([1, M])
grid

figure
plot(1: N, T_test, 'r-*', 1: N, T_sim2, 'b-o', 'LineWidth', 1)
legend('真实值', '预测值')
xlabel('预测样本')
ylabel('预测结果')
string = {'测试集预测结果对比'; ['RMSE=' num2str(rmse2)]};
title(string)
xlim([1, N])
grid

%% 相关指标计算
% R2

```

```

R1 = 1 - norm(T_train - T_sim1')^2 / norm(T_train - mean(T_train))^2;
R2 = 1 - norm(T_test - T_sim2')^2 / norm(T_test - mean(T_test))^2;

disp(['训练集数据的R2为: ', num2str(R1)])
disp(['测试集数据的R2为: ', num2str(R2)])

% MAE
mae1 = sum(abs(T_sim1' - T_train)) ./ M ;
mae2 = sum(abs(T_sim2' - T_test)) ./ N ;

disp(['训练集数据的MAE为: ', num2str(mae1)])
disp(['测试集数据的MAE为: ', num2str(mae2)])

% MBE
mbe1 = sum(T_sim1' - T_train) ./ M ;
mbe2 = sum(T_sim2' - T_test) ./ N ;

disp(['训练集数据的MBE为: ', num2str(mbe1)])
disp(['测试集数据的MBE为: ', num2str(mbe2)])

% MAPE
mape1 = sum(abs((T_sim1' - T_train)./T_train)) ./ M ;
mape2 = sum(abs((T_sim2' - T_test)./T_test)) ./ N ;

disp(['训练集数据的MAPE为: ', num2str(mape1)])
disp(['测试集数据的MAPE为: ', num2str(mape2)])

% MSE
mse1 = mean(((T_sim1' - T_train)./T_train).^2);
mse2 = mean(((T_sim2' - T_test)./T_test).^2);

for id=1:30
    range = [id*2000-2000+1:id*2000];
    T_s=T_test';
    [mse_T(:,id),R_T(:,id),mae_T(:,id),mape_T(:,id),rmse_T(:,id)] ...
        = LSTM_error(T_sim2(range), T_s(range));
end

%% 绘制散点图
sz = 50;

figure
scatter(T_train, T_sim1, sz, "magenta")
hold on
% plot(xlim, ylim, '--k')
xlabel('训练集真实值');
ylabel('训练集预测值');
xlim([min(T_sim1) max(T_sim1)])
ylim([min(T_sim1) max(T_sim1)])

```

```
title('训练集预测值 vs. 训练集真实值')
```

```
figure
scatter(T_test, T_sim2, sz, "blue")
hold on
% plot(xlim, ylim, '--k')
xlabel('测试集真实值');
ylabel('测试集预测值');
xlim([min(T_sim2) max(T_sim2)])
ylim([min(T_sim2) max(T_sim2)])
title('测试集预测值 vs. 测试集真实值')
```

问题三代码：#####MATLAB#####

```
clc;
clear;
close all;

nsga_optimization();

% ===== NSGA-II 优化 =====
function nsga_optimization()
    % 加载数据
    Pref = readmatrix('sheet1.csv');
    V = readmatrix('sheet2.csv');
    omega_r = readmatrix('sheet7.csv');
    omega_f = readmatrix('sheet8.csv');

    [num_turbines, num_time_steps] = size(Pref);
    P_t = sum(Pref); % 电网调度指令 (1 x 2000) 总功率需求

    % 固定参数
    Cp = 0.4;
    blade_radius = 63;
    air_density = 1.225;
    generator_efficiency = 0.9;
    gear_ratio = 100;

    figure; % 为每秒的优化过程准备图形
    vidObj = VideoWriter('optimization_output.avi'); % 动图视频保存
    open(vidObj);

    % 循环每秒进行优化
    for t = 100:120
        % 初始猜测：使用平均分配的功率作为初始值
        init_guess = repmat(P_t(t) / num_turbines, num_turbines, 1);
```



```

% 定义上下限：每个风机的功率参考值在 [0, 5] MW 之间，并且差值不超过1MW
lb = max(0, init_guess - 1e6); % 保证功率差值不超过1MW
ub = min(5 * 1e6, init_guess + 1e6); % 保证功率差值不超过1MW

% 定义目标函数
objective_func_multiobjective = @(P_ref_t) multiobjective_function(P_ref_t, Pref, t, V,
omega_r, Cp, blade_radius, air_density, generator_efficiency, gear_ratio);

% NSGA-II 参数设置
options = optimoptions('gamultiobj', ...
'Display', 'none', ...
'PopulationSize', 100, ... % 种群大小
'MaxGenerations', 100, ... % 最大代数
'ParetoFraction', 0.5, ... % 保留Pareto前沿解的比例
'CrossoverFraction', 0.8, ... % 交叉概率
'MutationFcn', @mutationadaptfeasible); % 使用自适应变异函数

% 计算优化前的目标函数值
fval_before = multiobjective_function(init_guess, Pref, t, V, omega_r, Cp, blade_radius,
air_density, generator_efficiency, gear_ratio);

% 使用 NSGA-II 进行优化
tic; % 计时器开始
[optimal_P_ref_t, fvals_after] = gamultiobj(objective_func_multiobjective, num_turbines,
[], [], ones(1, num_turbines), P_t(t), lb, ub, options);
elapsed_time = toc; % 计时器结束

% 计算优化后的目标函数值
fval_after = multiobjective_function(optimal_P_ref_t(1, :), Pref, t, V, omega_r, Cp,
blade_radius, air_density, generator_efficiency, gear_ratio);

% 保存优化后的结果
Pref(:, t) = optimal_P_ref_t(1, :);

% ==== 绘制对比图 ====
% 子图1：风机功率参考值对比
subplot(3, 1, 1);
plot(1:num_turbines, init_guess, '-o', 'DisplayName', '优化前');
hold on;
plot(1:num_turbines, Pref(:, t), '-x', 'DisplayName', '优化后');
hold off;
title(sprintf('第 %d 秒风机功率参考值', t));
xlabel('风机编号');
ylabel('功率参考值 (MW)');
legend show;
grid on;

```

```

% 子图2: 优化前后的主轴累积疲劳损伤对比
subplot(3, 1, 2);
bar([fval_before(1:num_turbines), fval_after(1:num_turbines)], 'grouped');
title('主轴累积疲劳损伤对比');
ylabel('累积疲劳损伤');
xlabel('风机编号');
legend({'优化前', '优化后'});
grid on;

% 子图3: 优化前后的推力累积疲劳损伤对比
subplot(3, 1, 3);
bar([fval_before(num_turbines+1:end), fval_after(num_turbines+1:end)], 'grouped');
title('推力累积疲劳损伤对比');
ylabel('累积疲劳损伤');
xlabel('风机编号');
legend({'优化前', '优化后'});
grid on;

% ===== 在图中添加标注 =====
fatigue_reduction_shaft = (fval_before(1) - fval_after(1)) / fval_before(1);
fatigue_reduction_thrust = (fval_before(2) - fval_after(2)) / fval_before(2);

% 仅在下方标注累积疲劳损伤的减小比例
%annotation('textbox', [0.15, 0.01, 0.85, 0.02], 'String', ...
%    sprintf('主轴疲劳损伤减小: %.2e%%, 推力疲劳损伤减小: %.2e%%', ...
%    fatigue_reduction_shaft * 100, fatigue_reduction_thrust * 100), ...
%    'EdgeColor', 'none', 'HorizontalAlignment', 'center', 'FontSize', 10);
%
% 捕获当前帧并写入视频文件
frame = getframe(gcf);
writeVideo(vidObj, frame);
end

close(vidObj);
end

% ===== 多目标函数 =====
function f = multiobjective_function(P_ref_t, Pref, t, V, omega_r, Cp, blade_radius, air_density,
generator_efficiency, gear_ratio)
% 获取风机数量
[num_turbines, ~] = size(Pref);

% 固定参数
k1 = 1; % Shaft fatigue constant
k2 = 1; % Thrust fatigue constant
C_t = 0.8; % Thrust coefficient
design_life = 42565440.4361; % 设计寿命
P_max = 5 * 10^6; % 假设每个风机的最大输出功率为 5MW

```

```

% 叶片扫过面积
swept_area = pi * blade_radius^2;

% 初始化累积疲劳损伤
total_fatigue_damage_shaft = zeros(1, num_turbines);
total_fatigue_damage_thrust = zeros(1, num_turbines);

% 计算累积疲劳损伤
for i = 1:num_turbines
    % ===== 主轴疲劳损伤的计算 =====
    % 根据参考功率和转速计算主轴扭矩
    omega_r_turbine = omega_r(i, t); % 低速轴转速
    Tshaft = P_ref_t(i) / (generator_efficiency * omega_r_turbine); % 计算主轴扭矩

    % 主轴疲劳损伤
    total_fatigue_damage_shaft(i) = k1 * (P_ref_t(i) / omega_r_turbine)^2;

    % ===== 推力疲劳损伤的计算 =====
    % 根据风速和参考功率计算推力
    V_wind = V(i, t); % 当前风机的风速
    F_thrust = C_t * 0.5 * air_density * swept_area * (V_wind^2) * (P_ref_t(i) / P_max); % 推力计算

    % 推力疲劳损伤
    total_fatigue_damage_thrust(i) = k2 * (F_thrust)^2;
end

% 返回目标函数值向量
f = [total_fatigue_damage_shaft, total_fatigue_damage_thrust];
end

clc;
clear;
close all;

% 初始化计时器和数据存储
opt_times = []; % 存储优化时间
var_reductions = []; % 存储参考功率方差的减小比例
shaft_reductions = []; % 存储主轴疲劳损伤的减小比例
thrust_reductions = []; % 存储推力疲劳损伤的减小比例
vars_before = []; % 优化前方差
vars_after = []; % 优化后方差

nsga_optimization();

```

```

% ===== NSGA-II 优化 =====
function nsga_optimization()
    % 加载数据
    Pref = readmatrix('sheet1.csv');
    V = readmatrix('sheet2.csv');
    omega_r = readmatrix('sheet7.csv');
    omega_f = readmatrix('sheet8.csv');

    [num_turbines, num_time_steps] = size(Pref);
    P_t = sum(Pref); % 电网调度指令 (1 x 2000) 总功率需求

    % 固定参数
    Cp = 0.4;
    blade_radius = 63;
    air_density = 1.225;
    generator_efficiency = 0.9;
    gear_ratio = 100;

    global opt_times var_reductions shaft_reductions thrust_reductions vars_before vars_after;

    % 循环每秒进行优化
    for t = 100:110
        % 初始猜测：使用平均分配的功率作为初始值
        init_guess = repmat(P_t(t) / num_turbines, num_turbines, 1);

        % 定义上下限：每个风机的功率参考值在 [0, 5] MW 之间，并且差值不超过1MW
        lb = max(0, init_guess - 1e6); % 保证功率差值不超过1MW
        ub = min(5 * 1e6, init_guess + 1e6); % 保证功率差值不超过1MW

        % 定义目标函数
        objective_func_multiobjective = @(P_ref_t) multiobjective_function(P_ref_t, Pref, t, V,
            omega_r, Cp, blade_radius, air_density, generator_efficiency, gear_ratio);

        % NSGA-II 参数设置
        options = optimoptions('gamultiobj', ...
            'Display', 'none', ...
            'PopulationSize', 100, ... % 种群大小
            'MaxGenerations', 100, ... % 最大代数
            'ParetoFraction', 0.5, ... % 保留Pareto前沿解的比例
            'CrossoverFraction', 0.8, ... % 交叉概率
            'MutationFcn', @mutationadaptfeasible, ...
            'MaxTime', 1); % 限制优化时间不超过1秒

        % 计算优化前的目标函数值
        fval_before = multiobjective_function(init_guess, Pref, t, V, omega_r, Cp, blade_radius,
            air_density, generator_efficiency, gear_ratio);
    end
end

```

```

% 开始计时
tic;
% 使用 NSGA-II 进行优化
[optimal_P_ref_t, fvals_after] = gamultiobj(objective_func_multiobjective, num_turbines,
[], [], ones(1, num_turbines), P_t(t), lb, ub, options);
% 结束计时并记录优化时间
elapsed_time = toc;
opt_times = [opt_times; elapsed_time];

% 计算优化后的目标函数值
fval_after = multiobjective_function(optimal_P_ref_t(1, :)', Pref, t, V, omega_r, Cp,
blade_radius, air_density, generator_efficiency, gear_ratio);

% 计算参考功率的方差
var_before = var(init_guess);
var_after = var(optimal_P_ref_t(1, :));
vars_before = [vars_before; var_before];
vars_after = [vars_after; var_after];

% 计算方差减小比例
var_reduction = (var_before - var_after) / var_before * 100;
var_reductions = [var_reductions; var_reduction];

% 计算主轴和推力疲劳损伤减小比例
shaft_reduction = (fval_before(1) - fval_after(1)) / fval_before(1) * 100;
thrust_reduction = (fval_before(2) - fval_after(2)) / fval_before(2) * 100;
shaft_reductions = [shaft_reductions; shaft_reduction];
thrust_reductions = [thrust_reductions; thrust_reduction];

% 保存优化后的结果
Pref(:, t) = optimal_P_ref_t(1, :);

% 输出优化前后的目标函数值
fprintf('第 %d 秒的优化已完成: \n', t);
fprintf('优化时间: %.2f 秒\n', elapsed_time);
fprintf('优化前参考功率方差: %.4e, 优化后参考功率方差: %.4e\n', var_before, var_after);
fprintf('方差减小比例: %.2f%%\n', var_reduction);
fprintf('主轴疲劳损伤减小比例: %.2f%%\n', shaft_reduction);
fprintf('推力疲劳损伤减小比例: %.2f%%\n\n', thrust_reduction);
end
var1=[vars_before, vars_after];
% 输出优化的时间、方差变化、疲劳损伤减小比例随时间变化的向量
fprintf('\n优化时间向量: \n');
disp(opt_times);
fprintf('优化前后参考功率方差对比: \n');
disp(var1);
fprintf('方差减小比例向量: \n');
disp(var_reductions);

```

```

fprintf('主轴疲劳损伤减小比例向量: \n');
disp(shaft_reductions);
fprintf('推力疲劳损伤减小比例向量: \n');
disp(thrust_reductions);
end

```

```

function f = multiobjective_function(P_ref_t, Pref, t, V, omega_r, Cp, blade_radius, air_density,
generator_efficiency, gear_ratio)

```

```

% 获取风机数量

```

```

[num_turbines, ~] = size(Pref);

```

```

% 固定参数

```

```

k1 = 1; % Shaft fatigue constant

```

```

k2 = 1; % Thrust fatigue constant

```

```

C_t = 0.8; % Thrust coefficient

```

```

design_life = 42565440.4361; % 设计寿命

```

```

P_max = 5 * 10^6; % 假设每个风机的最大输出功率为 5MW

```

```

% 叶片扫过面积

```

```

swept_area = pi * blade_radius^2;

```

```

% 初始化累积疲劳损伤

```

```

total_fatigue_damage_shaft = 0;

```

```

total_fatigue_damage_thrust = 0;

```

```

% 计算累积疲劳损伤

```

```

for i = 1:num_turbines

```

```

% ===== 主轴疲劳损伤的计算 =====

```

```

% 根据参考功率和转速计算主轴扭矩

```

```

omega_r_turbine = omega_r(i, t); % 低速轴转速

```

```

Tshaft = P_ref_t(i) / (generator_efficiency * omega_r_turbine); % 计算主轴扭矩

```

```

% 主轴疲劳损伤 (根据图片中的模型)

```

```

fatigue_damage_shaft = k1 * (P_ref_t(i) / omega_r_turbine)^2;

```

```

total_fatigue_damage_shaft = total_fatigue_damage_shaft + fatigue_damage_shaft;

```

```

% ===== 推力疲劳损伤的计算 =====

```

```

% 根据风速和参考功率计算推力

```

```

V_wind = V(i, t); % 当前风机的风速

```

```

F_thrust = C_t * 0.5 * air_density * swept_area * (V_wind^2) * (P_ref_t(i) / P_max); % 推
力计算, 考虑参考功率的影响

```

```

% 推力疲劳损伤 (考虑参考功率的影响)

```

```

fatigue_damage_thrust = k2 * (F_thrust)^2;

```

```

total_fatigue_damage_thrust = total_fatigue_damage_thrust + fatigue_damage_thrust;

```

```

end

```

```

% ===== 参考功率的方差

```



```

% ===== 参考功率的方差计算 =====
variance_P_ref = var(P_ref_t);

% 返回目标函数值向量，依次为：
% [主轴累积疲劳损伤，总推力累积疲劳损伤]
f = [total_fatigue_damage_shaft, total_fatigue_damage_thrust];
end

```

问题四代码：#####MATLAB#####

```

=====数据预处理代码=====
=====数据预处理代码=====

```

```

Data_WF3 = data_TS_WF3.WF_1.WT;
Data_WF2 = data_TS_WF2.WF_1.WT;

for time = 1:1:100
    k_start = 2000*(time-1)+1;
    k_end = 2000*time;
    inputs_WF3(k_start:k_end,:) = Data_WF3{1,time}.inputs;
    inputs_WF2(k_start:k_end,:) = Data_WF2{1,time}.inputs;
    outputs_WF3(k_start:k_end,:) = Data_WF3{1,time}.outputs;
    outputs_WF2(k_start:k_end,:) = Data_WF2{1,time}.outputs;
    states_WF3(k_start:k_end,:) = Data_WF3{1,time}.states;
    states_WF2(k_start:k_end,:) = Data_WF2{1,time}.states;
end

```

```

%% 数据预处理
d = 1.29; % 空气密度(kg/m3
R= 40; % 叶片半径(m——根据高速轴转速/pi获得

```

```

% 含噪声数据
v3 = inputs_WF3(:,2); % 风速
Pref3 = inputs_WF3(:,1); % pref
Ts3 = outputs_WF3(:,1); % 转矩
Ft3 = outputs_WF3(:,2); % 应力
pout3 = outputs_WF3(:,3);

```

```

% 不含噪声数据
v2 = inputs_WF2(:,2); % 风速
Pref2 = inputs_WF2(:,1); % pref
Ts2 = outputs_WF2(:,1); % 转矩
Ft2 = outputs_WF2(:,2); % 应力
pout2 = outputs_WF2(:,3);

```

```

%% EMD降噪-延迟相关性分析-预测
%% 降噪

```

```

xx=Ft; %需要降噪的信号
% xx=reshape(x,[2000,100]);
% 对数据矩阵作特征值分解;
for i=1:20
    x_s=xx((i-1)*100+1:i*100,:);
[U, S, V]=svd(Vin);
S(5:10,5:10)=0;
x_f((i-1)*100+1:i*100,:)=U*S*V';
end
Vin_new=U*S*V';
pout4=reshape(x_f,[200000,1]);
% 频谱分析 信噪比
Pxx= fft(pout3);
% figure;
% plot(20*log10(abs(Pxx)/max(abs(Pxx))), 'k');
% title('含噪声风速频谱分析')
% xlabel('频率 (Hz) ')
% ylabel('幅值 (dB) ')
sig=Pxx(1);
sigPower = sum(abs(sig(:)).^2)/length(sig(:));
sigPowerdB = 10*log10(sigPower);
%
Pxx(1)=0;
noisePower = sum(abs(Pxx(:)).^2)/(length(Pxx(:)));
noisePowerdB=10*log10(noisePower);
%
SNR=sigPowerdB-noisePowerdB;

% 画图
figure
plot(Ft(1:100,1),'r');hold on
plot(Ft_new(1:100,1),'b');
% plot(pout3(1:100,1),'g');

figure
subplot(1,2,1)
plot(v2(1:100,:), 'r');
hold on
plot(v3(1:100,:), 'b');
legend('不含噪声','含噪声')
xlabel('时间 (秒) ')
ylabel('风速 (米/秒) ')
subplot(1,2,2)
plot(v2(1:100,1), 'r');
hold on
plot(x_f(1:100,1), 'k');
legend('不含噪声','降噪后')
xlabel('时间 (秒) ')
ylabel('风速 (米/秒) ')

```

```

%% 延迟相关性
[c,lags]=xcorr(Ts2,Ts4);
figure
stem(lags,c)

%% 示例信号
% t = 0:0.01:10; % 时间向量
% signal1 = Ts2; % 信号1
% signal2 = Ts4; % 信号2 (滞后  $\pi/4$ )
%% 计算FFT
% N = length(signal1);
% Y1 = fft(signal1);
% Y2 = fft(signal2);
%
%% 计算相位差
% phase_diff = angle(Y2) - angle(Y1);
% lag_in_samples = phase_diff / (2 * pi) * N; % 转换为样本数

%%
% 假设x1和x2是两个预先定义好的时序数据向量
x1 = Ts2(6001:8000,:);
x2 = Ts4(6001:8000,:);

% 数据长度
N = length(x1);

% 傅里叶变换
X1 = fft(x1);
X2 = fft(x2);

% 计算频率
% frequencies = (-N/2:N/2-1)*(1/N);

% 计算相位
phases_x1 = angle(X1);
phases_x2 = angle(X2);

% 计算相位差
phase_diff = phases_x1 - phases_x2; % 正值表示 x1 领先 x2

% 如果需要，可以进一步分析相位差分布，比如绘制相位差的直方图
[binIndices]=h_maxphase(phase_diff);
title('Histogram of Phase Differences');
xlabel('Phase Difference (radians)');

```

```

ylabel('Frequency');
maxbin=[];
for i=1:5
    maxbin=[maxbin binIndices{end-i+1}{:,:}];
end
maxbin=sort(maxbin');

% 将相位差转换为时间滞后
% 假设采样周期为T，例如T=1如果数据是按秒采样的
for i=1:length(maxbin)
    dataindex=maxbin(i,1);
    maxbin(i,2)=(phase_diff(dataindex,:)/(2*pi*0.19206))*1;
end

for i=1:length(maxbin)
    dataindex=maxbin(i,1);
    bef=Ts2(dataindex-(round(maxbin(i,2))),:);
    maxbin(i,3)=abs(Ts2(dataindex,:)-bef)/bef;
end
mean(maxbin(:,3))

```

```

figure
plot(Ts2(144:149,:), 'r');
hold on
plot(Ts4(144:149,:), 'b');

```

```

%%
snan=s4;
snan(maxbin(:,1))=NaN;
s2=s2(1:2000,:);
s4=s4(1:2000,:);

```

```

R1 = 1 - norm(s2 - s4)^2 / norm(s2 - mean(s2))^2;
mse1 = mean(((s4 - s2)./s2).^2);
mape1 = mean(sum(abs((s4 - s2)./s2)));

```

```

R2 = 1 - norm(s2 - cleanedData)^2 / norm(s2 - mean(s2))^2;
mse2 = mean(((cleanedData - s2)./s2).^2);
mape2 = mean(sum(abs((cleanedData - s2)./s2)));

```

```

figure

```

```

subplot(1,2,1)
plot(s2,'r');hold on
plot(cleanedData,'b');
xlim([0 50]);
xlabel('时间（秒）');
ylabel('扭矩');
subplot(1,2,2)
plot(s2,'r');hold on
plot(s4,'b');
xlim([0 50]);
xlabel('时间（秒）');
ylabel('扭矩');
hold off
legend('准确值','含延迟数据');
legend('准确值','延迟优化数据');

```

```

=====主程序代码=====
=====主程序代码=====

```

```

clc
clear all
close all
global t sum_Ts3 sum_Ft3 sum_Ts2 sum_Ft2 sum_Ts1 sum_Ft1
sum_Ts=[];%不同时刻下扭矩汇总
sum_Ft=[];%不同时刻下推力汇总
X3=[];X2=[];X1=[];%记录最优功率分配值
Y3=[];Y2=[];Y1=[];%记录目标函数值
% 设置要生成的 GIF 文件名
% filename = '求解动图.gif';
for t=1:100      %时间

    disp(['当前求解时间为:    t=' num2str(t)]);

    load data
    Pref=Preff(t,:);
    %导入parato解集
    load AA
    for i=1:size(AA,1)
        y(i)=sum(f3(AA(i,:)));
    end
    % 使用 min 函数找到最小值及其索引
    [minValue, minIndex] = min(y);
    x=AA(minIndex,:);
    Y3=[Y3;f3(x)];
    X3=[X3;x];
    [Ts_out,Ft_out]=Q2_cal(x,t);%t时刻计算的扭矩、应力
    sum_Ts3=[sum_Ts3;real(Ts_out)];%结合前面时刻的扭矩
    sum_Ft3=[sum_Ft3;real(Ft_out)];%结合前面时刻的推力

```

```

%=====原始功率=====
Y1=[Y1;f1(Pref)];
X1=[X1;Pref];
[Ts_out,Ft_out]=Q2_cal(Pref,t);%t时刻计算的扭矩、应力
sum_Ts1=[sum_Ts1;real(Ts_out)];%结合前面时刻的扭矩
sum_Ft1=[sum_Ft1;real(Ft_out)];%结合前面时刻的推力

%=====均值功率=====
Y2=[Y2;f2(repelem(mean(Pref),100))];
X2=[X2;repelem(mean(Pref),100)];
[Ts_out,Ft_out]=Q2_cal(repelem(mean(Pref),100),t);%t时刻计算的扭矩、应力
sum_Ts2=[sum_Ts2;real(Ts_out)];%结合前面时刻的扭矩
sum_Ft2=[sum_Ft2;real(Ft_out)];%结合前面时刻的推力


% 绘制结果
subplot(3,1,1)
plot(1:100,repelem(mean(Pref),100),'bo', 'LineWidth', 0.5)
hold on
plot(1:100,Pref,'k*-', 'LineWidth', 0.5)
hold on
plot(1:100,x,'r^-', 'LineWidth', 0.5)
xlabel('风机编号');
ylabel('功率 (MW)');
legend('均值功率','原始功率','优化分配功率','Location','southwest')
title(['第', num2str(t), 's风机功率分配' ]);
hold off


% 绘制累积疲劳损伤
subplot(3, 1, 2);
% yyaxis right; % 设置右Y轴
plot(1:t, Y1(:,1), 'k*-', 'LineWidth', 1.5);
hold on
plot(1:t, Y2(:,1),'bo-', 'LineWidth', 1.5);
hold on
plot(1:t, Y3(:,1), 'r^-', 'LineWidth',1.5);


legend('原始功率','均值功率','优化分配功率','Location','southwest')
title('主轴累积疲劳损伤对比');
xlabel('时间 (s)');
ylabel('损伤');


% 绘制优化前后的功率分配对比
subplot(3, 1, 3);

```

```

plot(1:t, Y1(:,2),'k*-', 'LineWidth', 1.5);
hold on
plot(1:t, Y2(:,2),'bo-', 'LineWidth', 1.5);
hold on
plot(1:t, Y3(:,2), 'r^-', 'LineWidth', 1.5);

legend('原始功率','均值功率','优化分配功率','Location','southwest')
title('推力累积疲劳损伤对比');
xlabel('时间 (s)');
ylabel('损伤');

drawnow;

end

%%
function y=f3(x)

global t sum_Ts3 sum_Ft3 sum_Ts2 sum_Ft2 sum_Ts1 sum_Ft1
load data
d = 1.29; % 空气密度(kg/m3
R= 40; % 叶片半径(m——根据高速轴转速/pi获得
V=V(t,:);wr=wr(t,:);
lamda = 2*pi*(wr/60)*R./V; % 叶尖速比
% [Ts_out,Ft_out]=Q2_cal(x,t);%t时刻计算的扭矩、应力
A=pi*R^2;
Cp=x./(0.5*d*A*V.^3);
Ct=(4/3*lamda.*(1-lamda)./Cp).^0.5;
Ft_out=0.5*d*A*V.^2.*Ct;
Ts_out=x./wr;

%根据扭矩、推力计算累积疲劳值
%扭矩累积值
%推力累积值

[D_Ts,L_Ts]=Q1_cal([sum_Ts3;real(Ts_out)],t);%计算t时刻扭矩对应的疲劳损伤值D、等效
疲劳载荷L
[D_Ft,L_Ft]=Q1_cal([sum_Ft3;real(Ft_out)],t);%计算t时刻推力对应的疲劳损伤值D、等效疲
劳载荷L

%目标函数1： 主轴转矩等效疲劳载荷最小
%目标函数2： 塔架推力等效疲劳载荷最小
y(1)=sum(D_Ts);
y(2)=sum(D_Ft);
% y=sum(y);
% y=sum(Ts_out)+sum(Ft_out);

```


end

function y=f1(x)

global t sum_Ts3 sum_Ft3 sum_Ts2 sum_Ft2 sum_Ts1 sum_Ft1

load data

d = 1.29; % 空气密度(kg/m3

R= 40; % 叶片半径(m——根据高速轴转速/pi获得

V=V(t,:);wr=wr(t,:);

lamda = 2*pi*(wr/60)*R./V; % 叶尖速比

% [Ts_out,Ft_out]=Q2_cal(x,t);%t时刻计算的扭矩、应力

A=pi*R^2;

Cp=x./(0.5*d*A*V.^3);

Ct=(4/3*lamda.*(1-lamda)./Cp).^0.5;

Ft_out=0.5*d*A*V.^2.*Ct;

Ts_out=x./wr;

%根据扭矩、推力计算累积疲劳值

%扭矩累积值

%推力累积值

[D_Ts,L_Ts]=Q1_cal([sum_Ts1;real(Ts_out)],t);%计算t时刻扭矩对应的疲劳损伤值D、等效疲劳载荷L

[D_Ft,L_Ft]=Q1_cal([sum_Ft1;real(Ft_out)],t);%计算t时刻推力对应的疲劳损伤值D、等效疲劳载荷L

%目标函数1： 主轴扭矩等效疲劳载荷最小

%目标函数2： 塔架推力等效疲劳载荷最小

y(1)=sum(D_Ts);

y(2)=sum(D_Ft);

end

function y=f2(x)

global t sum_Ts3 sum_Ft3 sum_Ts2 sum_Ft2 sum_Ts1 sum_Ft1

% x=rand(1,100)*1e7;

load data

d = 1.29; % 空气密度(kg/m3

R= 40; % 叶片半径(m——根据高速轴转速/pi获得

V=V(t,:);wr=wr(t,:);

lamda = 2*pi*(wr/60)*R./V; % 叶尖速比

% [Ts_out,Ft_out]=Q2_cal(x,t);%t时刻计算的扭矩、应力

A=pi*R^2;

%使用经验公式

Cp=x./(0.5*d*A*V.^3);

Ct=(4/3*lamda.*(1-lamda)./Cp).^0.5;

Ft_out=0.5*d*A*V.^2.*Ct;

```

Ts_out=x./wr;

%根据扭矩、推力计算累积疲劳值
%扭矩累积值
%推力累积值

[D_Ts,L_Ts]=Q1_cal([sum_Ts2;real(Ts_out)],t);%计算t时刻扭矩对应的疲劳损伤值D、等效
疲劳载荷L
[D_Ft,L_Ft]=Q1_cal([sum_Ft2;real(Ft_out)],t);%计算t时刻推力对应的疲劳损伤值D、等效疲
劳载荷L

%目标函数1： 主轴转矩等效疲劳载荷最小
%目标函数2： 塔架推力等效疲劳载荷最小
y(1)=sum(D_Ts);
y(2)=sum(D_Ft);

end

```